



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Lavitra Kshitij
Madan

Compiler Design

Unit 2

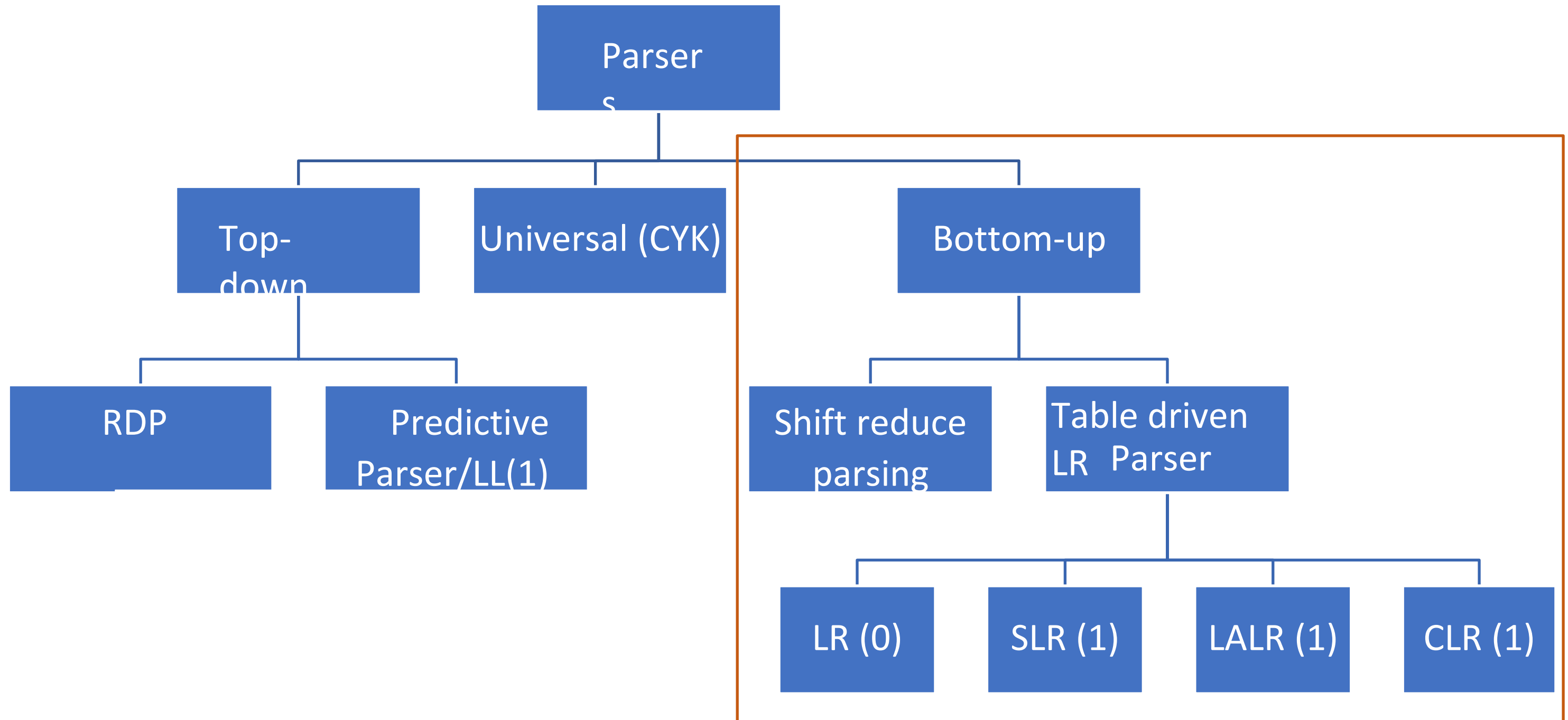
Bottom-up Parsers

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, you will learn about:

- Introduction to bottom-up parsing
- General style of bottom-up parsing: Shift/Reduce parser
- Conflicts during shift/reduce parsing with examples



Difference between top-down and bottom-up parsers

Top-down parsers have two requirements. The grammar must be:

- Left Factored
- Free of Left Recursion

It reads input from left-to-right and provides the leftmost derivation of the input string “w”.

Whereas, for the **bottom-up** parsers such treatment is not required.

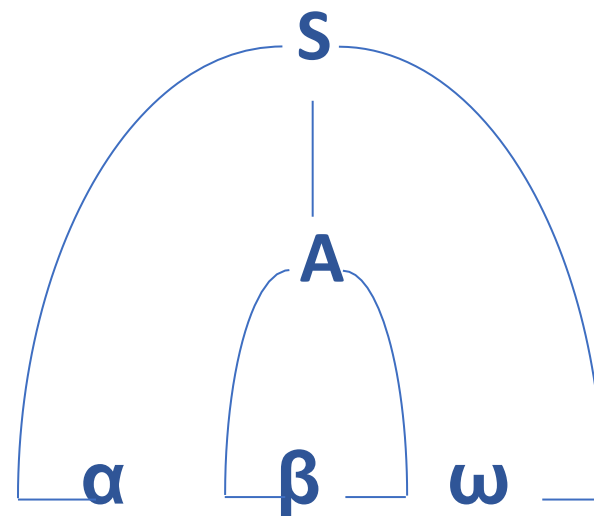
We still read the input left-to-right and the output is :

Rightmost Derivation of the string in reverse.

Some definitions

Handle : It is that substring on the stack that matches the body (RHS) of a production rule.

Handle pruning : A left-to-right, bottom-up parse works by iteratively searching for a handle, then reducing the handle to the left hand side of the production. The process is repeated until we get the Start Symbol.



For the
grammar: $S \rightarrow \alpha$
 $A \omega$
 $A \rightarrow \beta$

“ β ” is first reduced to “ A ” and then, “ $\alpha A \omega$ ” is reduced to “ S ”

Bottom-up parsing is the process of reducing a string “w” to the start symbol “S”.

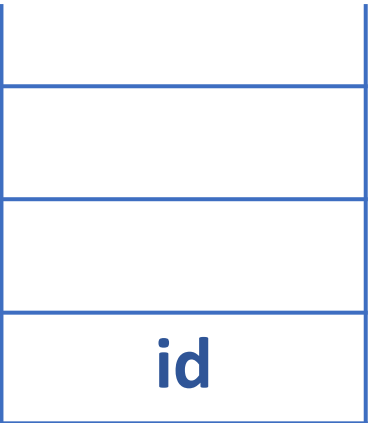
Consider the following

grammar: $E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid$

$F F \rightarrow id$

- Let the string ‘w’ be “id * id”
- The string will be parsed from left-to-right.
- Bottom-up parsers use stacks.



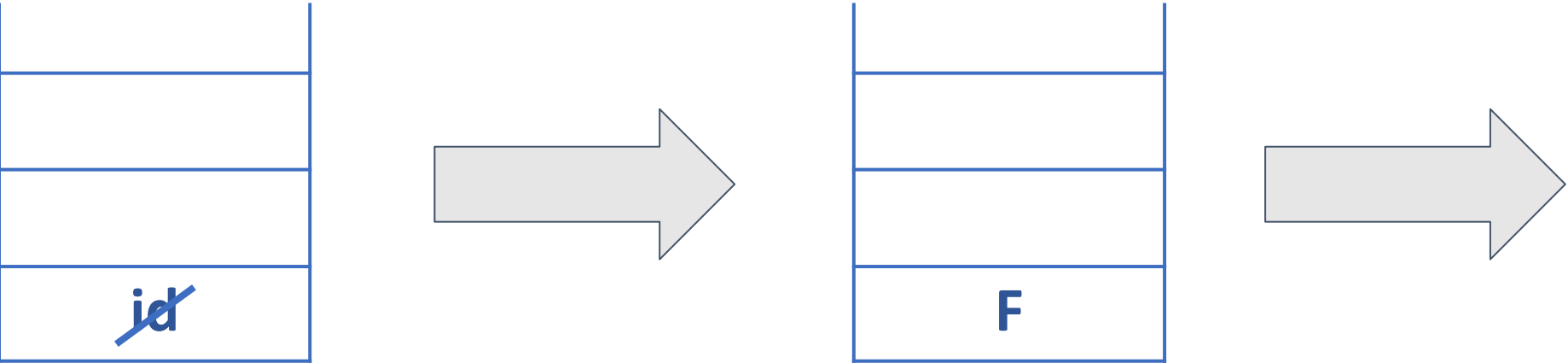
The substring (at Top-of-Stack or TOS) that right-matches the production rule is replaced by the left-hand-side of that production rule. This process is called “reduction”.

Notice that the TOS is “id” and it matches the RHS of the third production rule. We can reduce it to “F”.

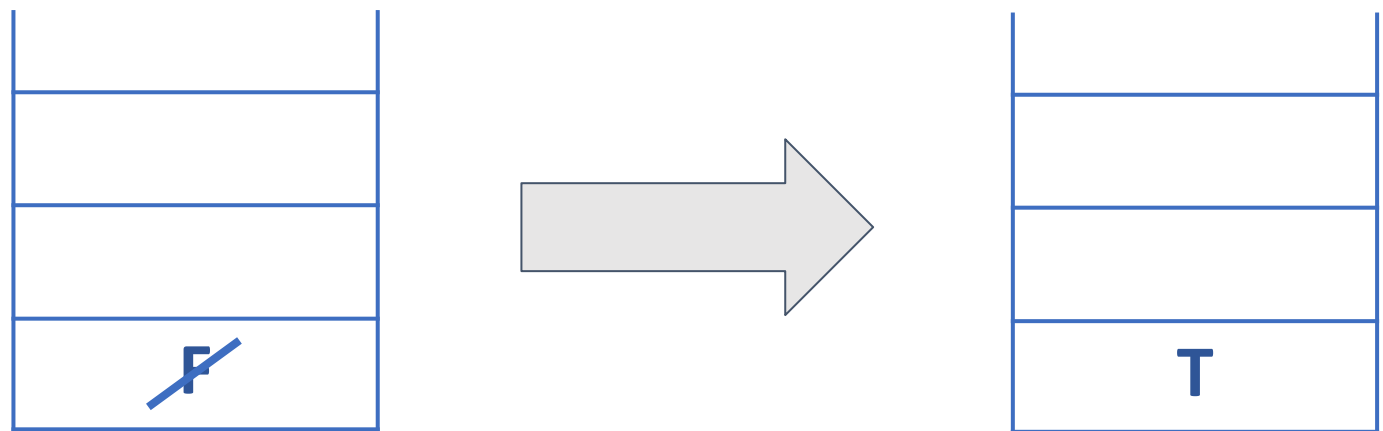
Grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid$$
$$F F \rightarrow id$$

$w = \text{“id * id”}$

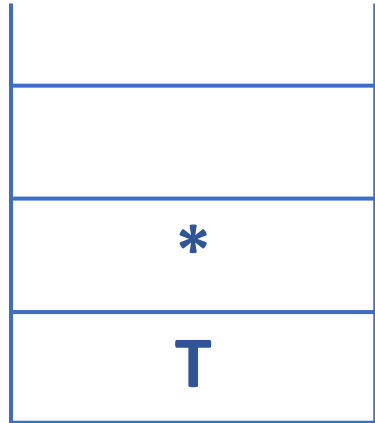


...



Similarly, we can reduce “F” to “T” using the second production rule.

Now, we should not reduce “T” to “E” using the first production rule because there is no production that has “E *” on its RHS. But, what we can do is that we can **shift** the next symbol “*” from the input onto the stack.



This step has a “conflict” since we had the choice to either shift or reduce. Conflicts may arise in ambiguous grammar. The less powerful compilers do not know what to do when conflicts arise

Grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

$w = "id * id"$

Bottom-up parsing example (continued)

id
*
T

Since we do not have a handle at TOS, we can shift another symbol onto the stack.

The TOS now has “id” which can be reduced to “F” using the third production rule.

Grammar:

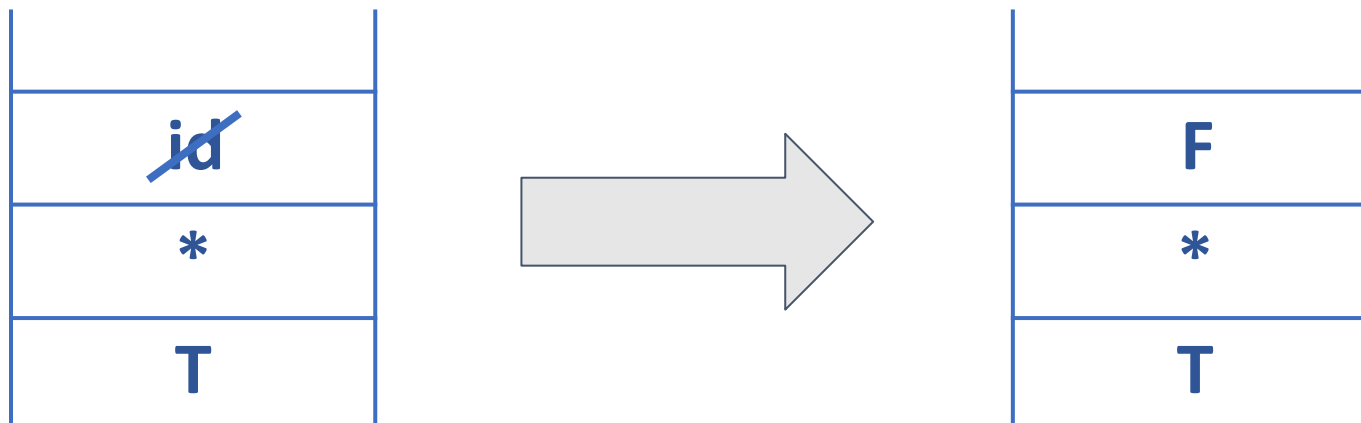
$E \rightarrow E + T \mid$

T

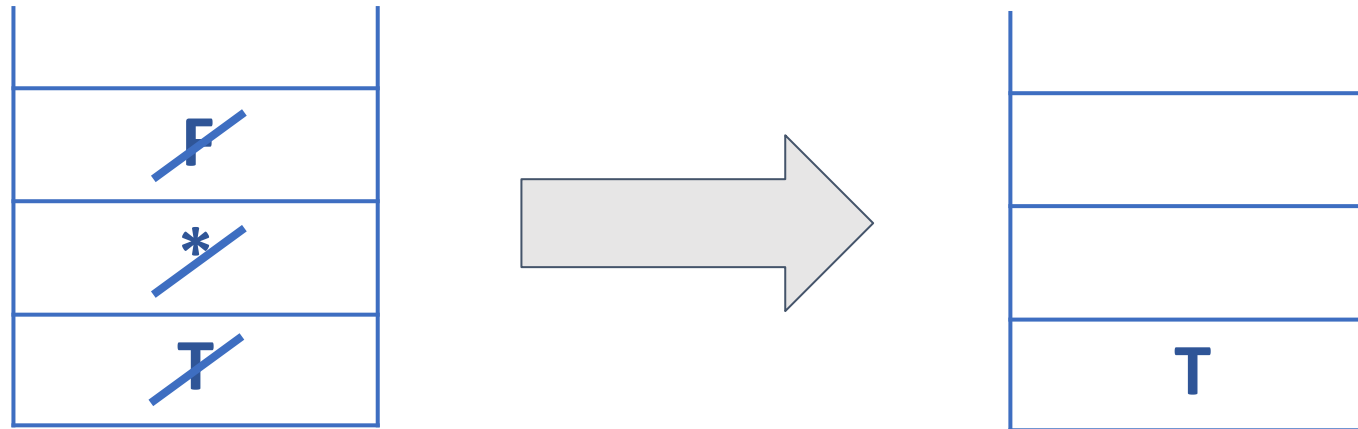
$T \rightarrow T * F \mid$

$F F \rightarrow id$

$w = \text{“id * id”}$

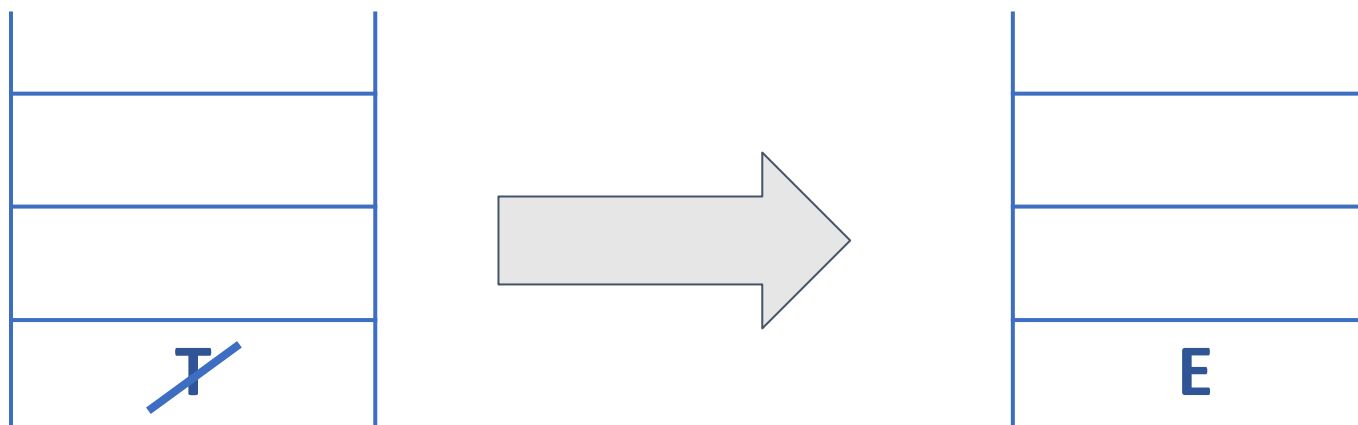


Bottom-up parsing example (continued)



Now, “ $T * F$ ” perfectly matches the RHS of the second production rule and so it can be reduced to just “ T ”.

Finally, the TOS now has only “ T ” which is the RHS of the first production rule and can be reduced to “ E ” using the same. We have reached the start symbol “ E ” and so, the string ‘ w ’ was successfully parsed using the given grammar.



Grammar:

$$E \rightarrow E + T \mid$$
$$T$$
$$T \rightarrow T * F \mid$$
$$F F \rightarrow \text{id}$$
$$w = \text{"id * id"}$$

Bottom-up parsing example (continued)

Summary of the
example problem:

id * id

F * id

T * id

T * F

T

E

Rightmost derivation
in reverse order:

$$\begin{aligned} E &\Rightarrow T \\ &\sim T * F \\ &\sim T * id \\ &\sim F * id \\ &\sim id * id \end{aligned}$$

Grammar:

$$E \rightarrow E + T \mid$$
$$T$$
$$T \rightarrow T * F \mid$$
$$F F \rightarrow id$$

w = "id * id"

The output of a Bottom-up parser, if $w \in L(G)$ is the rightmost derivation of the string but in **reverse** order.

General style of bottom-up parsing: Shift/Reduce parser

Shift/Reduce Parsing

General style of bottom-up parsing: Shift/Reduce parser

Bottom-up parsing is the process of reducing a string “w” to the start symbol “S”.

We will now use a table to solve the problem step-by-step. The table consists of three columns:

- Stack (status)
- Input buffer (status)
- Action (Shift, reduce, error, accept)

Note: Whatever occurs on the stack is known as **viable prefix**. The parser will never let you go past your handle. This will be covered in detail in the following slides.

Solution explained using a table (continued)

Consider the same grammar and input example. The initial configuration is as follows:

Stack	I/p buffer	Action
\$	id * id \$	

It consists of an empty stack and a full input buffer.

We first shift “id” onto the stack.

Stack	I/p buffer	Action
\$	id * id \$	shift id
\$ id	* id \$	

Grammar:
E → E + T | T
T → T * F | F
F → id

w = “id * id”

Solution explained using a table (continued)

At TOS, “id” can be reduced to

“F” Stack	I/p buffer	Action
\$	id * id \$	shift id
\$ id	* id \$	Reduce using (iii)
\$ F	* id \$	

“F” is further reduced to

“T” Stack	I/p buffer	Action
\$	id * id \$	shift id
\$ id	* id \$	Reduce using (iii)
\$ F	* id \$	Reduce using (ii)
\$ T	* id \$	

Grammar:

- (i) $E \rightarrow E + T \mid T$
- (ii) $T \rightarrow T * F \mid F$
- (iii) $F \rightarrow id$

w = “id * id”

Solution explained using a table (continued)

Since shifting “*” was preferred over reducing “T” to “E”, “*” is now shifted onto the stack.

Stack	I/p buffer	Action
\$	id * id \$	shift id
\$ id	* id \$	Reduce using (iii)
\$ F	* id \$	Reduce using (ii)
\$ T	* id \$	Shift *
\$ T *	id \$	

Grammar:

(i) $E \rightarrow E + T \mid T$

(ii) $T \rightarrow T * F \mid F$

(iii) $F \rightarrow id$

w = “id * id”

Solution explained using a table (continued)

Since TOS does not form the RHS of any rule, we now shift the last symbol “id” onto the stack.

Stack	I/p buffer	Action
\$	id * id \$	Shift id
\$ id	* id \$	Reduce using (iii)
\$ F	* id \$	Reduce using (ii)
\$ T	* id \$	Shift *
\$ T *	id \$	Shift id
\$ T * id	\$	

Grammar:

- (i) $E \rightarrow E + T \mid T$
- (ii) $T \rightarrow T * F \mid F$
- (iii) $F \rightarrow id$

w = “id * id”

Solution explained using a table (continued)

Using the third production rule, “id” is reduced to “F”.

Stack	I/p buffer	Action
\$	id * id \$	Shift id
\$ id	* id \$	Reduce using (iii)
\$ F	* id \$	Reduce using (ii)
\$ T	* id \$	Shift *
\$ T *	id \$	Shift id
\$ T * id	\$	Reduce using (iii)
\$ T * F	\$	

Grammar:

- (i) $E \rightarrow E + T \mid T$
- (ii) $T \rightarrow T * F \mid F$
- (iii) $F \rightarrow id$

w = “id * id”

Solution explained using a table (continued)

Using the second production rule, “T * F” is reduced to “T”.

Stack	I/p buffer	Action
\$	id * id \$	Shift id
\$ id	* id \$	Reduce using (iii)
\$ F	* id \$	Reduce using (ii)
\$ T	* id \$	Shift *
\$ T *	id \$	Shift id
\$ T * id	\$	Reduce using (iii)
\$ T * F	\$	Reduce using (ii)
\$ T	\$	

Grammar:

(i)

$E \rightarrow E + T \mid T$

(ii)

$T \rightarrow T * F \mid F$

(iii)

$F \rightarrow id$

$w = \text{“id * id”}$

Solution explained using a table (continued)

Finally, using the first production rule, “T” is reduced to “E”.

Stack	I/p buffer	Action
\$	id * id \$	Shift id
\$ id	* id \$	Reduce using (iii)
\$ F	* id \$	Reduce using (ii)
\$ T	* id \$	Shift *
\$ T *	id \$	Shift id
\$ T * id	\$	Reduce using (iii)
\$ T * F	\$	Reduce using (ii)
\$ T	\$	Reduce using (i)
\$ E	\$	

Grammar:

- (i) $E \rightarrow E + T \mid T$
- (ii) $T \rightarrow T * F \mid F$
- (iii) $F \rightarrow id$

w = “id * id”

Solution explained using a table (continued)

We have reached the start symbol and can accept the string.

Stack	I/p buffer	Action
\$	id * id \$	Shift id
\$ id	* id \$	Reduce using (iii)
\$ F	* id \$	Reduce using (ii)
\$ T	* id \$	Shift *
\$ T *	id \$	Shift id
\$ T * id	\$	Reduce using (iii)
\$ T * F	\$	Reduce using (ii)
\$ T	\$	Reduce using (i)
\$ E	\$	Accept

Grammar:

- (i) $E \rightarrow E + T \mid T$
- (ii) $T \rightarrow T * F \mid F$
- (iii) $F \rightarrow id$

w = "id * id"

Conflicts during shift/reduce Parsing

Conflicts during shift/reduce parsing

In the example discussed earlier, we had encountered a situation where we could either reduce using a production rule or shift a new symbol onto the stack. It was an example of a **conflict**.

There are two types of conflicts:

1. Shift-reduce conflict (s/r)
2. Reduce-reduce conflict (r/r)

A **shift-reduce conflict** occurs when there are two production rules such that the top-of-stack exactly matches the right-hand-side of one rule and is also a prefix of the right-hand-side of the other rule.

In this situation, we can either reduce the handle to the left-hand-side of the first production rule or we can read another symbol from the input buffer and push it onto the stack.

Example: The dangling-else problem.

Dangling-else problem (Shift-reduce conflict)

Consider the following grammar:

$S \rightarrow \text{if expr then stmt}$
 $\rightarrow \text{if expr then stmt else stmt}$
 $\rightarrow \text{other}$

If the status of the stack is:

$\$ \dots \text{if expr then stmt}$

And, the status of the input buffer is:

$\text{else} \dots \$$

We can either reduce the TOS to “S” or can push “else” onto the stack and try to match the second production rule.

Dangling-else problem (Shift-reduce conflict)

Consider the following

grammar: $S \rightarrow \text{if expr then stmt}$

$\rightarrow \text{if expr then stmt else stmt}$

$\rightarrow \text{other}$

If the status of the stack
is:

\$... if expr then stmt

And, the status of the input buffer is:

else ... \$

We can either reduce the TOS to “S” or can push “else” onto the stack and try to match the second production rule.

The yacc tool resolves this conflict by choosing the shift operation over the reduce operation and gives a warning!

Dangling-else problem (Shift-reduce conflict)

Pseudo-code in C of an example of this problem:

```
1.  int a = 10, b = 20, c = 30;
2.  if (a>b)
3.      if (a>c)
4.          printf("%d", a);
5.      else if (c>=a)
6.          printf("%d", c);
7.  else if (b>c)
8.      printf("%d", b);
9.  else
10.     printf("%d", c);
```

The else if on line 7 will be considered as a part of the inner if statement because the compiler shifts it rather than reducing lines 2 to 6 together.

A **reduce-reduce conflict** occurs when there are two production rules such that the top-of-stack exactly matches the right-hand-side of both the rules.

In this situation, we can reduce the handle to the left-hand-side of any of the two production rules.

The compiler will not know which of the two production rules to choose for reduction in such an ambiguous case.

The yacc tool resolves this conflict by simply choosing the first rule and gives a warning!

Example of reduce-reduce conflict

Consider the following

grammar:

stmt	→ id
parameter_list	(parameter_list)
parameter	parameter
parameter	parameter
expr	→ id
Expr_list	→ id id(expr_list)
	→ expr expr_list, expr

If the status of the stack
is:

\$... id (**id**

And, the status of the input buffer
is:

, id) ... \$

We can reduce the TOS (“id”) to “expr” or to “parameter”.
Both are valid!

Func (5)
or
Func (7)

How to
differentiate
between a
procedure
and an
expression



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Philip
Joseph

Compiler Design

Unit 2

Bottom-Up Parsing

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, we will learn about:

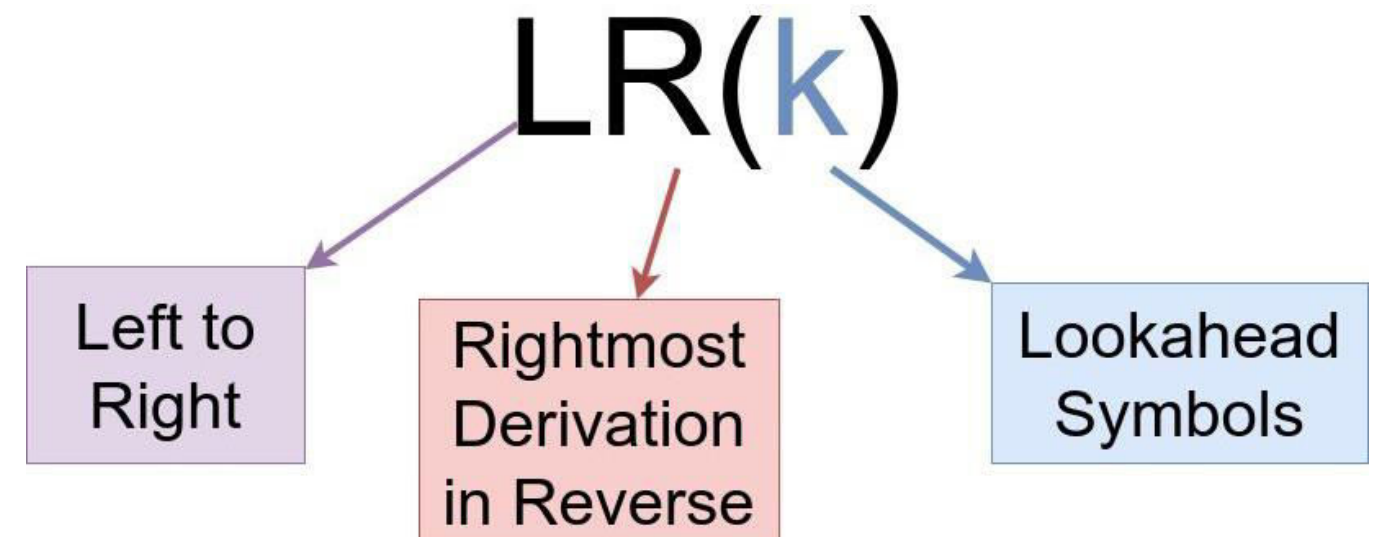
- Types of LR Parsers
- LR(0) Item
- Model of an LR Parser
- Construction of LR(0) Parser with example

Introduction to Table-driven LR Parsers

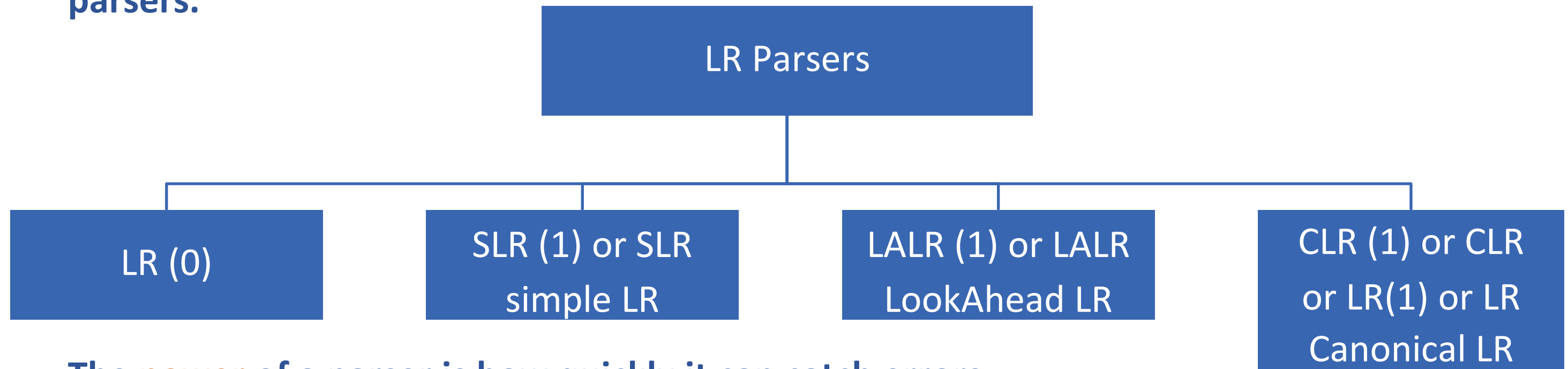
- A class of grammar for which we can build shift-reduce parser (BUPs) is called **LR grammars**.
- LR parsing in most general cases is non-backtracking and shift-reduce parsing.
- **LL(1)-Grammar $\subset$ LR(1)-Grammar**

Introduction to Table-driven LR Parsers

- In general, it is LR(k) and is a set of table-driven parsers, where the first **L** indicates that input is parsed from left to right.
 - The **R** indicates that the output of a bottom-up parser is the **Rightmost derivation** of the input string in **Reverse**.
 - **k** denotes the number of symbols used for lookahead.
 - 'k' is always greater than or equal to 1.
 - The default value of 'k' is 1.



There are 4 types of LR parsers.



The **power** of a parser is how quickly it can catch errors.

The power of these parsers increase as we move from left-to-right in the order given above.

The number of states required by the 4 types of parsers is-

$$(n_{LR(0)} = n_{SLR} = n_{LALR}) \leq n_{CLR}$$

- All the four types of LR parsers use DFA (Deterministic Finite Automaton)
- The DFA of LR(0) and SLR(1) parsers is known as **LR(0) automata**. This is because it makes use of **LR(0) items**.
- The DFA of LALR and CLR parsers is known as **LR(1) automata** because it uses **LR(1) items**.

Searching for handle:

When using a shift/reduce parser, we must decide whether to shift or reduce at each point.

- We only want to reduce when we know we have a handle.
- Question: How can we tell that we might be looking at a handle?
- An **item** is a production with a dot in right-hand-side of the production. Example : $A \rightarrow XY.Z$
- “.” indicates where we are in this production.
- When . is at the end of the production, it is called a **final item** or a **kernel item** and indicates that the handle is on TOS and we can perform reduction.

For the example used in the previous slide, the possible items are:

$A \rightarrow .XYZ$

$A \rightarrow X.YZ$

$A \rightarrow XY.Z$

$A \rightarrow XYZ.$

“ $A \rightarrow X.YZ$ ” means that X is seen (possibly on stack) and Y, Z are expected to be seen next.

“ $A \rightarrow XYZ.$ ” means that XYZ is seen. The entire right-hand-side of the production is on the stack and so it can be reduced.

At any instant in time, the contents of the left side of the parser can be described using the following process:

- Trace out, from the start symbol, the series of productions that have not yet been completed and where we are in each production.
- For each production, in order, output all of the symbols up to the point where we change from one production to the next.

Idea: At each point, track

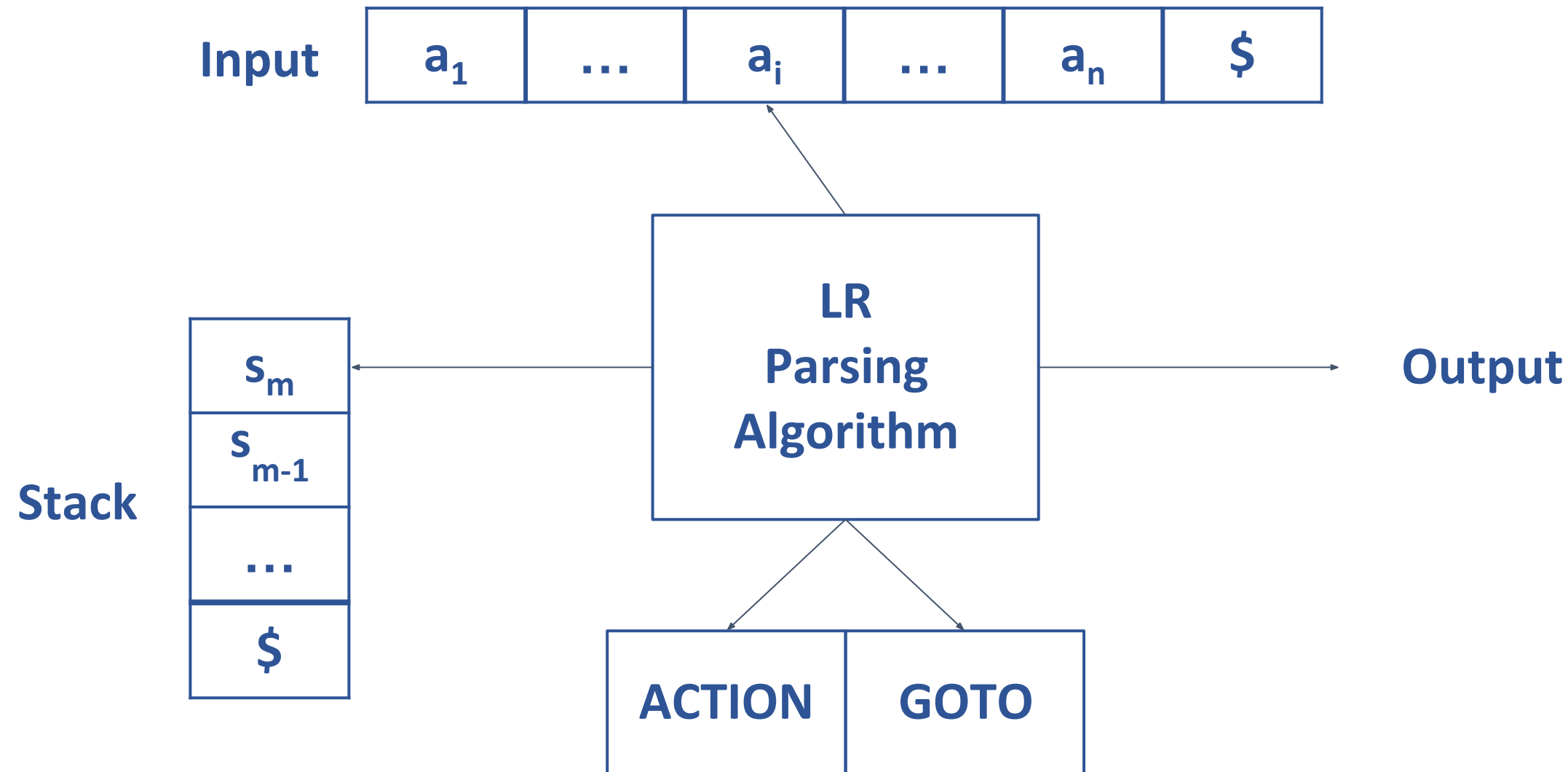
- **Which production we are in, and**
- **Where we are in that production.**

At each point, we can do one of two things:

- **Match the next symbol of the candidate left-hand side with the next symbol in the current production, or**
- **If the next symbol of the candidate left-hand side is a nonterminal, nondeterministically guess which production to try next.**

An Important Result:

- There are only finitely many productions, and within those productions only finitely many positions.
- At any point in time, we only need to track where we are in one production.
- There are only finitely many options we can take at any one point.
- We can use a finite automaton as our recognizer.



The “ACTION” and “GOTO” parts make up the **LR Parsing Table**.

The structure of the LR Parsing Table and the LR Parsing Program is the same for all the LR Parsers.

But, different parsers differ in the content of the table.

More about the “ACTION” part:

- It is like a guide about when to shift or when to reduce.
- Any conflicts (shift/reduce or reduce/reduce) will arise here.

Steps for constructing the LR automaton

1. Augment the grammar
2. Number your productions in the original grammar
3. Follow the LR(0) Parsing Algorithm
4. Create the parsing table

Steps for constructing the LR automaton

Let us try to understand the process by solving a problem step-by-step.

Examine whether the following grammar is in LR(0) or not.

$$S \rightarrow A A$$
$$A \rightarrow a A | b$$

We have to make an LR(0) automata (DFA)

There are two functions that will help us create the DFA

- Closure

- Goto

Possible items are:

$$S \rightarrow . A A$$
$$S \rightarrow A . A$$
$$S \rightarrow A A .$$

Augment the grammar

Augmenting the grammar involves introduction of a new start symbol. This is done to make sure that only the start symbol is on the stack when the input buffer has \$ to accept the string

$$\begin{array}{lll} S' & \rightarrow & S \\ S & \rightarrow & A A \\ A & \rightarrow & a A \\ A & \rightarrow & b \end{array}$$

Example problem solution (continued)

Number your productions in the original grammar

$S' \rightarrow S$

1. $S \rightarrow A A$

2. $A \rightarrow a A$

3. $A \rightarrow b$

We will start with the initial item $\Rightarrow S' \rightarrow . S$

Example problem solution (continued)

Follow the LR(0) Parsing

Algorithm

1. Start with the initial item.
2. If there is a non-terminal immediately after the “.” in any of the items, add all the productions from that non-terminal.
3. For every newly added item, repeat step 2.
4. If no more items can be added, it becomes a closure/state.
5. Use the goto function to define a transition for each symbol after the dot.
6. Repeat the process until no new states can be added to the automaton.

Example problem solution (continued)

Follow the LR(0) Parsing

Algorithm Step 1: Starting

Starting with the initial item.

$$S' \rightarrow .S$$

Step 2: Adding productions of NTs after the dot

Since “S” follows the dot, we put the first production.

$$.S$$

$$S \rightarrow .A A$$

Production rules:

$$\begin{array}{lcl} S' & \rightarrow & S \\ 1. & \rightarrow & A A \\ & & S \\ & \rightarrow & a A \end{array}$$

Example problem solution (continued)

Follow the LR(0) Parsing Algorithm

Step 3: Repeating step 2 until no more items

Since “A” follows the dot, we put productions 2 and

- 3.
- $S' \rightarrow .S$
 - $S \rightarrow .A A$
 - $A \rightarrow a A$
 - $A \rightarrow b$

Step 4: Making closure

Since no more items can be added to the state, we can now make it a closure. It is done by adding the productions in a box and naming it.

State		
<u>10</u>		
S'	$\rightarrow$	$.S$
S	$\rightarrow$	$.A A$
A	$\rightarrow$	$.a A$

Production rules:

- $S' \rightarrow S$
- 1. $\rightarrow A A$
- $S \rightarrow a A$

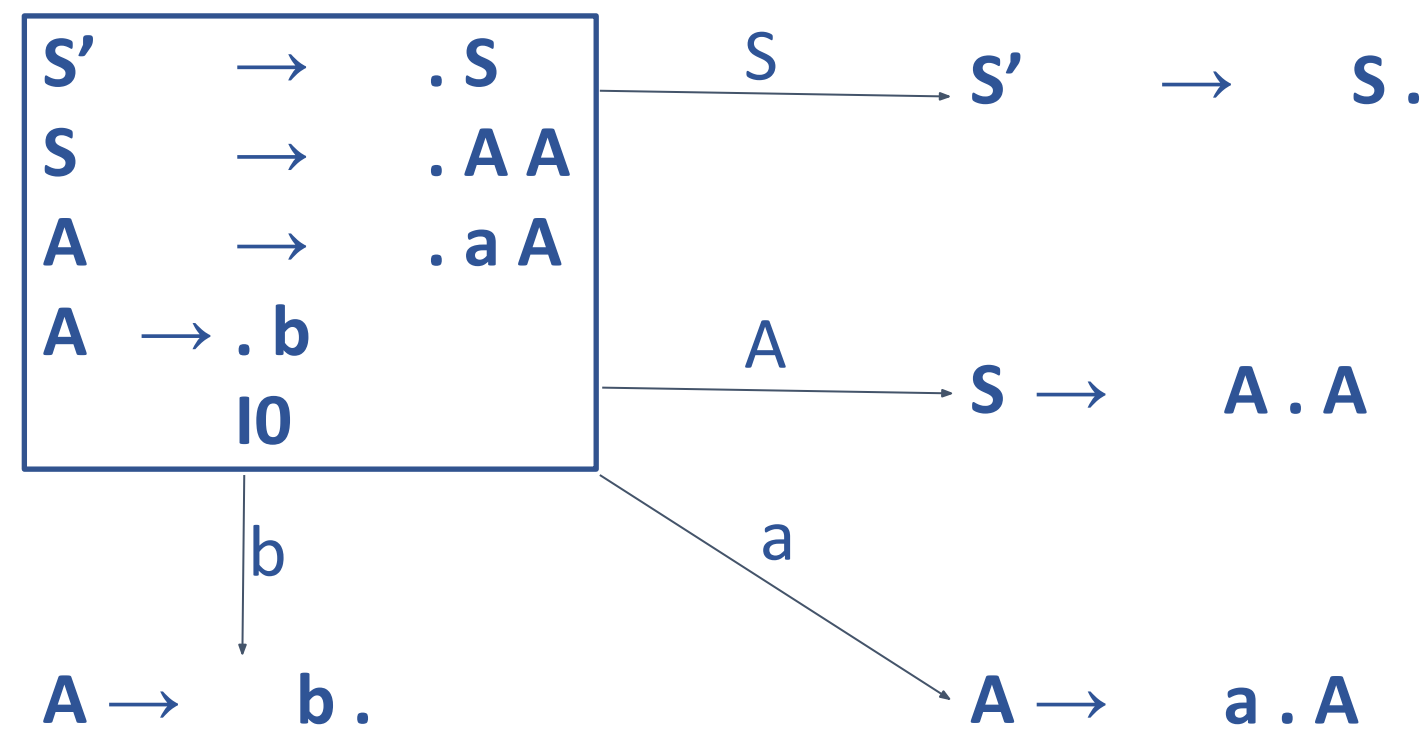
Example problem solution (continued)

Follow the LR(0) Parsing Algorithm

Step 5: Transitions using the goto function

Goto (I, X) $\Rightarrow$ Define transition from state 'I' on every symbol 'X' after the dot.

We will have transitions on "S", "A", "a" and "b".



Production rules:

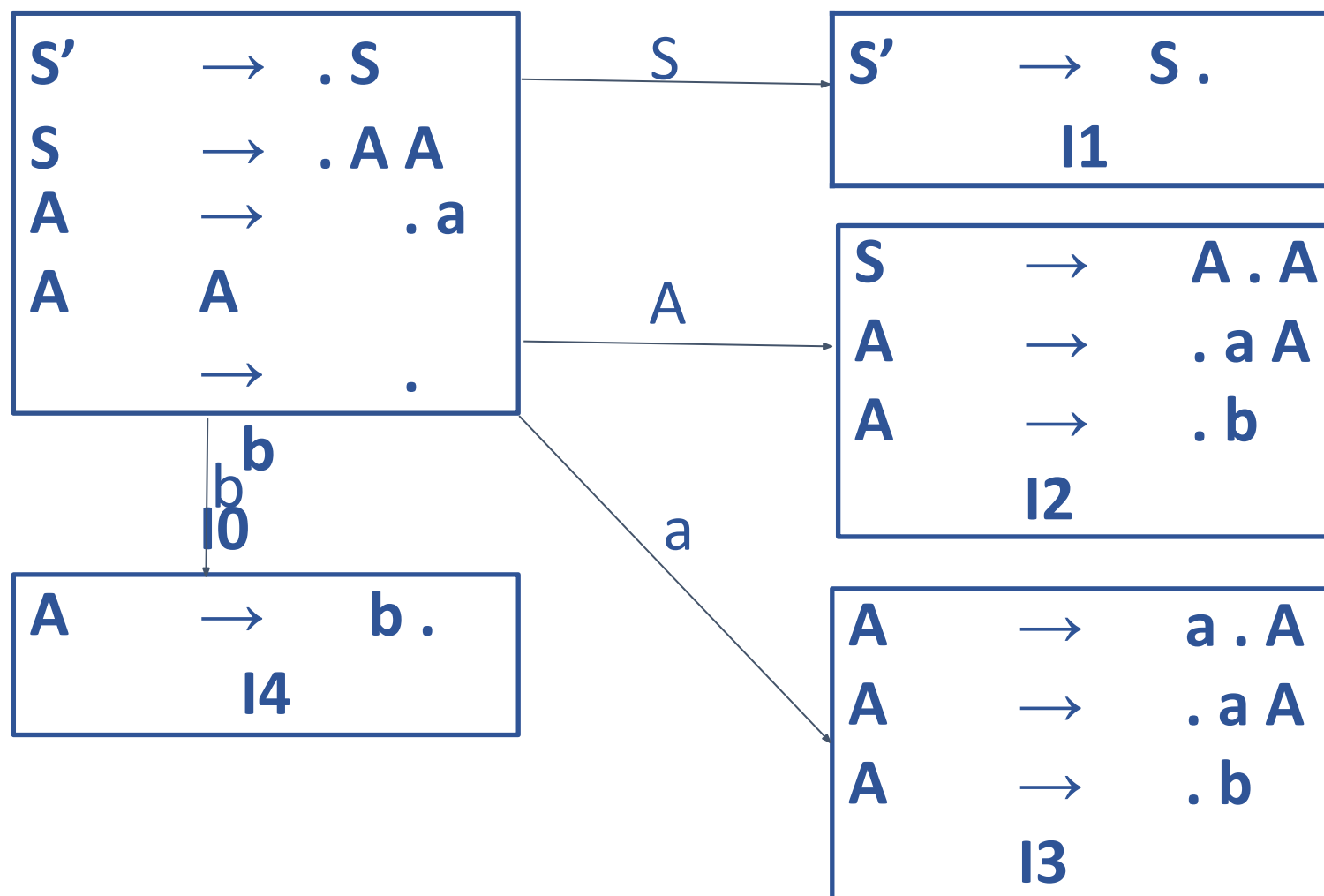
- $S' \rightarrow S$
- 1. $S \rightarrow A A$
- 2. $A \rightarrow a A$
- 3. $A \rightarrow b$

Example problem solution (continued)

Follow the LR(0) Parsing Algorithm

Step 6: Repeat steps 2-5 until no more states can be added

$S' \rightarrow S.$ and $A \rightarrow b.$ are already completed and closure can be used on them. The other two transitions require steps 2-5.

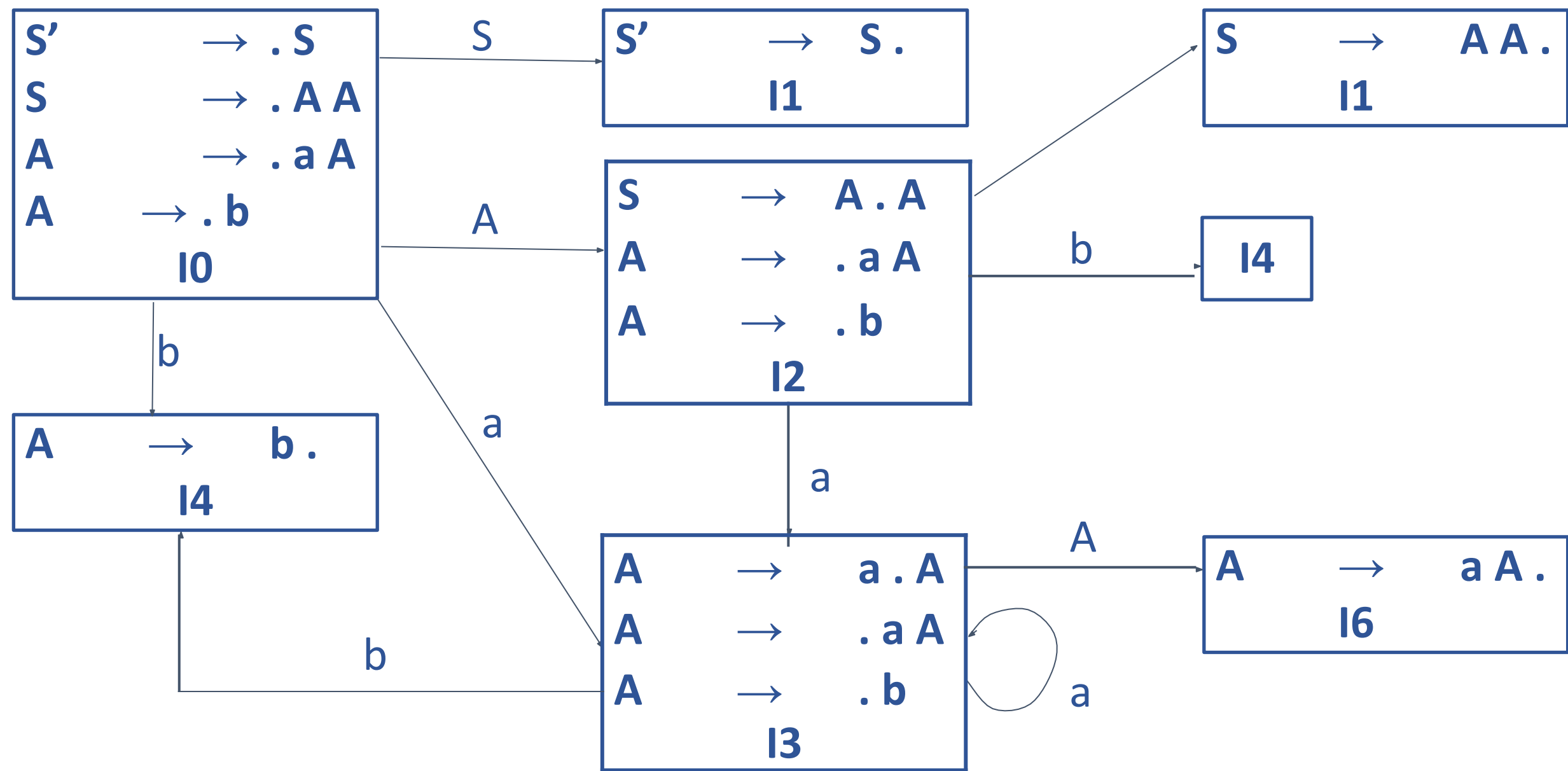


Production rules:

- $S' \rightarrow S$
- 1. $S \rightarrow AA$
- 2. $A \rightarrow aA$
- 3. $A \rightarrow b$

Example problem solution (continued)

Follow the LR(0) Parsing Algorithm (Step 6)



- The canonical collection of LR(0) items is represented by $C = \{ I_0, I_1, I_2, I_3, I_4, I_5, I_6 \}$
- Notice the transition from state I_2 to state I_4 . It can be made to directly point to the box that actually contains all the items that correspond to the state I_4 , just like how the transition from state I_2 to I_3 and from state I_3 to I_4 is, but it will make the automata look very cluttered.
- $S' \rightarrow S.$
The aforementioned item is known as the **special final item**. It indicates that the start symbol is on the stack and thus the given grammar can be accepted by the parser.

The LR(0) parsing table consists of three parts-

- State
- Action
- Goto

The state numbers from the automaton diagram are used to fill the **'state'** column of the parsing table.

The **'action'** column has sub-columns for each terminal used in the grammar and also for "\$" symbol.

The **'goto'** column has sub-columns for each non-terminal used in the grammar.

Example problem solution (continued)

State	Action			Goto	
	a	b	\$	S	A
0					

We will start filling the LR Parsing Table now.

For the state I0, transition over “S” leads to state I1 and so, we fill the ‘S’ sub-column in the ‘goto’ column as “1”. Similarly, the sub-column ‘A’ gets a value “2”.

Transition on a symbol is called **shift**. It is denoted by “ s_x ”, where X is the next state.

When a state has a final item, we must **reduce** it. It is denoted by “ r_x ”, where X is the next state.

Example problem solution (continued)

State	Action			Goto	
	a	b	\$	S	A
0	s ₃	s ₄		1	2

State 1 contains the special final item. “Accept” should be put under the ‘\$’ sub-column in the ‘Action’ column.

State	Action			Goto	
	a	b	\$	S	A
0	s ₃	s ₄		1	2
1			Accept		

Example problem solution (continued)

Similarly filling for states I2 and I3.

State	Action			Goto	
	a	b	\$	S	A
0	s ₃	s ₄		1	2
1			Accept		
2	s ₃	s ₄			5
3	s ₃	s ₄			6

States I4, I5 and I6 have a final item each and so, we must reduce them. In LR(0) we place “r” in the **entire action column** for that state and this is a disadvantage of this parser(as it does not perform any lookahead). The “X” subscript takes the value of the rule number that the production rule corresponds to.

Remember that we had numbered all the productions as part of the second step of the

Example problem solution (continued)

State	Action			Goto	
	a	b	\$	S	A
0	s ₃	s ₄		1	2
1			Accept		
2	s ₃	s ₄			5
3	s ₃	s ₄			6
4	r ₃	r ₃	r ₃		
5	r ₁	r ₁	r ₁		
6	r ₂	r ₂	r ₂		

Note that if we have multiple entries in a cell of the parsing table, the grammar is not in LR(0).

In our example grammar, since we do not have multiple entries, we say that the given grammar:

$$S \rightarrow A A$$
$$A \rightarrow a A \mid b \text{ is in LR(0).}$$

This concludes the solution of the example problem.

In the next class, we will see how to parse a string using the table we have developed!



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Lavitra Kshitij
Madan

Compiler Design

Unit 2

Table-driven BUP

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, we will learn about:

- LR(0) parsers (example problems)
- Disadvantages of LR(0) parsers

Recall the example from our previous lecture.

We had successfully created an LR(0) automaton and
LR(0) parsing table for the following grammar:

$$S \rightarrow A A$$
$$A \rightarrow a A \quad | \quad b$$

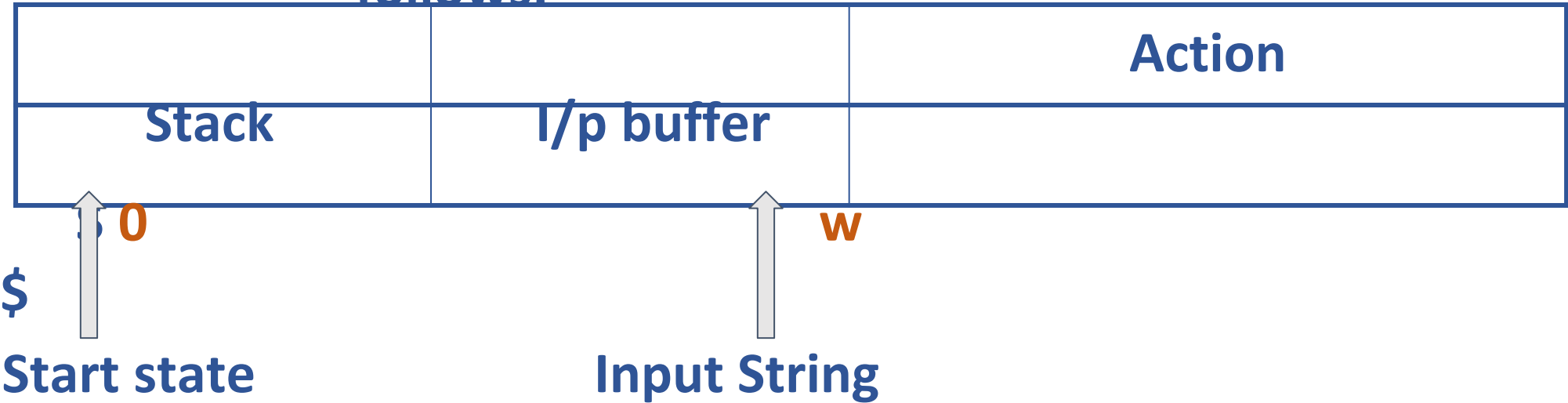
Here is the parsing table for reference.

State	Action			Goto	
	a	b	\$	S	A
0	s ₃	s ₄		1	2
1			Accept		
2	s ₃	s ₄			5
3	s ₃	s ₄			6
4	r ₃	r ₃	r ₃		
5	r ₁	r ₁	r ₁		
6	r ₂	r ₂	r ₂		

Parsing a string using LR(0)

Let us now see the moves made by LR(0) parser on an example string “abb”.

The initial configuration is as follows:



We stop parsing when,for an action, we get the output as “Accept” according to the parsing table.

Parsing a string using LR(0)

Look at the top of the stack and the next symbol in the input buffer to be read. Find the corresponding cell in the parsing table and perform that action.

Stack	I/p buffer	Action
\$ 0	abb \$	$A[0, a] = s_3$ Push 'a' on stack Push 3 on stack
\$ 0a3	bb \$	

Now continue
similarly.

Stack	I/p buffer	Action
\$ 0	abb \$	$A[0, a] = s_3$ Push 'a' on stack Push 3 on stack
\$ 0a3	bb \$	$A[3, b] = s_4$ Push 'b' on stack Push 4 on stack
\$ 0a3b4	b \$	

Stack	I/p buffer	Action
\$ 0	abb \$	$A[0, a] = s_3$ Push 'a' on stack Push 3 on stack
\$ 0a3	bb \$	$A[3, b] = s_4$ Push 'b' on stack Push 4 on stack
\$ 0a3b4	b \$	$A[4, b] = r_3$ {Reduce previous 'b' using 3rd rule} Pop 4 from stack Pop 'b' from stack Push 'A' on stack {TOS must be a state number and so, use Goto function}

If size of RHS of reducing production is x , then pop $2x$ symbols off the stack.

Hence, in our case, $A \rightarrow b$
Size of RHS = $|b| = 1$
Therefore, we pop 2 symbols off the stack and push the LHS of the

Parsing a string using LR(0)

Stack	I/p buffer	Action
:	:	:
\$ 0a3A6	b \$	$A[6, b] = r_2 \quad (A \rightarrow a A)$ Pop 6, 'A', 3, 'a' from stack Push 'A' on stack Push Goto[0,A] = 2
\$ 0A2	b \$	

Parsing a string using LR(0)

Stack	I/p buffer	Action
:	:	:
\$ 0a3A6	b \$	$A[6, b] = r_2 \quad (A \rightarrow a A)$ Pop 6, 'A', 3, 'a' from stack Push 'A' on stack Push Goto[0,A] = 2
\$ 0A2	b \$	$A[2, b] = s_4$ Push 'b' on stack Push 4 on stack
\$ 0A2b4	\$	

Parsing a string using LR(0)

Stack	I/p buffer	Action
:	:	:
\$ 0a3A6	b \$	$A[6, b] = r_2 \quad (A \rightarrow a A)$ Pop 6, 'A', 3, 'a' from stack Push 'A' on stack Push Goto[0,A] = 2
\$ 0A2	b \$	$A[2, b] = s_4$ Push 'b' on stack Push 4 on stack
\$ 0A2b4	\$	$A[4, \$] = r_3 \quad (A \rightarrow b)$ Pop 4, 'b' from stack Push 'A' on stack Push Goto[2, A] = 5
\$ 0A2A5	\$	

Parsing a string using LR(0)

Stack	I/p buffer	Action
:	:	:
\$ 0A2b4		$A[4, \$] = r_3 \quad (A \rightarrow b)$ Pop 4, 'b' from stack Push 'A' on stack Push Goto[2, A] = 5
\$ 0A2A5		$A[5, \$] = r_1 \quad (S \rightarrow A A)$ Pop 5, 'A', 2, 'A' from stack Push 'S' on stack Push Goto[0, S] = 1
\$ 0S	\$	

Parsing a string using LR(0)

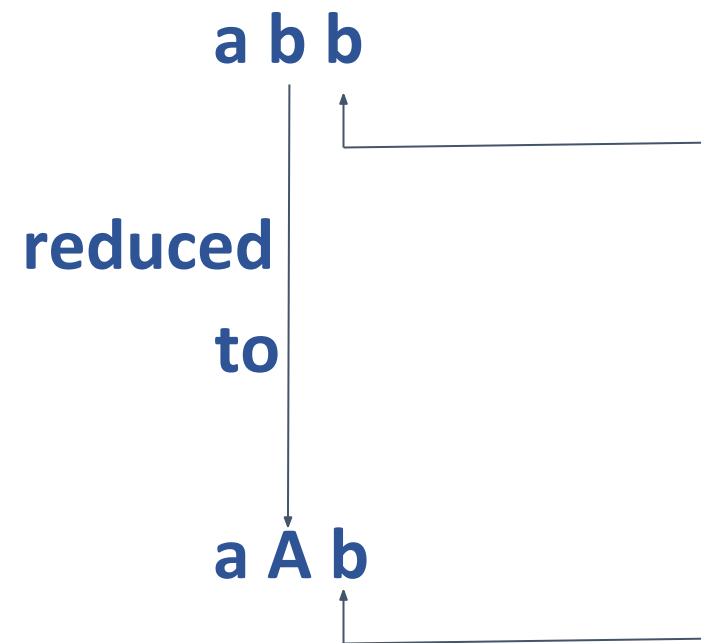
Stack	I/p buffer	Action
:	:	:
\$ 0A2b4		$A[4, \$] = r_3 \quad (A \rightarrow b)$ Pop 4, 'b' from stack Push 'A' on stack Push Goto[2, A] = 5
\$ 0A2A5		$A[5, \$] = r_1 \quad (S \rightarrow A A)$ Pop 5, 'A', 2, 'A' from stack Push 'S' on stack Push Goto[0, S] = 1
\$ 0S1		$A[1, \$] = \text{Accept}$

Parsing is
successful.

Note that LR(0) does not care about next symbol. It does **not** use lookahead.

Whenever there is a final item, we put the reduce move in the entire row corresponding to the state that contains the final item.

Considering the string,



Being at this position, we decide to reduce the previous 'b'.

Hence, we replace 'b' by 'A' using $A \rightarrow b$

Irrespective of the symbol we have here, we always reduce previous 'b' to 'A'.

This behaviour can be erroneous

Is the following grammar in LR(0)?

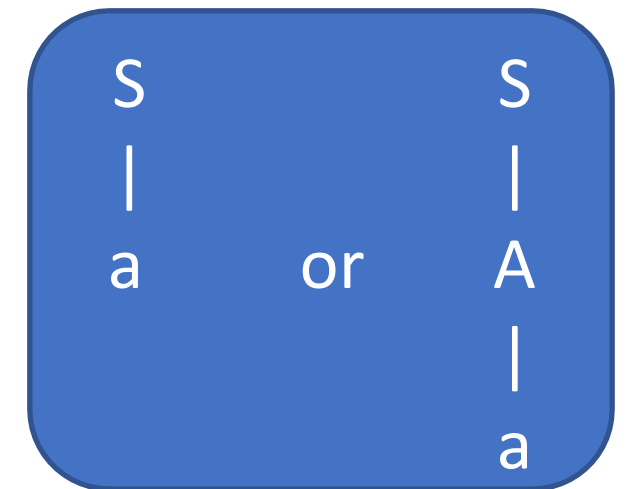
$$\begin{array}{lcl} S & \rightarrow & A \\ & | & aA \\ & & a \end{array}$$

Points to remember

- In such questions, make a quick check if the grammar is ambiguous. Giving the DFA and the table is not necessary.
- Just check if any string can be derived in more than one ways.
- In this example, “a” can be derived directly from ‘S’ or indirectly from ‘S’ via ‘A’.
- Once proved that the grammar is ambiguous, conclude that the grammar is not in LR(0).

Note: No parser can work with ambiguous grammar.

However, let us take this example to easily understand when r/r conflicts can occur.



LR(0) Parsing - Example 18.1

Is the following grammar in LR(0)?

$$\begin{array}{lcl} S & \rightarrow & A \\ & | & aA \\ & & a \end{array}$$

Solution:

Augmenting and numbering the grammar-

1. $S \rightarrow A$

2. $S \rightarrow a$

3. $A \rightarrow a$

LR(0) Parsing - Example 18.1 (continued)

Step 1: Starting

Starting with the initial item.

Step 2: $\overset{S'}{\text{Adding}} \overset{\rightarrow}{\text{productions of NTs after the dot}}$

S'	$\rightarrow$	$.S$
S	$\rightarrow$	$.A$
S	$\rightarrow$	$.a$

Step 3: Repeating step 2 until no more

<u>items</u> S'	$\rightarrow$	$.S$
S	$\rightarrow$	$.A$
S	$\rightarrow$	$.a$
A	$\rightarrow$	$.a$

Production rules:

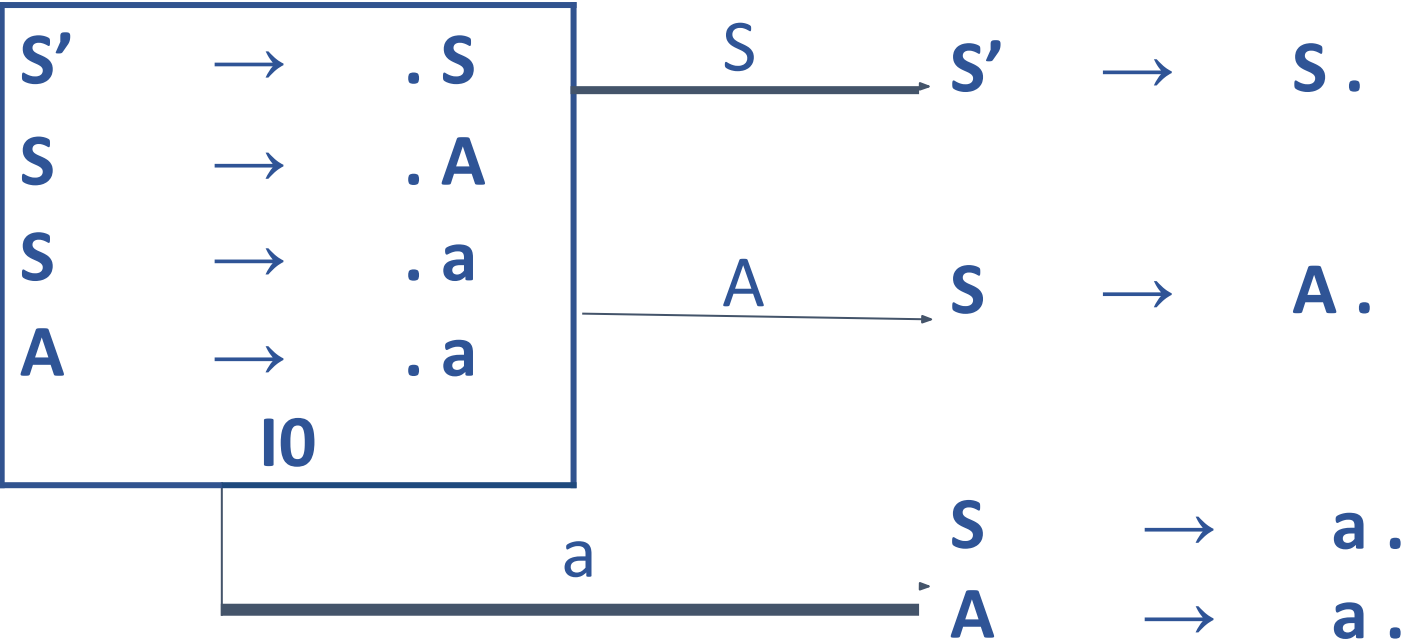
S'	$\rightarrow$	S
1. S	$\rightarrow$	A
2. S	$\rightarrow$	a
3. A	$\rightarrow$	a

LR(0) Parsing - Example 18.1
(continued)

Step 4: Making
closure

S'	$\rightarrow$	$.S$
S	$\rightarrow$	$.A$
S	$\rightarrow$	$.a$
A	$\rightarrow$	$.a$
I0		

Step 5: Transitions using the goto function



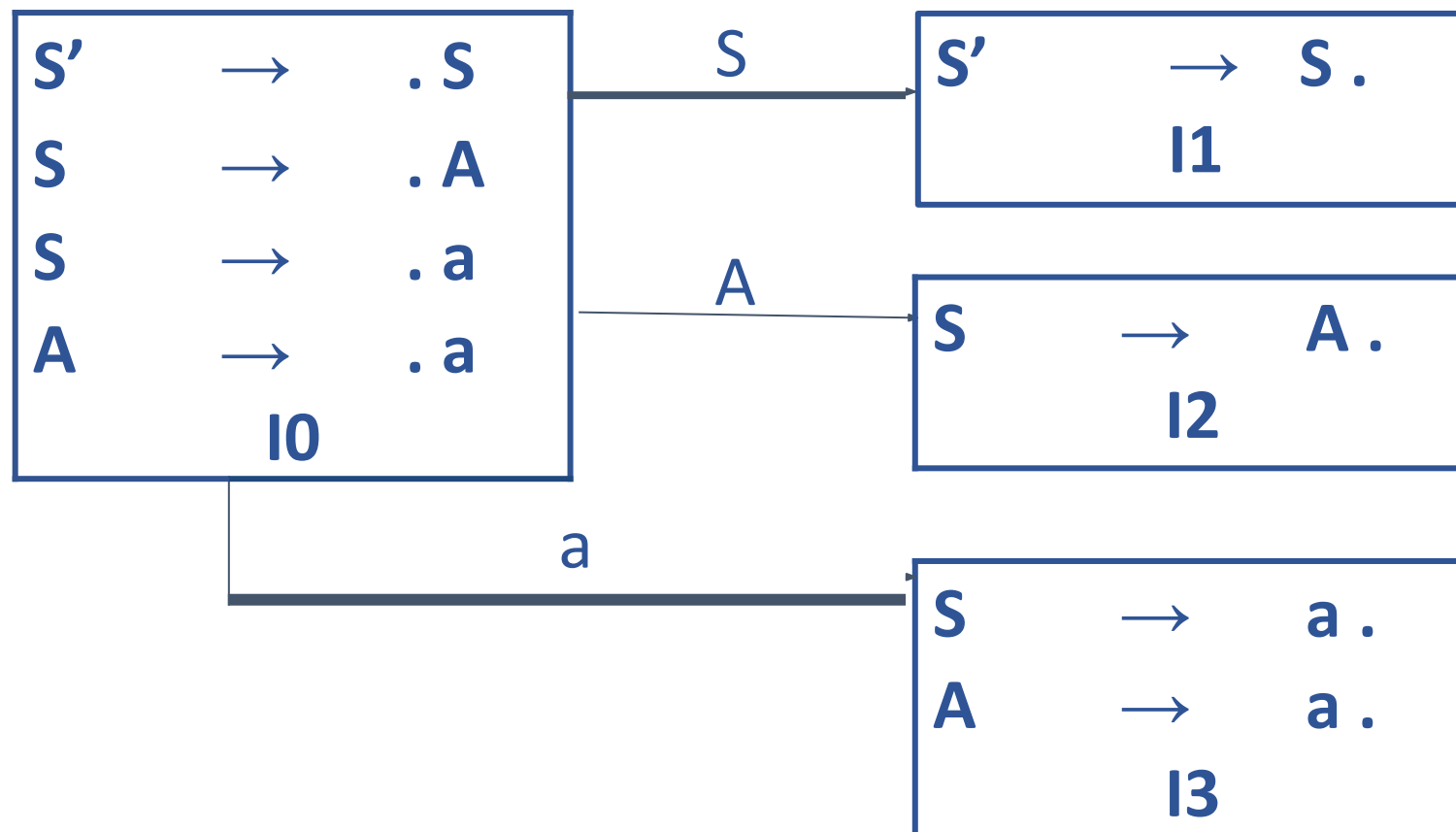
Production rules:

$S' \rightarrow S$
1. $\rightarrow A$

S

LR(0) Parsing - Example 18.1 (continued)

Step 6: Repeat steps 2-5 until no more states can be added



Production rules:

- $S' \rightarrow S$
- 1. $S \rightarrow A$
- 2. $S \rightarrow a$
- $A \rightarrow a$

LR(0) Parsing - Example 18.1
(continued)

State	Action		Goto	
	a	\$	S	A
0	s ₃		1	2

State	Action		Goto	
	a	\$	S	A
0	s ₃		1	2
1		Accept		

LR(0) Parsing - Example 18.1
(continued)

State	Action		Goto	
	a	\$	S	A
0	s ₃		1	2
1		Accept		
2	r ₁	r ₁		
3	r ₂ / r ₃	r ₂ / r ₃		

The grammar is not in LR(0) family since there exist
2 r/r (reduce/reduce) conflicts.

Recall that:
A reduce-reduce conflict occurs when there are two production rules such that the top-of-stack exactly matches the right-hand-side of both the rules

Is the following grammar in
LR(0)?

$A \rightarrow a A b \mid B b B a$

$\rightarrow$

$A \quad \lambda$

$\rightarrow$

$B \quad \lambda$

$\rightarrow$

LR(0) Parsing - Example 18.2

Is the following grammar in LR(0)?

$$A \rightarrow aAb \mid BbBa$$

$\rightarrow$
 $A \rightarrow \lambda$

Solution:

Augmenting and numbering the grammar-

1. $S \rightarrow AaAb$
2. $S \rightarrow BbBa$
3. $A \rightarrow \lambda$
4. $B \rightarrow \lambda$

Is the grammar ambiguous?

No?

Construct LR(0) automata and check.

LR(0) Parsing - Example 18.2 (continued)

Steps 1-4:

S'	$\rightarrow$	$.S$
S	$\rightarrow$	$.AaAb$
S	$\rightarrow$	$.BbBa$
A	$\rightarrow$	$.$
B	$\rightarrow$	$.$
10		

STOP!

There is a conflict here since there are two final items in the state (reduce/reduce conflict).

Conclusion:

Since there are two final items, this grammar does not belong to LR(0).

Production rules:

	S'	$\rightarrow$	S
1.		$\rightarrow$	$AaAb$
S			
2.		$\rightarrow$	$BbBa$

A snapshot of the table may be given.

State	Action		Goto		
	a	\$	S	A	B
0	r_3/r_4	r_3/r_4			

There are only two strings that can be generated using this grammar: “ab” and “ba”.

Even though the grammar is not ambiguous, there is a problem. It is because the problem is in LR(0). The LR(0) parser is not looking ahead into what symbol is on the input buffer.

Stack

\$

I/p

Buffer

a b \$

The first symbol in the input buffer is “a”. So, we must reduce $A \rightarrow \lambda$.

If the first symbol was “b”, then we must reduce $B \rightarrow \lambda$.

So, the problem is with the parser. It takes decisions without checking properly. The parser is weak. If we look-ahead, it will work properly - CLR / LR(1).

Production rules:

S'	$\rightarrow$	S
1.	$\rightarrow$	$AaAb$
S		
2.	$\rightarrow$	$BbBa$

Is the following grammar in

LR(0)? $T \rightarrow E \mid T$

→

$T \rightarrow id$

→

LR(0) Parsing - Example 18.3

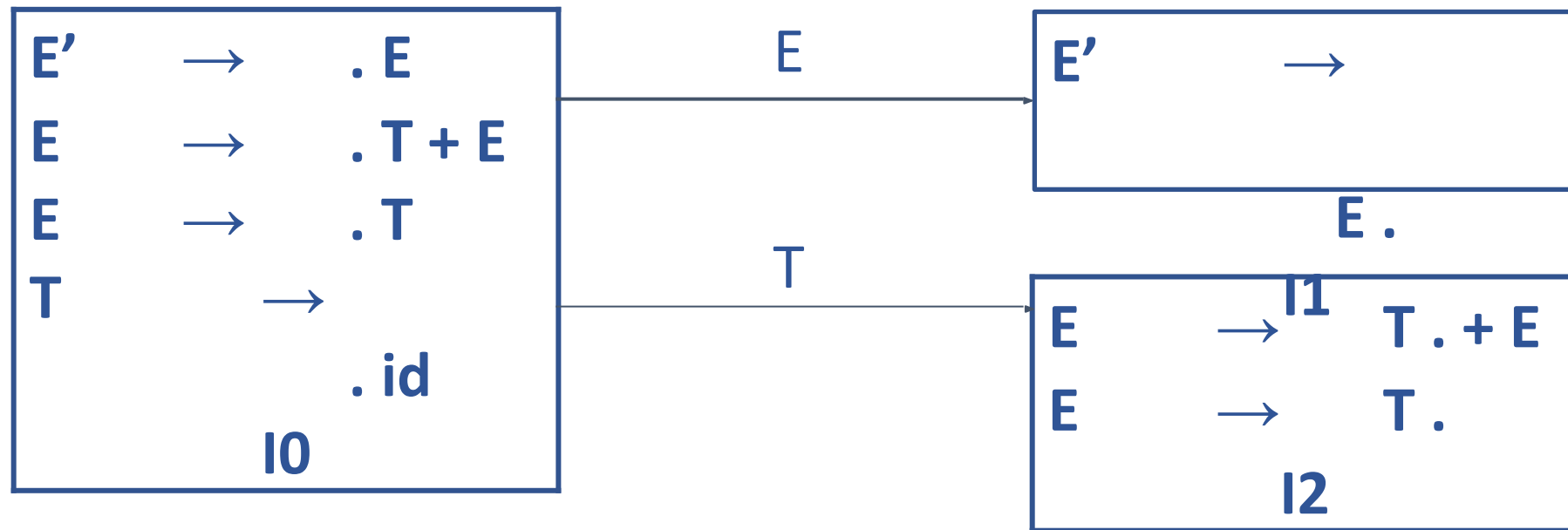
Is the following grammar in LR(0)?

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{id}$$

Solution:

Augmenting and numbering the grammar-

- | | | | |
|----|-----|---------------|-------------|
| | E | $\rightarrow$ | E |
| 1. | E | $\rightarrow$ | $T + E$ |
| 2. | E | $\rightarrow$ | T |
| 3. | T | $\rightarrow$ | id |



Notice that, in the state I2, there is a shift/reduce conflict. The state can transition over “+” to a new state, based on the first item. Or, a reduction can happen using rule 2, since the second item in the state is a final item.

Production rules

- $\rightarrow T + E$

E

A snapshot of the table is given below:

State	Action			Goto	
	id	+	\$	E	T
2	r_2	s_4/r_2	r_2		

Conclusion:

Since there is a shift/reduce conflict on “+” in the state I2, the grammar is not in LR(0).

But, is it really a problem? This grammar can generate: id
id + id
id + id + id ...

- When we are working with just “id”, the next symbol upon shift is ‘\$’ and if we just look-ahead to check if it is ‘\$’, we can simply reduce it.
- If there was a “+” as the next symbol on the buffer, then it can be shifted.
- LR(1) works better in such cases.

LR(0) is not good enough

- LR(0) is the **simplest** technique in the LR family. Although that makes it the easiest to learn, these parsers are **too weak** to be of practical use for anything but a very limited set of grammars.
- The fundamental limitation of LR(0) is the zero, meaning no lookahead tokens are used. It is a stifling constraint to have to make decisions using only what has already been read, without even glancing at what comes next in the input.
- If we could peek at the next token and use that as part of the decision making, we will find that it allows for a much larger class of grammars to be parsed.



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Lavitra Kshitij
Madan

Compiler Design

Unit 2

Table-driven BUP

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, we will learn about:

- SLR Parsers
- Follow set
- Comparison between LR(0) and SLR
- Conflicts in LR(0) and SLR parsers
- Viable prefix and its importance

- Considering SLR(1), where the S stands for simple. SLR(1) parsers use the same LR(0) automata and items and have the same table structure and parser operation, so everything you have already learned about LR(0) applies here.
- The difference comes in assigning table actions, where we are going to use one token of lookahead to help arbitrate among the conflicts.
- If we think back to the kind of conflicts we encountered in LR(0) parsing, it was the reduce actions that cause us grief. A state in an LR(0) parser can have at most one reduce action and cannot have both shift and reduce instructions.

- Since a reduce is indicated for any final item, this dictates that each final item must be in a state by itself.
- But let's revisit the assumption that if the item is complete, the parser must choose to reduce. Is that always appropriate? If we peeked at the next upcoming token, it may tell us something that invalidates that reduction.
- Basically, it says that we should not blindly put the 'reduce' move in the entire action part of the parsing table, but perform some sort of lookahead and see what is next in the buffer and then take the decision!

How does it look-ahead?

It places the reduce move only in the set **follow(LHS)**.

The set $\text{follow}(X)$ is defined as the set of all terminals that follow X in the production rules.

Consider the grammar from the previous example (Example 18.3 of previous lecture):

- | | | | |
|----|------|---------------|-------------|
| | E' | $\rightarrow$ | E |
| 1. | E | $\rightarrow$ | $T + E$ |
| 2. | E | $\rightarrow$ | T |
| 3. | T | $\rightarrow$ | id |

Recall that there was a shift/reduce conflict in state I2:

E	→	T . + E
E	→	T .
I2		

Follow(E) is only '\$' since it is the start symbol.
The reduce move is placed only under those terminals in the table that are a part of this follow set.

A snapshot of the SLR table is:

State	Action			Goto	
	id	+	\$	E	T
2		s ₄	r ₂		

Production rules:

E'

→

E

1.

→

T + E

E

There is no shift-reduce conflict now for that particular state.

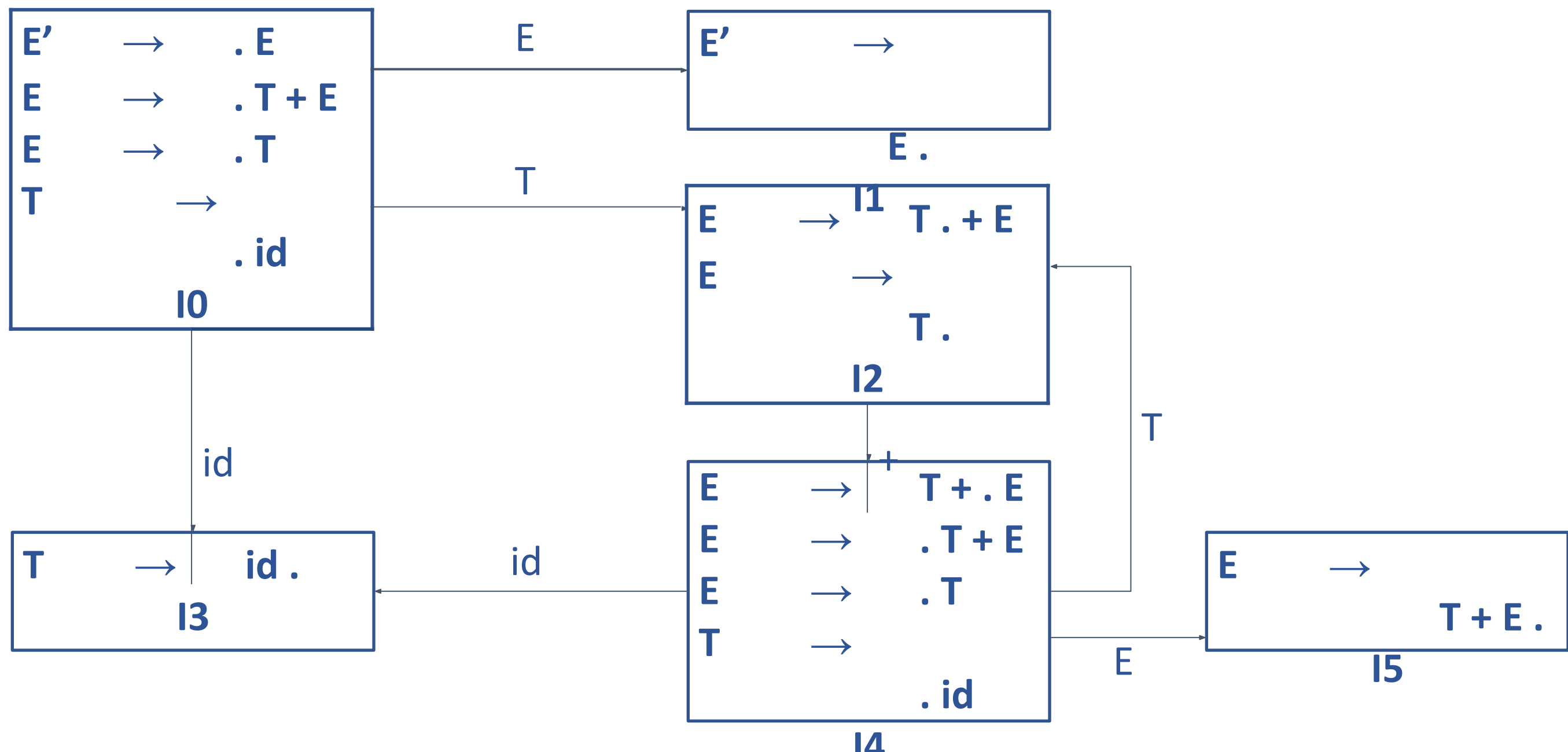
Let us continue building the solution to check if this grammar belongs to SLR or not

Continuing similarly, $\text{follow}(T) = \{+, \$\}$ because “+” follows ‘T’ in the second production rule and “\$” follows ‘T’ in the third rule.

Production rules:

E'	$\rightarrow$	E
1. E	$\rightarrow$	$T + E$

Let us finish building the LR(0) automata.



Let us now build the SLR parsing table.

State	Action			Goto	
	id	+	\$	E	T
0	s ₃			1	2
1			Accept		
2		s ₄	r ₂		
3		r ₃	r ₃		
4	s ₃			5	2
5			r ₁		

Production rules:

$E' \rightarrow E$
1. $E \rightarrow T + E$

E

A blank in the parsing table corresponds to error.

SLR(1) detects errors faster than LR(0) as SLR(1) parsing table has more blanks than corresponding LR(0) parsing table.

Advantage of SLR over LR(0)

The simple improvement that SLR(1) makes on the basic LR(0) parser is to reduce only if the next input token is a member of the follow set of the non-terminal being reduced. When filling in the table, we don't assume a reduce on all inputs as we did in LR(0), we selectively choose the reduction only when the next input symbols in a member of the follow set.

Let us now parse the
string: $w = \text{"id + id"}$
using the parsing table we have developed as
part of the example.

The initial configuration is as follows:

Stack	I/p buffer	Action
\$ 0	id + id \$	

Let us start parsing the string step-by-step.

Stack	I/p buffer	Action
\$ 0	id + id \$	$A[0, id] = s_3$ Push 'id', 3 onto stack
\$ 0 id 3	+ id \$	

Stack	I/p buffer	Action
\$ 0	id + id \$	$A[0, id] = s_3$ Push 'id', 3 onto stack
\$ 0 id 3	+ id \$	$A[3, +] = r_3 (T \rightarrow id)$ Pop 3, 'id' from stack Push 'T' onto stack Push Goto[0, T] = 2
\$ 0 T 2	+ id \$	

Stack	I/p buffer	Action
\$ 0	id + id \$	$A[0, id] = s_3$ Push 'id', 3 onto stack
\$ 0 id 3	+ id \$	$A[3, +] = r_3 (T \rightarrow id)$ Pop 3, 'id' from stack Push 'T' onto stack Push Goto[0, T] = 2
\$ 0 T 2	+ id \$	$A[2, +] = s_4$ Push '+', 4 onto stack
\$ 0 T 2 + 4	id \$	

Stack	I/p buffer	Action
\$ 0	id + id \$	$A[0, id] = s_3$ Push 'id', 3 onto stack
\$ 0 id 3	+ id \$	$A[3, +] = r_3 (T \rightarrow id)$ Pop 3, 'id' from stack Push 'T' onto stack Push Goto[0, T] = 2
\$ 0 T 2	+ id \$	$A[2, +] = s_4$ Push '+', 4 onto stack
\$ 0 T 2 + 4	id \$	$A[4, id] = s_3$ Push 'id', 3 onto stack
\$ 0 T 2 + 4 id 3	\$	

Stack	I/p buffer	Action
:	:	:
\$ 0 T 2 + 4 id 3	\$	$A[3, \$] = r_3 (T \rightarrow id)$ Pop 3, 'id' from stack Push 'T' onto stack Push Goto[4, T] = 2
\$ 0 T 2 + 4 T 2	\$	

Stack	I/p buffer	Action
:	:	:
\$ 0 T 2 + 4 id 3		$A[3, \$] = r_3 (T \rightarrow id)$ Pop 3, 'id' from stack Push 'T' onto stack Push Goto[4, T] = 2
\$ 0 T 2 + 4 T 2		$A[2, \$] = r_2 (E \rightarrow T)$ Pop 2, 'T' from stack Push 'E' onto stack Push Goto[4, E] = 5
\$ 0 T 2 + 4 E 5	\$	

Stack	I/p buffer	Action
:	:	:
\$ 0 T 2 + 4 E 5	\$	A[5, \$] = r (E → T + E) 1 Pop 5, 'E', 4, '+', 2, 'T' Push 'E' onto stack Push Goto[0, E] = 1
\$ 0 E 1	\$	

Stack	I/p buffer	Action
:	:	:
\$ 0 T 2 + 4 E 5		\$ A[5, \$] = r (E → T + E) 1 Pop 5, 'E', 4, '+', 2, 'T' Push 'E' onto stack Push Goto[0, E] = 1
\$ 0 E 1		\$ A[1, \$] = Accept

Comparison between LR(0) and SLR

Similarities between LR(0) and SLR parsers:

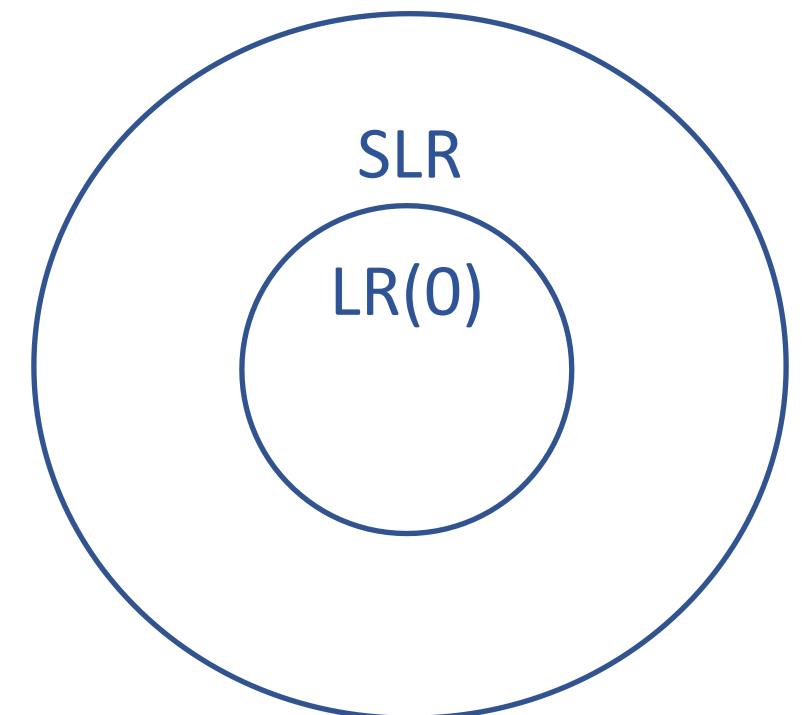
- Both use LR(0) automata
- Same way to fill the Goto part for the shift move in the parsing table.

The only difference between the two:

- Where to place the reduce move

The addition of just one token of lookahead and use of the follow set greatly expands the class of grammars that can be parsed without conflict.

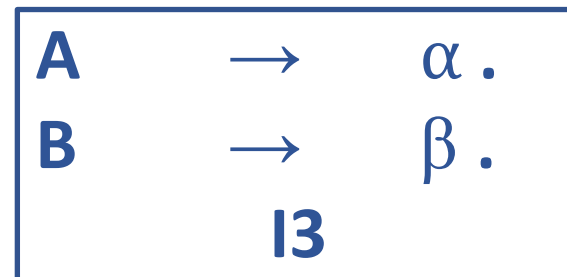
If the grammar is in LR(0), it is definitely in SLR and if it is in SLR, it may or may not be in LR(0).



Conflicts in LR(0) and SLR(1)

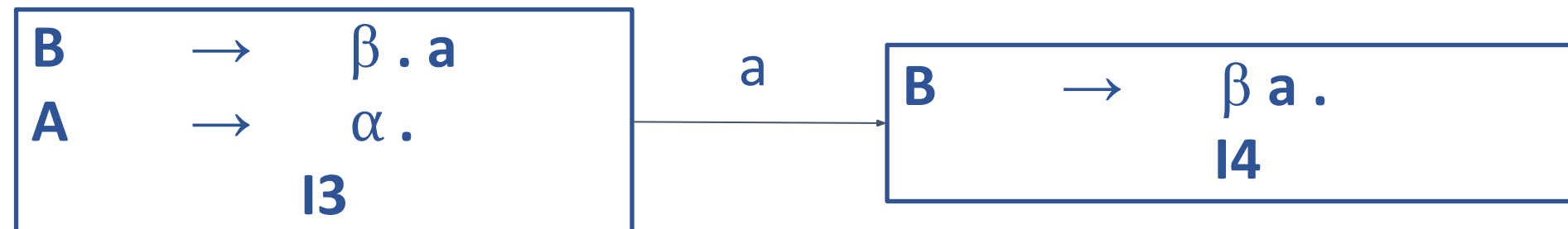
There exist a r/r conflict in LR(0) if and only if there are two or more final items in a state.

Example:



A shift/reduce (s/r) conflict occurs in LR(0) when there is a final item and a shift on a terminal.

Example:



There exist a r/r conflict in SLR if and only if there are two or more final items in a state and the intersection of the follow sets of the left-hand-sides of both the items is not empty.

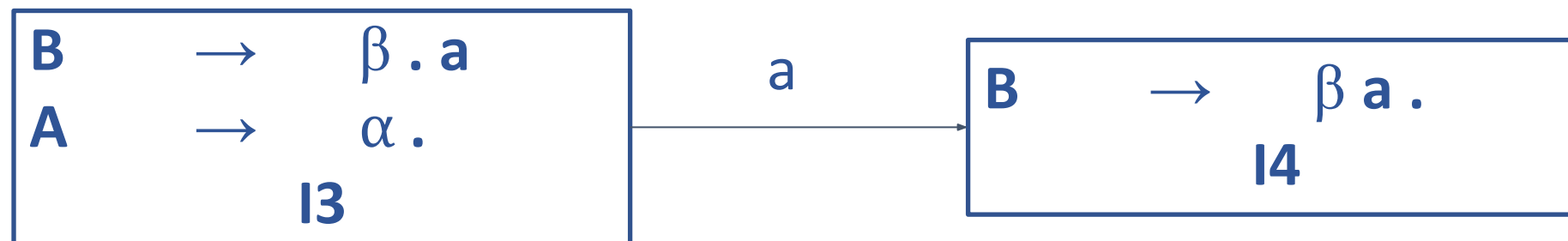
Example:

A	$\rightarrow$	$\alpha .$
B	$\rightarrow$	$\beta .$
I3		

If f (There is something in common),
then there exists a reduce-reduce conflict.

A shift/reduce (s/r) conflict occurs in SLR when there is a final item and a shift on a terminal, and the follow of the left-hand-side of the final item contains that terminal which may be shifted next.

Example:



If $\text{follow}(A)$ contains 'a', then there is a shift-reduce conflict.

Viablr Prefix

Recall the shift-reduce parser example

The string was accepted by the

Stack	I/p buffer	Action
\$	id * id \$	Shift id
\$ id	* id \$	Reduce using (iii)
\$ F	* id \$	Reduce using (ii)
\$ T	* id \$	Shift *
\$ T *	id \$	Shift id
\$ T * id	\$	Reduce using (iii)
\$ T * F	\$	Reduce using (ii)
\$ T	\$	Reduce using (i)
\$ E	\$	Accept

Grammar:

- (i) $E \rightarrow E + T \mid T$
- (ii) $T \rightarrow T * F \mid F$
- (iii) $F \rightarrow id$

$w = \text{"id * id"}$

We observe that at any point of time, the stack contents must be a prefix of a right sentential form. However, not all prefixes of a right sentential form can appear on the stack.

For example, consider the rightmost derivation:

Rightmost derivation of $\text{id} * \text{id}$	Set of prefixes of a right sentential form	Viable Prefixes
$E \rightarrow T$	T	T
$\rightarrow T * F$	$T, T *, T * F$	$T, T *, T * F$
$\rightarrow T * \text{id}$	$T, T *, T * \text{id}$	$T, T *, T * \text{id}$
$\rightarrow F * \text{id}$	$F, F *, F * \text{id}$	F
$\rightarrow \text{id} * \text{id}$	$\text{id}, \text{id} *, \text{id} * \text{id}$	id

Here, “**id ***” is a prefix of a right sentential form. But it can never appear on the stack!

This is because we will always reduce by **F -> id** before shifting “*****”.

The prefixes of right sentential forms that can appear on the stack of a shift-reduce parser are called **viable prefixes**. Its a building block for recognizing handles.

By definition, a viable prefix is a prefix of a right sentential form that does not continue past the right end of the rightmost handle of that sentential form. It’s a **viable prefix** because it is a prefix of the **handle**.

We will see the importance of viable prefixes in the upcoming lectures when we learn about more powerful parsers.

The entire parsing algorithm is based on the idea that the LR(0) automaton can recognize viable prefixes and reduce them appropriately.

Recognizing Viable Prefixes Idea: To recognize viable prefixes, we must

- Recognize a sequence of partial RHSs of productions, where
- Each sequence can eventually reduce to part of the missing suffix of its predecessor.



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Philip
Joseph

Compiler Design

Unit 2

Table-driven BUP

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, we will practice -

- Questions related to different Bottom up parsing techniques covered so far.

Provide a production with the shortest RHS that will introduce s/r conflict in SLR parser for the following grammar -

S $\rightarrow$ **A b**

A $\rightarrow$ **c b A d**

A $\rightarrow$ **c A d**

A $\rightarrow$ **λ**

Question 1 -

Solution

Add a production

$$A \rightarrow \begin{matrix} b & | \\ d \end{matrix}$$

to the given grammar

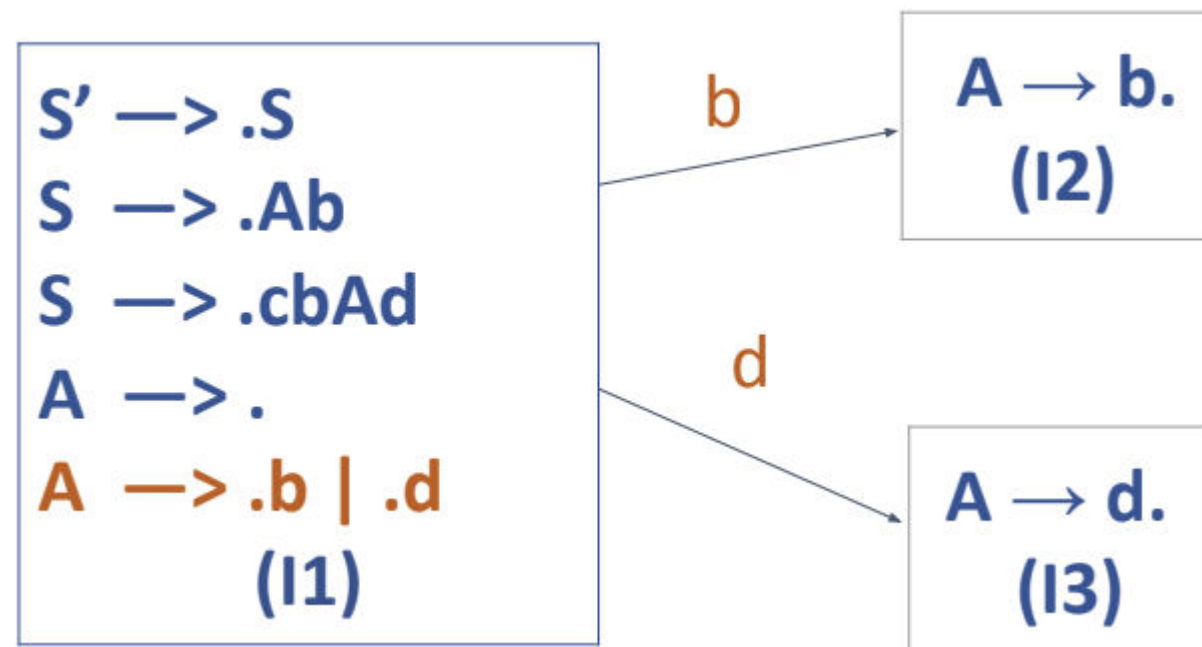
- $S \rightarrow A b$
- $A \rightarrow c b A d$
- $A \rightarrow c A d$
- $A \rightarrow \lambda$
- $A \rightarrow b | d$

Question 1 -

Solution

To ensure an s/r conflict, the terminal shift from I1 should be 'b' and 'd' to satisfy the condition for s/r conflict in SLR parser because I0 has a final state $A \rightarrow \cdot$ and $\text{Follow}(A) = \{ b, d \}$ should contain that terminal 'b' and 'd' which may be shifted next.

If $\text{follow}(A)$ contains 'b' or 'd', then there is a shift-reduce conflict.



Identify whether the given grammar is LL(1), LR(0), or SLR(1)?

$$X \rightarrow Yz \mid$$
$$Y \rightarrow \begin{matrix} a \\ bZ \mid \end{matrix}$$
$$Z \rightarrow \begin{matrix} \lambda \\ \lambda \end{matrix}$$

Question 2 -

Solution

- To check if a grammar is LL(1), one option is to construct the LL(1) parsing table and check for any conflicts.
- Build the FIRST and FOLLOW sets for each of the nonterminals. Here, we get -
 - $\text{FIRST}(X) = \{ a, b, z \}$
 - $\text{FIRST}(Y) = \{ b, \lambda \}$
 - $\text{FIRST}(Z) = \{ \lambda \}$
- FOLLOW sets are given by
 - $\text{FOLLOW}(X) = \{ \$ \}$
 - $\text{FOLLOW}(Y) = \{ z \}$
 - $\text{FOLLOW}(Z) = \{ z \}$

Question 2 -

Solution

Build the following LL(1) parsing table

-

	a	b	z	\$
X	$X \rightarrow a$	$X \rightarrow Yz$	$X \rightarrow Yz$	
Y		$Y \rightarrow bZ$	$Y \rightarrow \lambda$	
Z			$Z \rightarrow \lambda$	

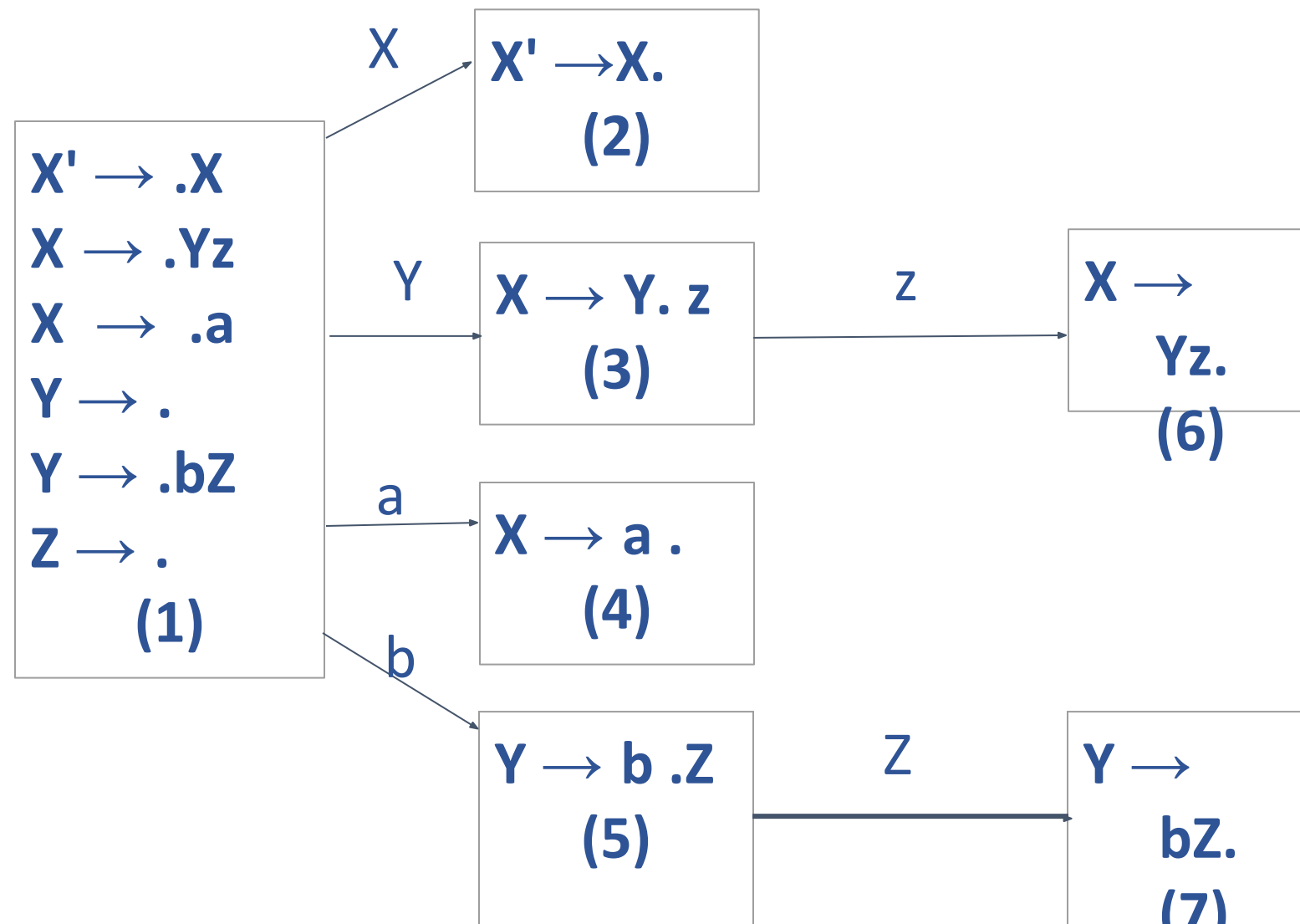
Since we can build this parsing table with no conflicts, the grammar is in LL(1).

Question 2 -

Solution

To check if a grammar is LR(0) or SLR(1), we begin by building up all of the LR(0) sets for the grammar.

In this case, assuming that X is the start symbol, we get the following -



Question 2 -

Solution

- We can see that the grammar is **not in LR(0)** because there is a shift-reduce conflict in state (1).
- This is because, since there is a shift item $X \rightarrow .a$ and final item $Y \rightarrow .$, we can't tell whether to shift the 'a' or reduce the empty string.
- More generally, no grammar with ϵ -productions is LR(0).
- However, there remains the possibility that grammar might be SLR(1).

Question 2 -

Solution

- To check if the grammar is in SLR(1), we augment each reduction with the lookahead set for the particular nonterminals.
- The shift/reduce conflict in state (1) has been eliminated in SLR(1) because we only reduce when the lookahead is 'z', which doesn't conflict with any of the other items.
- The reduce/reduce conflict in state (1) is not eliminated in SLR(1) because $\text{follow}(Y) \cap \text{follow}(Z) \neq \Phi$
- Thus this grammar is not in SLR(1).



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Philip Joseph

Compiler Design

Unit 2

Table-driven BUP

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, we will practice -

- Questions related to LR(0) and SLR Parsers.

Let us try to solve the following problem step-by-step.

Construct LR(0) automata and LR(0) & SLR parsing tables for the following grammar.

S	→	d A	 	a B
A	→	b A	 	c
B	→	b B	 	c

Recall the steps for constructing the LR automaton:

1. Augment the grammar
2. Number your productions in the original grammar
3. Follow the LR(0) Parsing Algorithm

LR(0) Parsing - Example (continued)

Augmenting and numbering the grammar

	S'	$\rightarrow$	S
1.	S	$\rightarrow$	$d A$
2.	S	$\rightarrow$	$a B$
3.	A	$\rightarrow$	$b A$
4.	A	$\rightarrow$	c
5.	B	$\rightarrow$	$b B$
6.	B	$\rightarrow$	c

Recall the LR(0) Parsing Algorithm

1. Start with the initial item.
2. If there is a non-terminal after the “.” in any of the productions, add all the productions from that non-terminal, i.e., where that non-terminal is on the left-hand-side.
3. For every newly added item, repeat step 2.
4. If no more items can be added, it becomes a closure/state.
5. Use the goto function to define a transition for each symbol after the dot.
6. Repeat the process until no more new states can be added to the automaton.

LR(0) Parsing - Example (continued)

Step 1: Starting

Starting with the initial item.

$$S' \rightarrow . S$$

Step 2: Adding productions of NTs after the dot

Since “S” follows the dot, we put the first production.

$$S \rightarrow . S$$

$$S \rightarrow . d A$$

$$S \rightarrow . a B$$

Step 3: Repeating step 2 until no more

items There are no productions that non-
have immediately following a dot. terminal

Production rules:

	S'	$\rightarrow$	S
1.	S	$\rightarrow$	$d A$
2.	S	$\rightarrow$	$a B$
3.	A	$\rightarrow$	$b A$
4.	A	$\rightarrow$	c
5.	B	$\rightarrow$	$b B$
6.	B	$\rightarrow$	c

LR(0) Parsing - Example (continued)

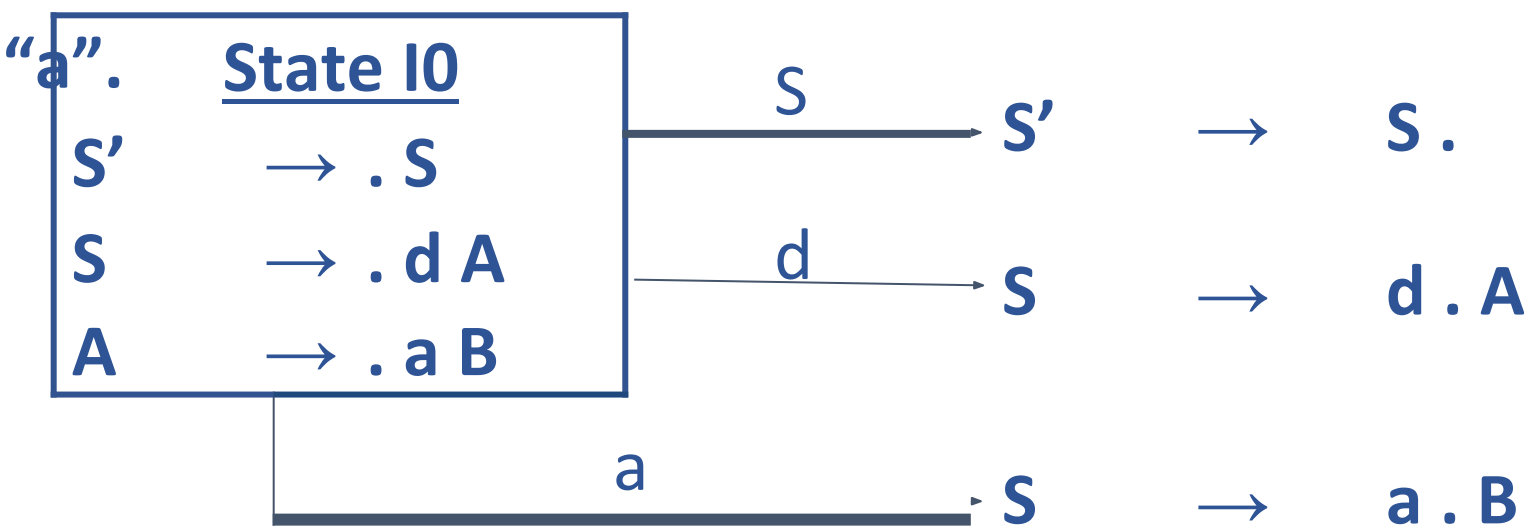
Step 4: Making closure

Since no more items can be added to the state, we can now make it a closure. It is done by adding the productions in a box and naming it.

State I0		
S'	→	. S
S	→	. d A
A	→	. a B

Step 5: Transitions using the goto function

We will have transitions on “S”, “d” and

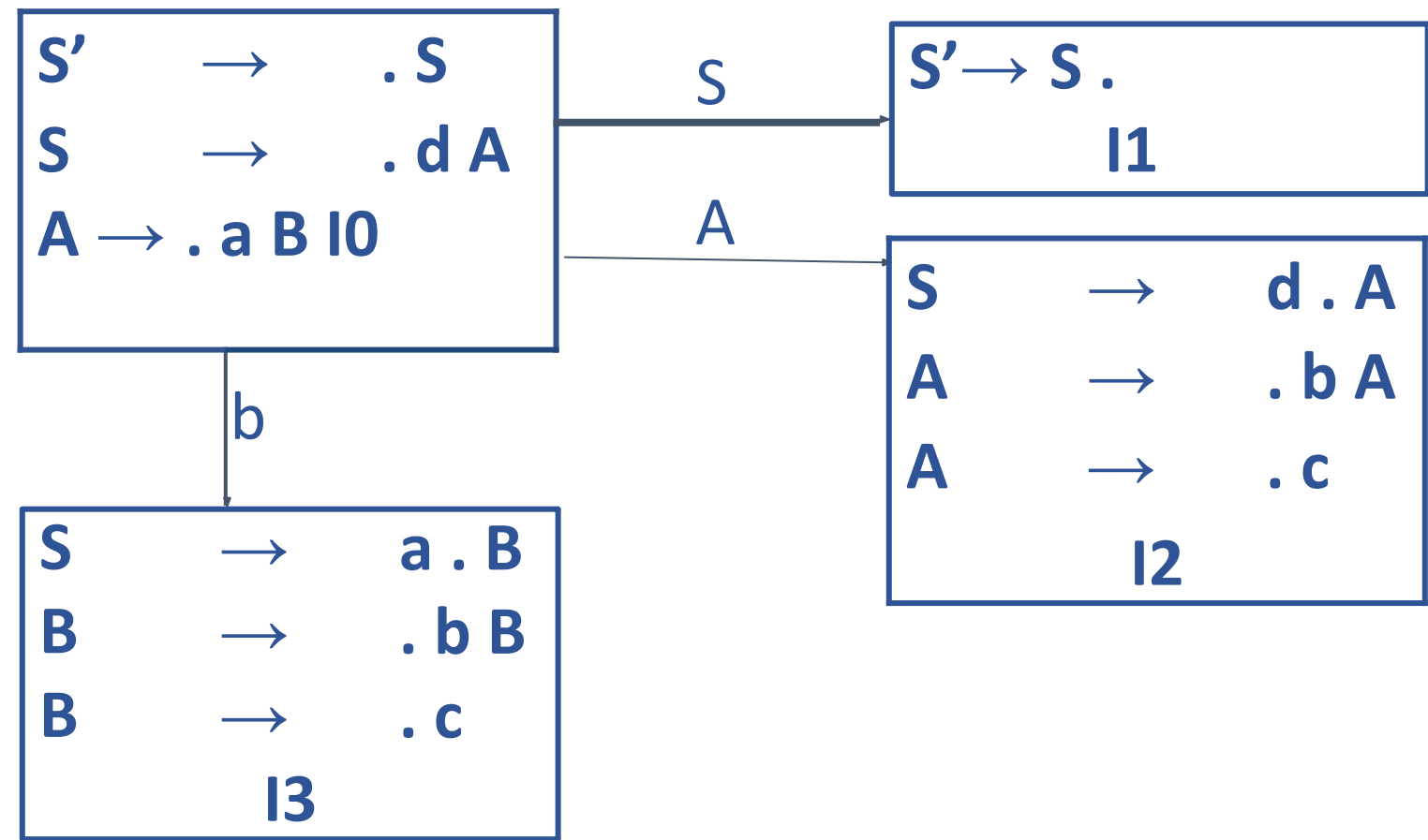


Production rules:

- | | | | |
|----|----|---|-----|
| | S' | → | S |
| 1. | S | → | d A |
| 2. | S | → | a B |
| 3. | A | → | b A |
| 4. | A | → | c |
| 5. | B | → | b B |
| 6. | B | → | c |

LR(0) Parsing - Example (continued)

Step 6: Repeat steps 2-5 until no more states can be added
 $S' \rightarrow S$. is already completed and closure can be used on it.
The other two transitions require steps 2-5.



Production rules:

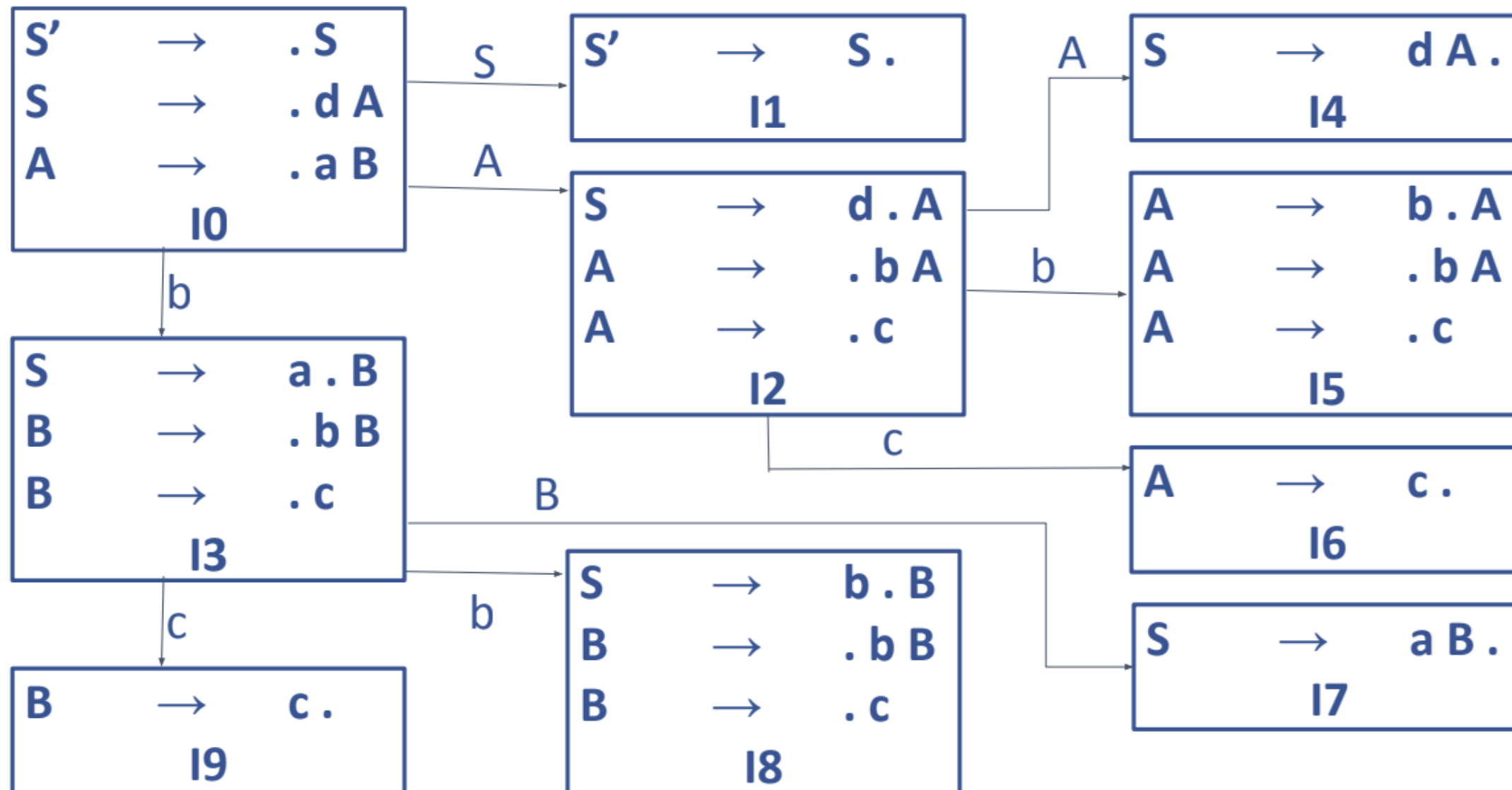
- | | | | |
|----|------|---------------|-------|
| | S' | $\rightarrow$ | S |
| 1. | S | $\rightarrow$ | $d A$ |
| 2. | S | $\rightarrow$ | $a B$ |
| 3. | A | $\rightarrow$ | $b A$ |
| 4. | A | $\rightarrow$ | c |
| 5. | B | $\rightarrow$ | $b B$ |
| 6. | B | $\rightarrow$ | c |

LR(0) Parsing - Example (continued)

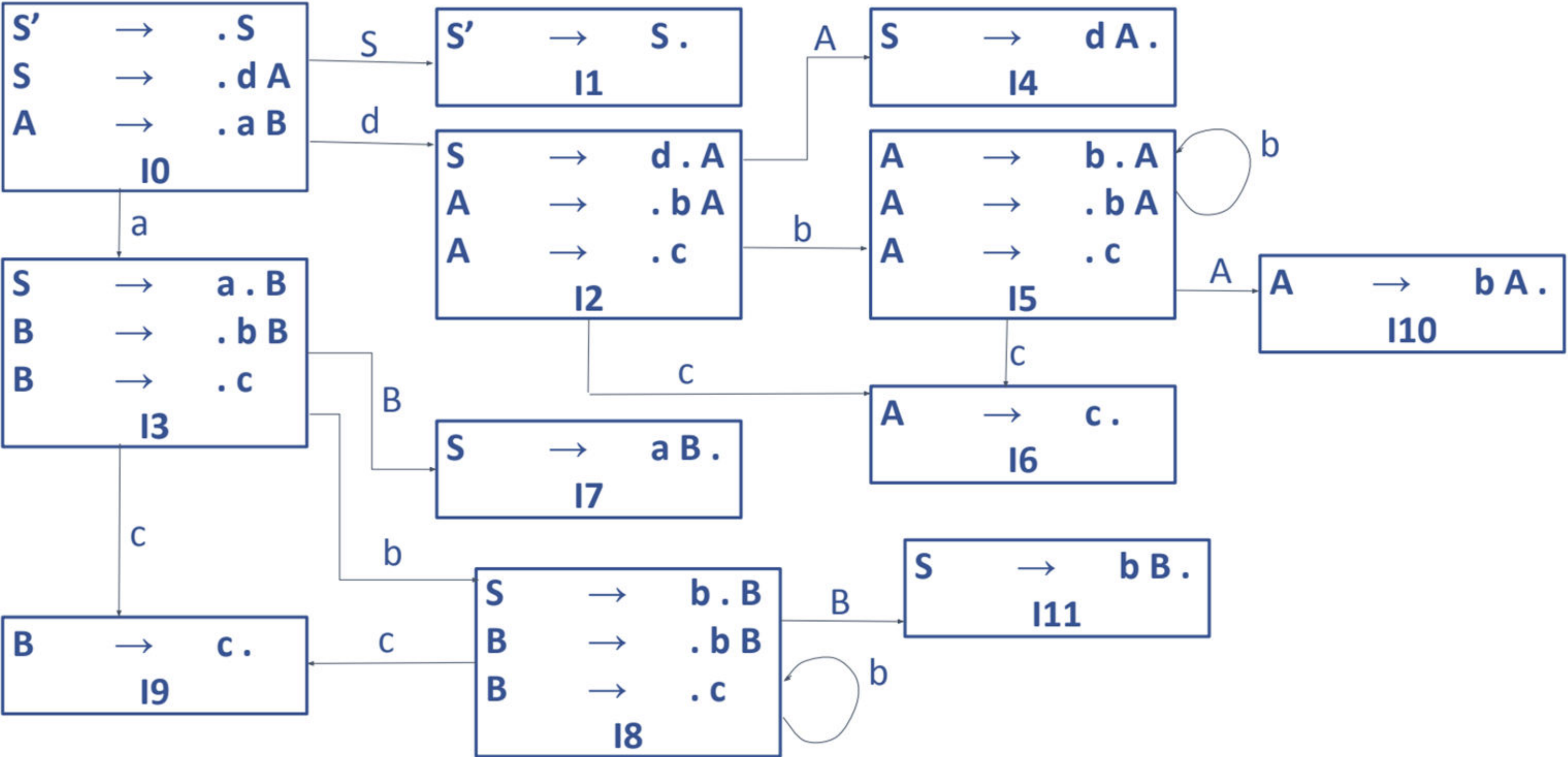
Step 6: Repeat steps 2-5 until no more states can be added

$S' \rightarrow S$. is already completed and closure can be used on it.

The other two transitions require steps 2-5.



LR(0) Parsing - Example (continued)



LR(0) Parsing - Example (continued)

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0								

We will start filling the LR(0) Parsing Table now.

For the state I0, transition over “S” leads to state I1 and so, we fill the ‘S’ sub-column in the ‘goto’ column as “1”. Similarly, transition over “d” and “a” lead to states I2 and I3 respectively. We mark them as s_2 and s_3 in the respective sub-columns of the ‘action’ column.

LR(0) Parsing - Example (continued)

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0	s ₃			s ₂		1		

State 1 contains the special final item. “Accept” should be put under the ‘\$’ sub-column in the ‘Action’ column.

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0	s ₃			s ₂		1		
1					Accept			

LR(0) Parsing - Example (continued)

Similarly filling for states I2, I3, I5 and

I8.

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0	s ₃			s ₂		1		
1					Accept			
2		s ₅	s ₆				4	
3		s ₈	s ₉					7
5		s ₅	s ₆				10	
8		s ₈	s ₉					11

States I4, I6 I7, I9, I10 and I11 have a final item each and so, we must reduce them. To reduce, we must place “r_x” in the **entire action column** for that state, where “X” takes the value of the production rule number.

LR(0) Parsing - Example (continued)

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0	s ₃			s ₂		1		
1					Accept			
2		s ₅	s ₆				4	
3		s ₈	s ₉					7
4	r ₁	r ₁	r ₁	r ₁	r ₁			
5		s ₅	s ₆				10	
6	r ₄	r ₄	r ₄	r ₄	r ₄			
: : : : : : : : :								

Table continued on next slide

LR(0) Parsing - Example (continued)

State	Action					Goto		
	a	b	c	d	\$	S	A	B
	:	:	:	:	:	:	:	:
7	r ₂	r ₂	r ₂	r ₂	r ₂			
8		s ₈	s ₉					11
9	r ₆	r ₆	r ₆	r ₆	r ₆			
10	r ₃	r ₃	r ₃	r ₃	r ₃			
11	r ₅	r ₅	r ₅	r ₅	r ₅			

Note: These states are also called Canonical LR(0) collection. $C = \{I_0, I_1, I_2, \dots, I_{11}\}$

SLR(1) Parsing - Example (continued)

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0								

Let us also fill the SLR(1) Parsing Table now. Let us build the follow table first.

$\text{follow}(S) = \$$

$\text{follow}(A) = \text{follow}(S) =$

$\$ \text{follow}(B) = \text{follow}(S) =$

$\$$

Production rules:

- S'
- $\rightarrow$
- S
1.
- S
- $\rightarrow$
- $d A$
2.
- S
- $\rightarrow$
- $a B$
3.
- A
- $\rightarrow$
- $b A$
4.
- A
- $\rightarrow$
- c
5.
- B
- $\rightarrow$
- $b B$
6.
- B
- $\rightarrow$
- c

SLR(1) Parsing - Example (continued)

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0	s ₃			s ₂		1		
1					Accept			
2		s ₅	s ₆				4	
3		s ₈	s ₉					7
4					r ₁			
5		s ₅	s ₆				10	
6					r ₄			
:	:	:	:	:	:	:	:	:

Table continued on next slide

SLR(1) Parsing - Example (continued)

State	Action					Goto		
	a	b	c	d	\$	S	A	B
:	:	:	:	:	:	:	:	:
7					r_2			
8		s_8	s_9					11
9					r_6			
10					r_3			
11					r_5			

Let us now parse the
string: $w = \text{“abc”}$
using the LR(0) and SLR parsing tables we have developed as
part of the example.

The initial configuration is as follows:

Stack	I/p buffer	Action
\$ 0	a b c \$	

Let us start parsing the string step-by-
step.

Compiler Design

Parsing a string -

LR(0)



Stack	I/p buffer	Action
\$ 0	a b c \$	$A[0, a] = s_3$ Push 'a', 3 onto stack
\$ 0 a 3	b c \$	$A[3, b] = s_8$ Push 'b', 8 onto stack
\$ 0 a 3 b 8	c \$	$A[8, c] = s_9$ Push 'c', 9 onto stack
\$ 0 a 3 b 8 c 9	\$	$A[9, \$] = r_6 \text{ (B} \rightarrow \text{c)}$ Pop 9, 'c' from stack Push 'B' onto stack Push Goto[8, B] = 11
Table continued on next slide		

Production rules:

S'

1. S

2. S

3. A

4. A

5. B

6. B

→

→

→

→

→

→

S

d A

a B

b A

c

b B

c

Compiler Design

Parsing a string -

LR(0)



Stack	I/p buffer	Action
:	:	:
\$ 0 a 3 b 8 B 11		\$ A[11, \$] = r ₅ (B → b B) Pop 11, 'B', 8, 'b' Push 'B' onto stack Push Goto[3, B] = 7
\$ 0 a 3 B 7		\$ A[7, \$] = r ₂ (S → a B) Pop 7, 'B', 3, 'a' Push 'S' onto stack Push Goto[0, S] = 1
\$ 0 S 1		\$ A[1, \$] = r ₁ (S → S)

Production rules:

- S' → S
- S → d A
- S → a B
- A → b A
- A → c
- B → b B
- B → c

The string has been successfully parsed. Accept

When the same string is parsed using SLR, we will get the same

Let us try to solve another problem.

Construct LR(0) automata and LR(0) & SLR parsing tables for the following grammar.

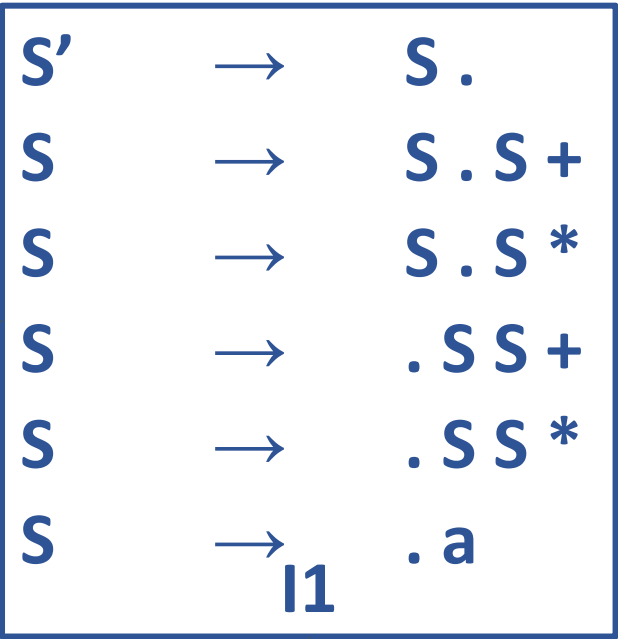
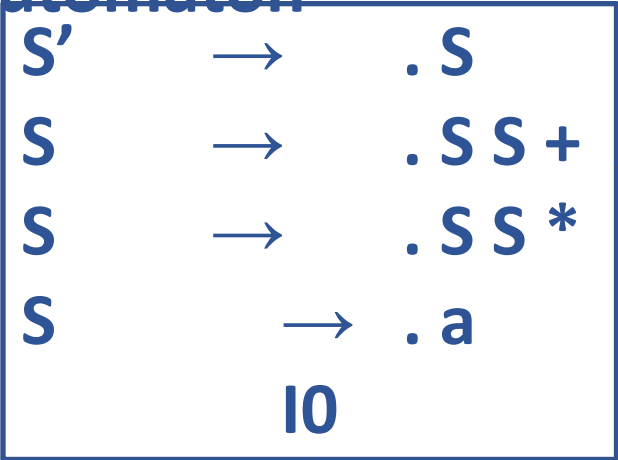
$$S \rightarrow SS+ \mid SS* \mid a$$

Augmenting and numbering the grammar

- | | | | |
|----|------|---------------|---------|
| | S' | $\rightarrow$ | S |
| 1. | S | $\rightarrow$ | $S S +$ |
| 2. | S | $\rightarrow$ | $S S *$ |
| 3. | S | $\rightarrow$ | a |

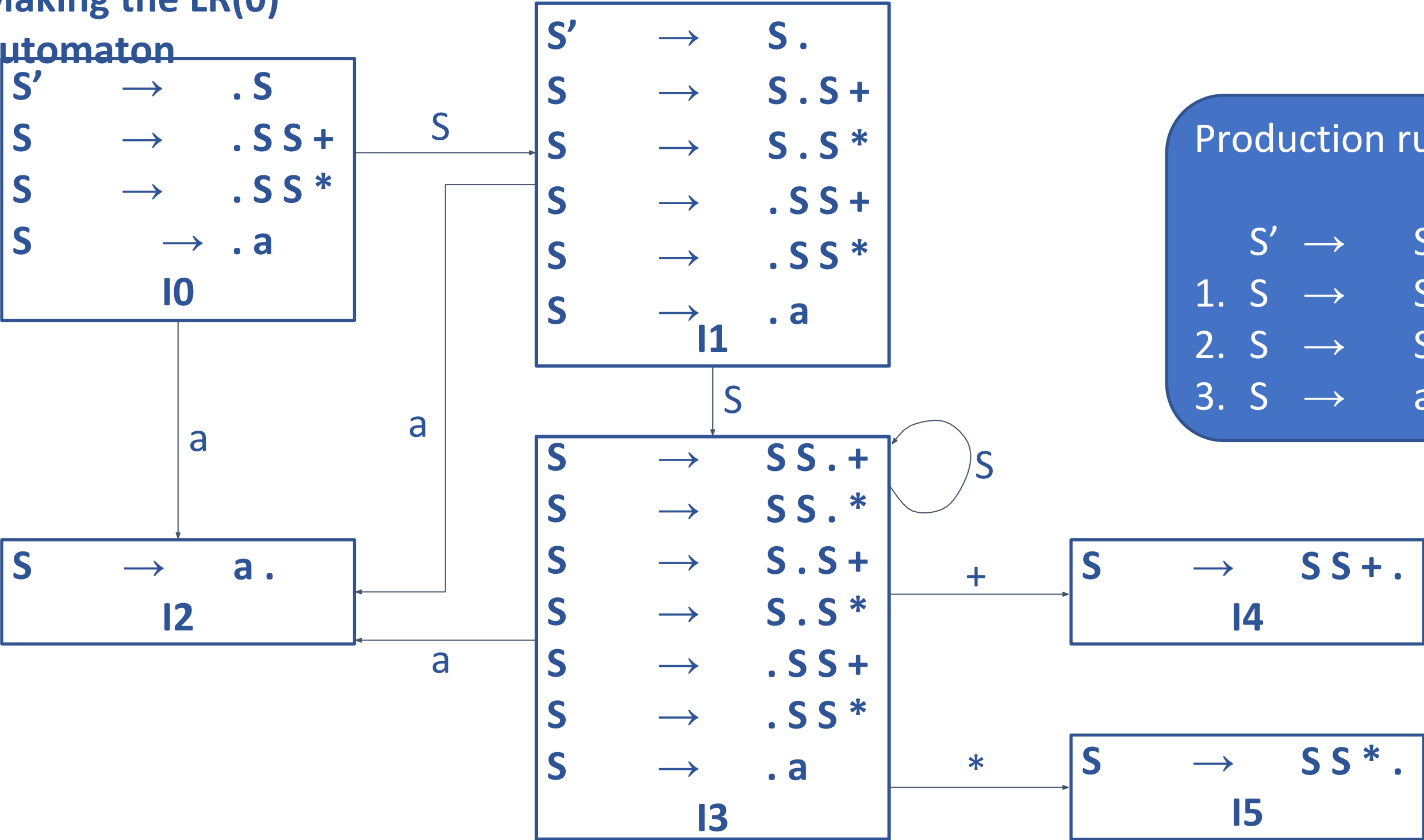
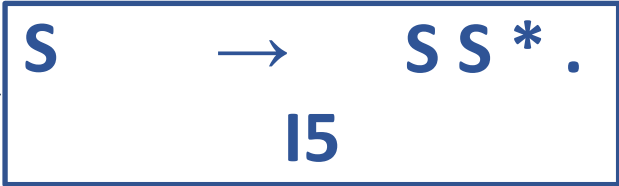
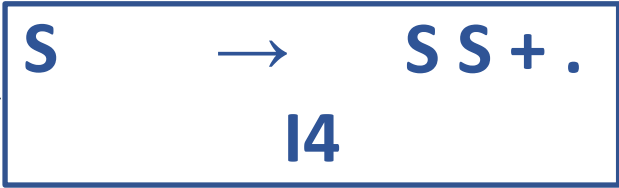
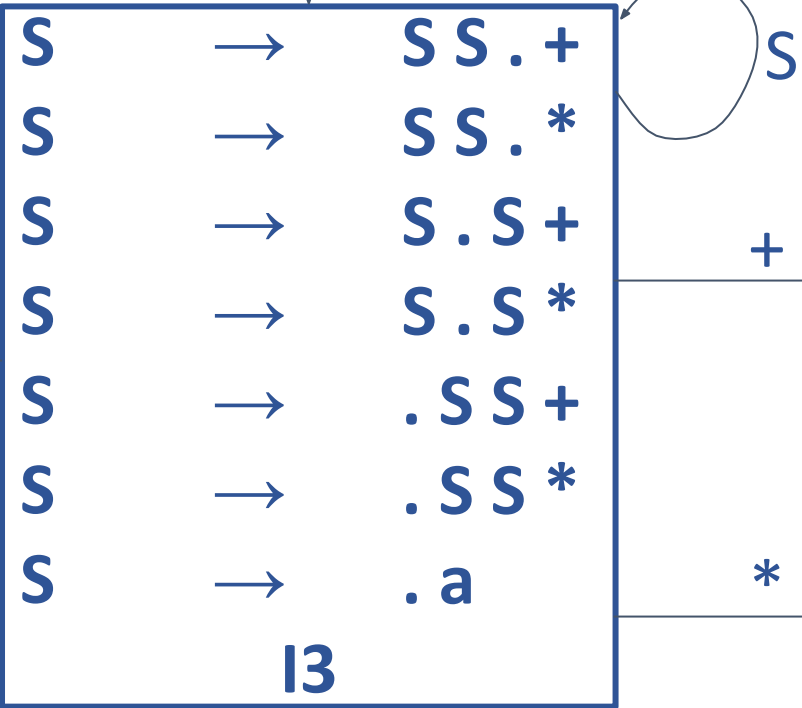
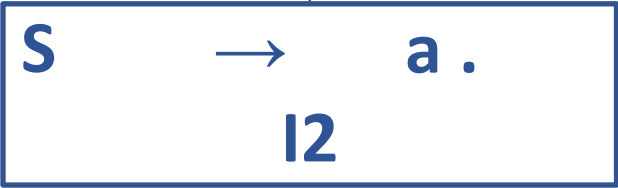
$\text{follow}(S) = \{\$, +, *, a\}$

Making the LR(0) automaton



Production rules:

- $S' \rightarrow S$
- 1. $S \rightarrow S S +$
- 2. $S \rightarrow S S *$
- 3. $S \rightarrow a$



Making the LR(0) parsing
table

State	Action				Goto
	+	*	a	\$	S
0			s ₂		1
1			s ₂	Accept	3
2	r ₃	r ₃	r ₃	r ₃	
3	s ₄	s ₅	s ₂		3
4	r ₁	r ₁	r ₁	r ₁	
5	r ₂	r ₂	r ₂	r ₂	

Production rules:

S'

→

S

1.

→

S S +

S

Note that since the follow of S includes all terminals and ‘\$’, the SLR parsing table will be exactly the same as LR(0) parsing table. The grammar belongs to both, LR(0) and SLR, since there are no conflicts!

Parsing a string - Example 2

Let us now parse the
string: $w = \text{“}a\ a\ +\ a\ a\ \text{”}$
using the common parsing table we have developed as part
of the example.

The initial configuration is as follows:

Stack	I/p buffer	Action
\$ 0	a a + a a * \$	

Let us start parsing the string step-by-
step.

Parsing a string - Example 2

Stack	I/p buffer	Action
\$ 0	a a + a a * \$	$A[0, a] = s_2$ Push 'a', 2 onto stack
\$ 0 a 2	a + a a * \$	$A[2, a] = r_3$ ($S \rightarrow a$) Pop 2, 'a' from stack Push 'S' onto stack Push Goto[0, S] = 1
\$ 0 S 1	a + a a * \$	$A[1, a] = s_2$ Push 'a', 2 onto stack
\$ 0 S 1 a 2	+ a a * \$	$A[2, +] = r_3$ ($S \rightarrow a$) Pop 2, 'a' from stack Push 'S' onto stack Push Goto[1, S] = 3

Production rules:

- $S' \rightarrow S$
1. $S \rightarrow S S +$
2. $S \rightarrow S S *$
3. $S \rightarrow a$

Parsing a string - Example 2

Stack	I/p buffer	Action
\$ 0 S 1 S 3	+ a a * \$	$A[3, +] = s_4$ Push '+', 4 onto stack
\$ 0 S 1 S 3 + 4	a a * \$	$A[4, a] = r_1 (S \rightarrow S S +)$ Pop 4, '+', 3, 'S', 1, 'S' Push 'S' onto stack Push Goto[0, S] = 1
\$ 0 S 1	a a * \$	$A[1, a] = s_2$ Push 'a', 2 onto stack
\$ 0 S 1 a 2	a * \$	$A[2, a] = r_3 (S \rightarrow a)$ Pop 2, 'a' from stack Push 'S' onto stack Push Goto[1, S] = 3
:	:	:

Production rules:

- $S' \rightarrow S$
1. $S \rightarrow S S +$
2. $S \rightarrow S S *$
3. $S \rightarrow a$

Parsing a string - Example 2

Stack	I/p buffer	Action
\$ 0 S 1 S 3	a * \$	$A[3, a] = s_2$ Push 'a', 2 onto stack
\$ 0 S 1 S 3 a 2	* \$	$A[2, *] = r_3 (S \rightarrow a)$ Pop 2, 'a' from stack Push 'S' onto stack Push Goto[3, S] = 3
\$ 0 S 1 S 3 S 3	* \$	$A[3, *] = s_5$ Push '*', 5
\$ 0 S 1 S 3 S 3 * 5	\$	$A[5, \$] = r_2 (S \rightarrow S S *)$ Pop 5, '*', 3, 'S', 3, 'S' Push 'S' onto stack Push Goto[1, S] = 3
\$ 0 S 1 S 3	\$	$A[3, \$] = \text{Reject}$

production rules:

- $S' \rightarrow S$
1. $S \rightarrow S S +$
2. $S \rightarrow S S *$
3. $S \rightarrow a$

Construct the LR(0) and SLR parsing tables for
the following grammar:

$S \rightarrow L = R \mid R$

$L \rightarrow * R \mid id$

$R \rightarrow L$

Example 3 (continued)

Augmenting and numbering the grammar

- S'
- $\rightarrow$
- S
1.
- S
- $\rightarrow$
- $L = R$
2.
- S
- $\rightarrow$
- R
3.
- L
- $\rightarrow$
- $* R$
4.
- L
- $\rightarrow$
- id
5.
- R
- $\rightarrow$
- L

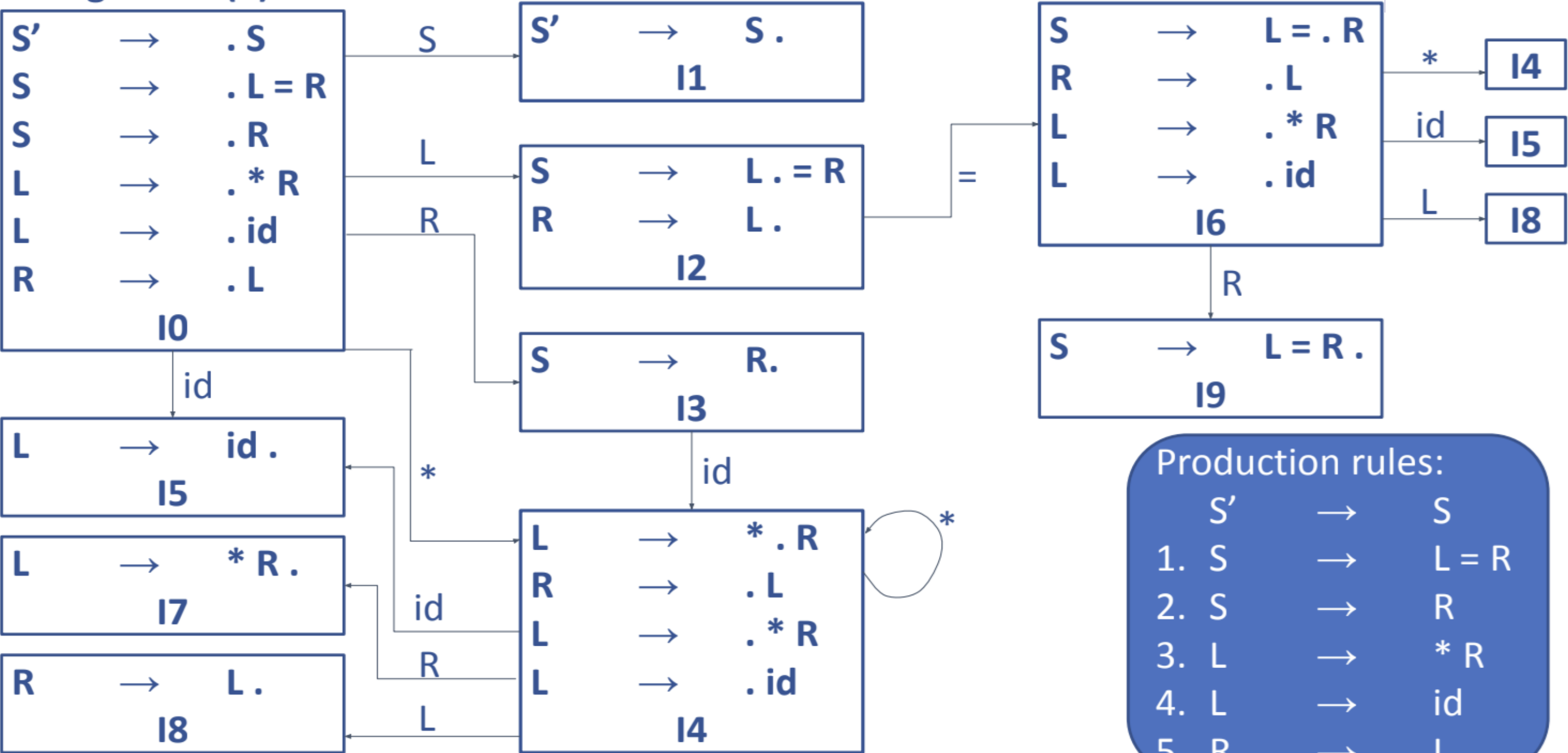
$\text{follow}(S) = \{\$, \}$

$\text{follow}(L) = \{=, \$\}$

$\text{follow}(R) = \{\$, =\}$

S	L	R
\$	=	\$
	\$	=

Making the LR(0) automaton



Note that there are shift-reduce conflicts in states I2 and I3.

State	ACTION				GOTO		
	*	=	id	\$	S	L	R
0	s ₄		s ₅		1	2	3
1				Accept			
2	r ₅	s ₆ / r ₅	r ₅	r ₅			

Production rules:

- S' → S
1. S → L = R
2. S → R
3. L → * R
4. L → id
5. R → L

Let us, however, continue making the LR(0) parsing table

State	ACTION				GOTO		
	*	=	id	\$	S	L	R
0	s ₄		s ₅		1	2	3
1				Accept			
2	r ₅	s ₆ / r ₅	r ₅	r ₅			
3	r ₂	r ₂	s ₄ / r ₂	r ₂			
4	s ₄		s ₅			8	7
5	r ₄	r ₄	r ₄	r ₄			
6	s ₄		s ₅			8	9
7	r ₃	r ₃	r ₃	r ₃			
8	r ₅	r ₅	r ₅	r ₅			
9	r ₁	r ₁	r ₁	r ₁			

Production rules:

- S'
- $\rightarrow$
- S
1.
- S
- $\rightarrow$
- $L = R$
2.
- S
- $\rightarrow$
- R
3.
- L
- $\rightarrow$
- $* R$
4.
- L
- $\rightarrow$
- id
5.
- R
- $\rightarrow$
- L

Example 3 (continued)

Let us now make the SLR(1) parsing table

State	ACTION				GOTO		
	*	=	id	\$	S	L	R
0	s ₄		s ₅		1	2	3
1				Accept			
2		s ₆ / r ₅		r ₅			

s/r conflict occurs in because follow(R) is same as the shifted terminal '='

Production rules:

- S' → S
1. S → L = R
2. S → R
3. L → * R
4. L → id
5. R → L

S	L	R
\$	=	\$
	\$	=

State	ACTION				GOTO		
	*	=	id	\$	S	L	R
0	s ₄		s ₅		1	2	3
1				Accept			
2		s ₆ / r ₅		r ₅			
3				r ₂			
4	s ₄		s ₅			8	7
5		r ₄		r ₄			
6	s ₄		s ₅			8	9
7		r ₃		r ₃			
8		r ₅		r ₅			
9				r ₁			

Production rules:

S'

→

S

1.

→

L = R

S

→

R

2.

S	L	R
\$	=	\$
	\$	=

What went wrong?

Even though the SLR(1) parser removed one of the two s/r conflicts that were seen in LR(0) parser, the other s/r conflict could not be removed. SLR(1) is not very powerful.

This means that we need more powerful parsers: those which will look into particular productions/derivations and not generalise things by simply looking into the follow table.

In the next lecture, we will look into parsers such as CLR(1)/LR(1) and LALR(1).

Construct the LR(0) and SLR parsing tables for the following grammar:

$$\begin{array}{lcl} S & \rightarrow & A a A b \quad | \quad B b B a \\ A & \rightarrow & \lambda \\ B & \rightarrow & \lambda \end{array}$$

Augmenting and numbering the grammar

- S'
- $\rightarrow$
- S
1.
- S
- $\rightarrow$
- $AaAb$
2.
- S
- $\rightarrow$
- $BbBa$
3.
- A
- $\rightarrow$
- λ
4.
- B
- $\rightarrow$
- λ

$\text{follow}(S) = \{\$, \}$
 $\text{follow}(A) = \{a, b\}$
 $\text{follow}(B) = \{a, b\}$

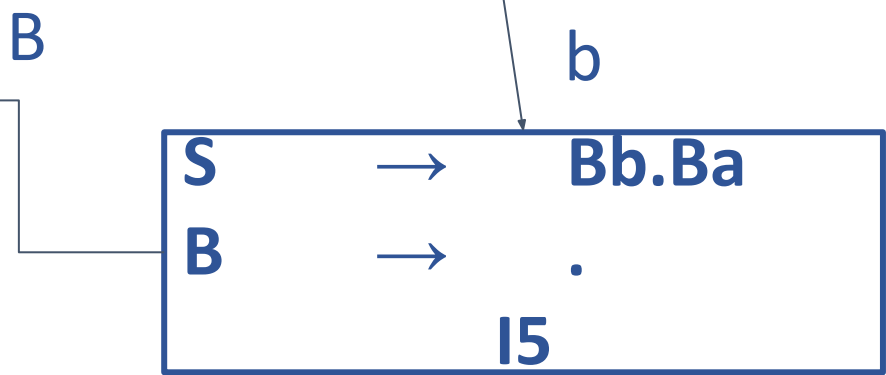
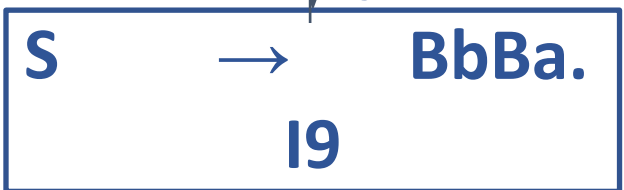
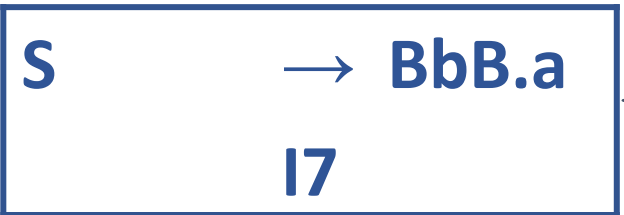
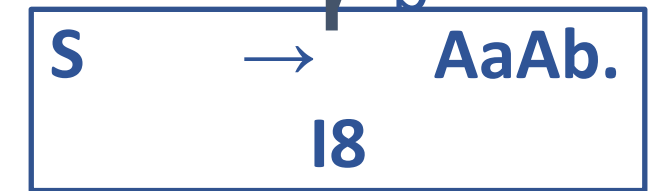
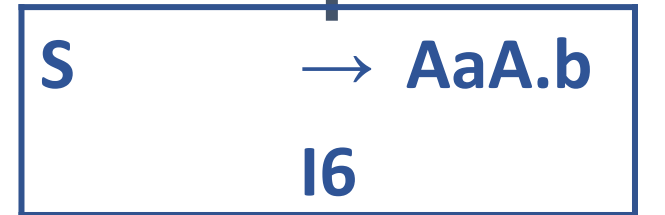
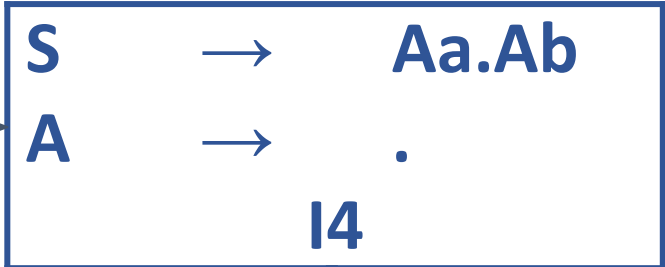
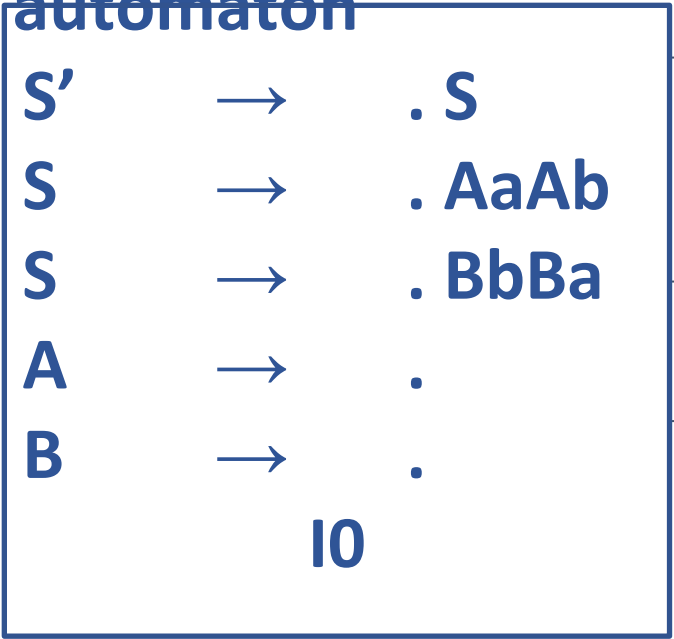
S	A	B
\$	a	a
	b	b

Making the LR(0) automaton

S'	$\rightarrow$	$\cdot S$
S	$\rightarrow$	$\cdot AaAb$
S	$\rightarrow$	$\cdot BbBa$
A	$\rightarrow$	$\cdot$
B	$\rightarrow$	$\cdot$
IO		

There are 2 final items in this itemset which leads to a reduce-reduce conflict.
Hence, the given grammar is not in LR(0).

Making the LR(0) automaton



Production rules:

- $S' \rightarrow S$
- 1. $S \rightarrow AaAb$
- 2. $S \rightarrow BbBa$
- 3. $A \rightarrow \lambda$
- 4. $B \rightarrow \lambda$

Let us now make the SLR(1) parsing table

State	ACTION			GOTO		
	a	b	\$	S	A	B
0	r ₃ /r ₄	r ₃ /r ₄		1	2	3
1			Accept			

Production rules:

$S' \rightarrow S$

1. $S \rightarrow AaAb$

2. $S \rightarrow BbBa$

3. $A \rightarrow \lambda$

4. $B \rightarrow \lambda$

Reduce-reduce conflict occurs
as follow(A)=follow(B)

S	A	B
\$	a	a
	b	b

State	ACTION			GOTO		
	a	b	\$	S	A	B
0	r ₃ /r ₄	r ₃ /r ₄		1	2	3
1			Accept			
2	s ₄					
3		s ₅				
4	r ₃	r ₃	r ₃		6	
5	r ₄	r ₄	r ₄			7
6		s ₈				
7	s ₉	r ₃	r ₃			
8			r ₁			
9			r ₂			

Production rules:

- $S' \rightarrow S$
- 1. $S \rightarrow AaAb$
- 2. $S \rightarrow BbBa$
- 3. $A \rightarrow \lambda$
- 4. $B \rightarrow \lambda$

Construct the LR(0) and SLR parsing tables for the following grammar:

$$\begin{array}{lcl} S & \rightarrow & A a \mid b A c \mid d c \mid b d \\ A & \rightarrow & d \end{array}$$

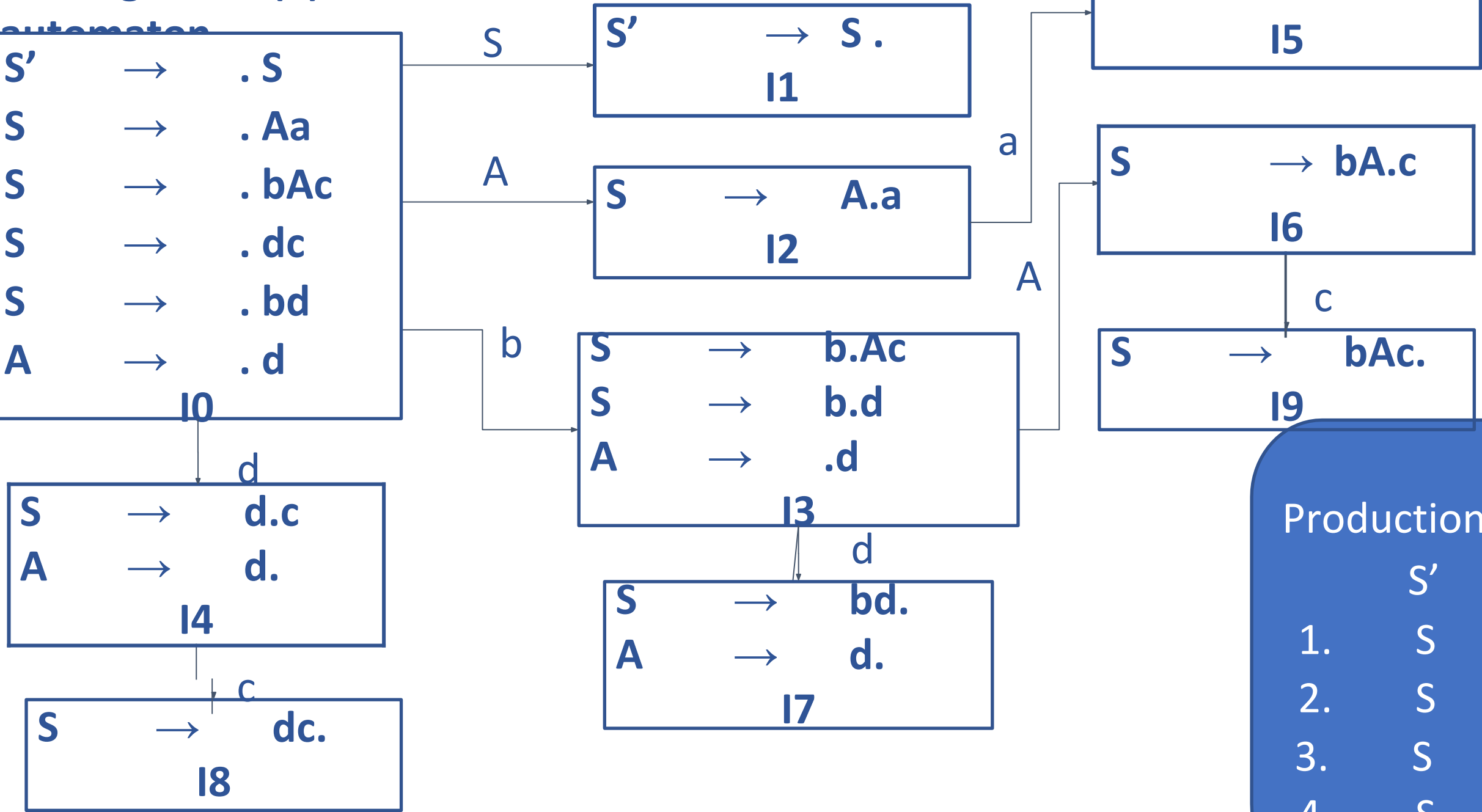
Augmenting and numbering the grammar

- S'
- $\rightarrow$
- S
1.
- S
- $\rightarrow$
- Aa
2.
- S
- $\rightarrow$
- bAc
3.
- S
- $\rightarrow$
- dc
4.
- S
- $\rightarrow$
- bd
5.
- A
- $\rightarrow$
- d

$\text{follow}(S) = \{\$, \}$
 $\text{follow}(A) = \{a, c\}$

S	A
\$	a
	c

Making the LR(0)



Production rules:

- $S' \rightarrow S$
- 1. $S \rightarrow Aa$
- 2. $S \rightarrow bAc$
- 3. $S \rightarrow dc$
- 4. $S \rightarrow bd$
- 5. $A \rightarrow d$

State	ACTION					GOTO	
	a	b	c	d	\$	S	A
0		s ₃		s ₄		1	2
1					Accept		
2	s ₅						
3				s ₆			5
4	r ₅	r ₅	s ₈ /r ₅	r ₅	r ₅		
5	r ₁	r ₁	r ₁	r ₁	r ₁		
6			s ₉				
7	r ₄ /r ₅	r ₄ /r ₅	r ₄ /r ₅	r ₄ /r ₅	r ₄ /r ₅		
8	r ₃	r ₃	r ₃	r ₃	r ₃		
9	r ₂	r ₂	r ₂	r ₂	r ₂		

Production rules:

$S' \rightarrow S$

1. $S \rightarrow Aa$

2. $S \rightarrow bAc$

3. $S \rightarrow dc$

4. $S \rightarrow bd$

5. $A \rightarrow d$

S	A
\$	a
	c

State	ACTION					GOTO	
	a	b	c	d	\$	S	A
0		s ₃		s ₄		1	2
1					Accept		
2	s ₅						
3				s ₆			5
4	r ₅		s ₈ /r ₅				
5					r ₁		
6			s ₉				
7	r ₅		r ₅		r ₄		
8	r ₃	r ₃	r ₃	r ₃	r ₃		
9	r ₂	r ₂	r ₂	r ₂	r ₂		

Production rules:

S' → S

1. S → Aa

2. S → bAc

3. S → dc

4. S → bd

5. A → d

S	A
\$	a
	c

Construct the LR(0) and SLR parsing tables for the following grammar:

$S \rightarrow A a \mid b A c \mid B c \mid b B a$

$A \rightarrow d$

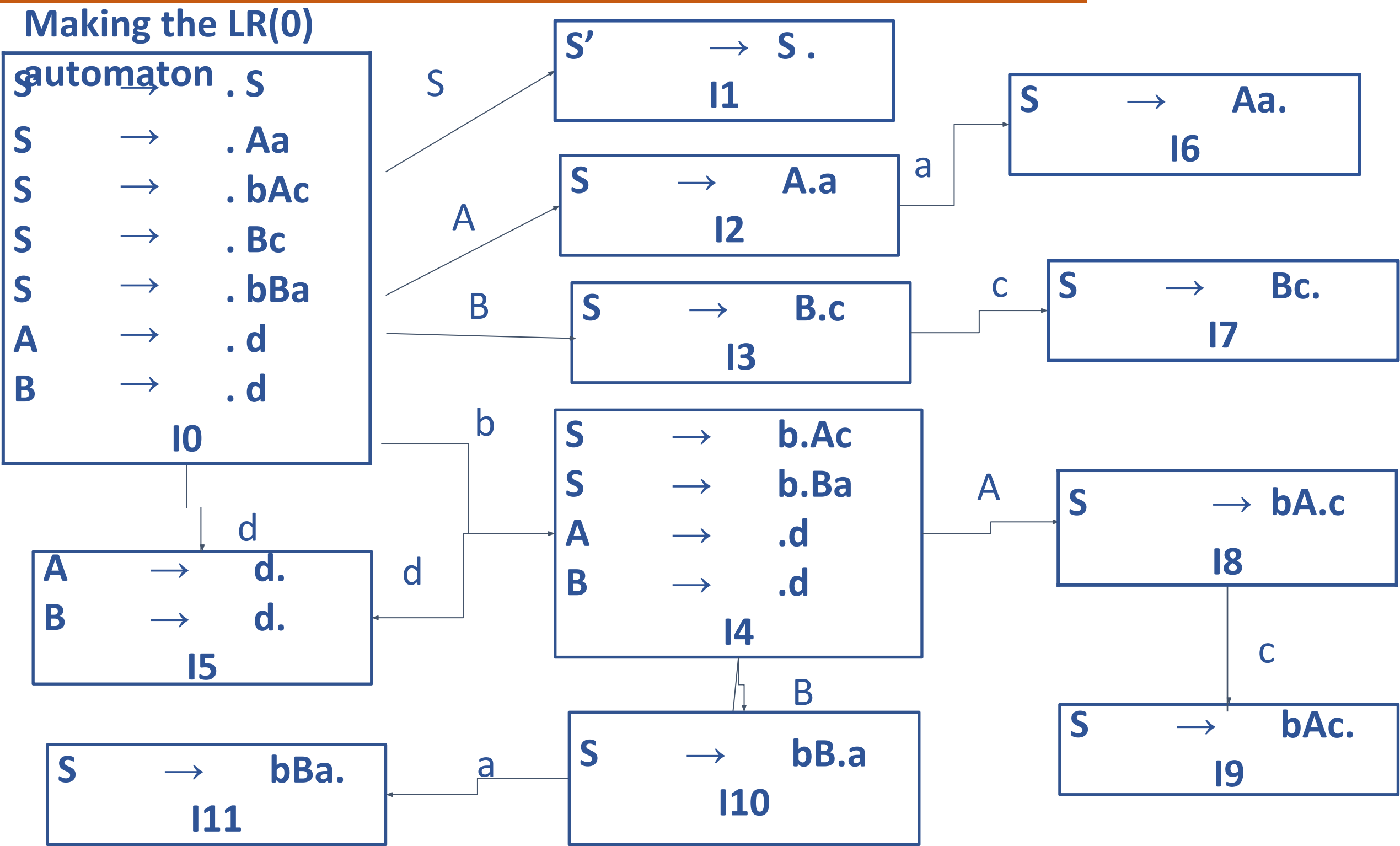
$B \rightarrow d$

Augmenting and numbering the grammar

- $S' \rightarrow S$
1. $S \rightarrow Aa$
2. $S \rightarrow bAc$
3. $S \rightarrow Bc$
4. $S \rightarrow bBa$
5. $A \rightarrow d$
6. $B \rightarrow d$

$\text{follow}(S) = \{\$, \}$
 $\text{follow}(A) = \{a, c\}$
 $\text{follow}(B) = \{a, c\}$

S	A	B
\$	a	a
	c	c



Let us now make the LR(0) parsing

state	ACTION					GOTO		
	a	b	c	d	\$	S	A	B
0		s ₄		s ₅		1	2	3
1					Accept			
2	s ₆							
3			s ₇					
4				s ₅		8	10	
5	r ₅ /r ₆	r ₅ /r ₆	r ₅ /r ₆	r ₅ /r ₆	r ₅ /r ₆			

Reduce-reduce conflict occurs at l4 as there are two final items.

Production rules: S' → S

1. S → Aa

2. S → bAc

3. S → Bc

4. S → bBa

5. A → d

S	A	B
\$	a	a
	c	c

Let us now make the LR(0) parsing

state	ACTION					GOTO		
	a	b	c	d	\$	S	A	B
0		s ₄		s ₅		1	2	3
1					Accept			
2	s ₆							
3			s ₇					
4				s ₅		8	10	
5	r ₅ /r ₆		r ₅ /r ₆					

Reduce-reduce conflict occurs
as follow(A)=follow(B)

Production
rules: S' →
S

1. S → Aa

2. S → bAc

3. S → Bc

4. S → bBa

5. A → d

S	A	B
\$	a	a
	c	c



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Lavitra Kshitij
Madan

Compiler Design

Unit 2

Table-driven BUP

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, we will learn about:

- The need for LR(1)
- LR(1) automaton and LR(1) items
- CLR Parsers (with example problem)
- LALR Parsers
- Conflicts in LALR and CLR parsers

Analysis of SLR(1) and the need for LR(1)

Analysis of SLR(1)

- Exploits lookahead in a small space.
- Small automaton – same number of states as in as LR(0).
- Works on many more grammars than LR(0)
- Too weak for most grammars: lose context from not having extra states.

In the previous lecture, we saw the need for parsers that are more powerful than SLR. The need is for those parsers that look into particular derivations while looking-ahead.

LR(1) Automata and LR(1) Item

LALR (LookAhead LR) and CLR (Canonical LR) parsers use **LR(1) automata** because these parsers use **LR(1) items**.

These make use of canonical collection of LR(1) items.

While an 'LR(0) automata' uses 'LR(0) item' or simply 'item', that looks something like:

$$A \rightarrow X . Y Z$$

An 'LR(1) item' is made up of two

parts: LR(0) item + Look-ahead

For example:

$$A \rightarrow X . Y Z , \{a, b\}$$

Whenever we have a final item in a

state, e.g.: $S \rightarrow a A . , \{a,$

$b\}$ lookahead means, the reduce move in the
place lookahead symbols

– a, b

rather than placing them in $\rightarrow$ entire row (as in

LR(0)) or

$\rightarrow$ follow(LHS) (as in SLR)

When using LR(1), we start with:



When using LR(0), we start with:



A snapshot of the table looks like this for both the cases:

State	Action	Goto
	\$	S
1	Accept	

Bottom-up predictive parsing with

- L: Left-to-right scan
- R: Rightmost derivation in Reverse
- (1): One token lookahead
- Substantially more powerful than the other methods we have covered so far
- Tries to more intelligently find lookahead token at each step by

The Intuition behind LR(1)

- Guess which series of productions we are reversing.
- Use this information to maintain information about what lookahead to expect.
- When deciding whether to shift or reduce, use lookahead to disambiguate.

The Power of LR(1)

- Any LR(0) grammar is LR(1).
- Any LL(1) grammar is LR(1).
- Any deterministic CFL (a CFL parseable by a deterministic pushdown automaton) has an LR(1) grammar.

LR(1) Automata are Huge; Rarely used in Practice

- In a grammar with n terminals, could in theory be $O(2n)$ times as large as the LR(0) automaton.
- Replicate each state with all $O(2n)$ possible lookaheads.
- LR(1) tables for practical programming languages can have hundreds of thousands or even millions of states.
- Consequently, LR(1) parsers are rarely used in practice.

Suppose we have an LR(1)

item, $A \rightarrow \alpha . B \beta , a$ – after dot, we have a NT (B),
hence we introduce the productions of B

$B \rightarrow \square$
 $\cdot Y ,$

– The lookahead for this item is calculated from $A \rightarrow \alpha . B \beta , a$ [Parent item] [Child item]
as because of this item, $B \rightarrow \cdot$ [Child item]
Y was added.
The lookahead at this point = first (βa)

We start with the augmented production and with lookahead '\$', i.e.,

$$S' \rightarrow . S , \$$$

Consider the following

grammar:

$$\begin{array}{ll} \rightarrow & L = R \mid R \\ L & \rightarrow * R \mid id \\ R & \rightarrow L \end{array}$$

Note: We have already seen that this grammar is not in SLR

Consider the following

grammar:

$$\begin{array}{lll} \rightarrow & L = R \mid R \\ L & \rightarrow & * R \mid \text{id} \\ R & \rightarrow & L \end{array}$$

Augmenting and numbering the

grammar: $S' \rightarrow S$

1. $S \rightarrow L = R$
2. $S \rightarrow R$
3. $L \rightarrow * R$
4. $L \rightarrow \text{id}$
5. $R \rightarrow L$

Example 21.1 solution (continued)

Step 1: Starting

$$S' \rightarrow .S, \$$$

Step 2-4: Repeatedly adding productions of NTs after “.”s Recall that

Look-ahead(child-item) = follow(NT) in the parent item

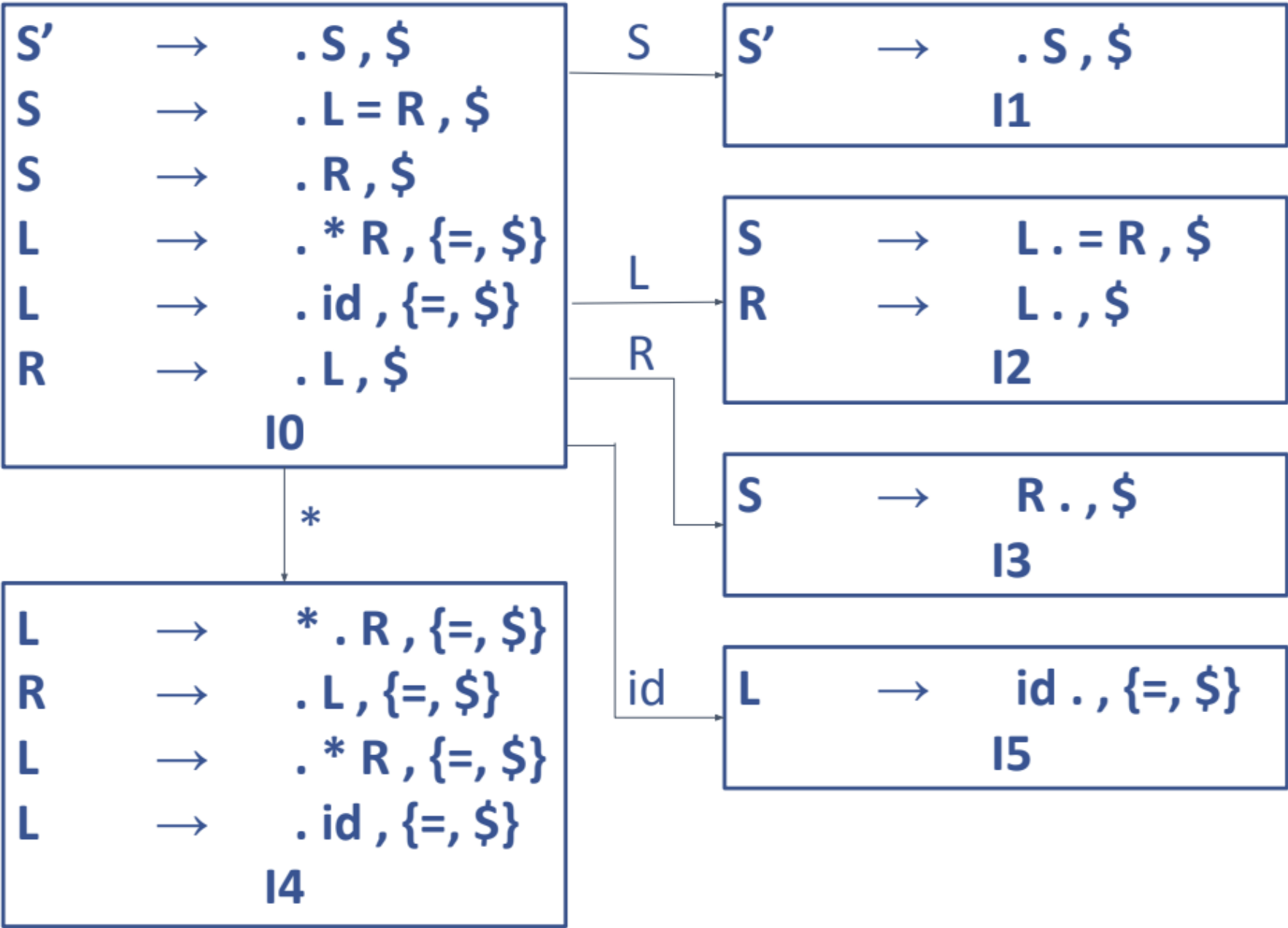
S'	$\rightarrow$	$.S, \$$
S	$\rightarrow$	$.L = R, \$$
S	$\rightarrow$	$.R, \$$
L	$\rightarrow$	$. * R, \{=, \$\}$
L	$\rightarrow$	$. id, \{=, \$\}$
R	$\rightarrow$	$id . L, \$$

Production rules:

- | | | |
|-----|------------------|---------|
| | $S' \rightarrow$ | S |
| 1. | $\rightarrow$ | $L = R$ |
| S | | |
| 2. | $\rightarrow$ | R |

Example 21.1 solution (continued)

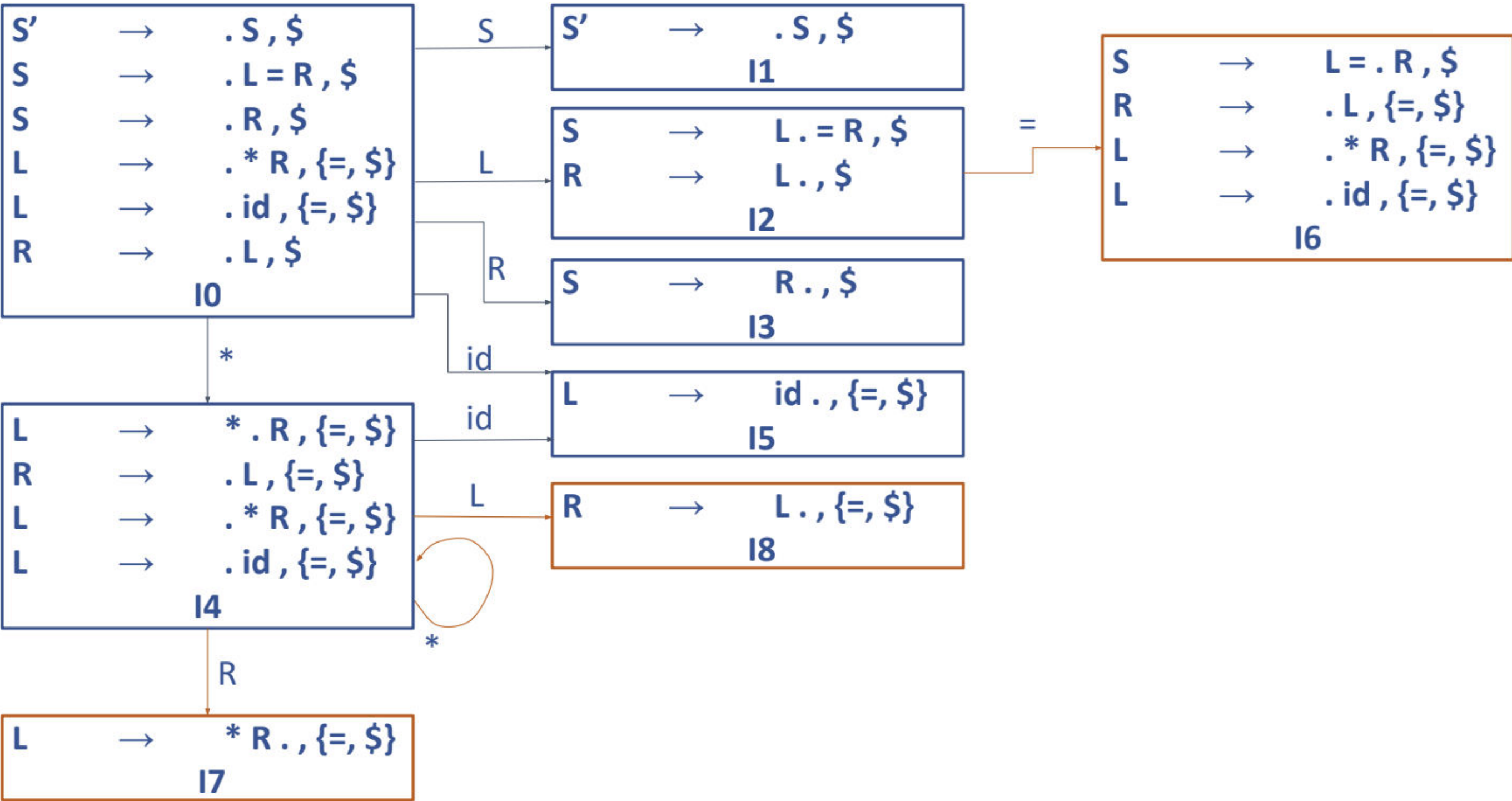
Step 5: Transitions using the goto



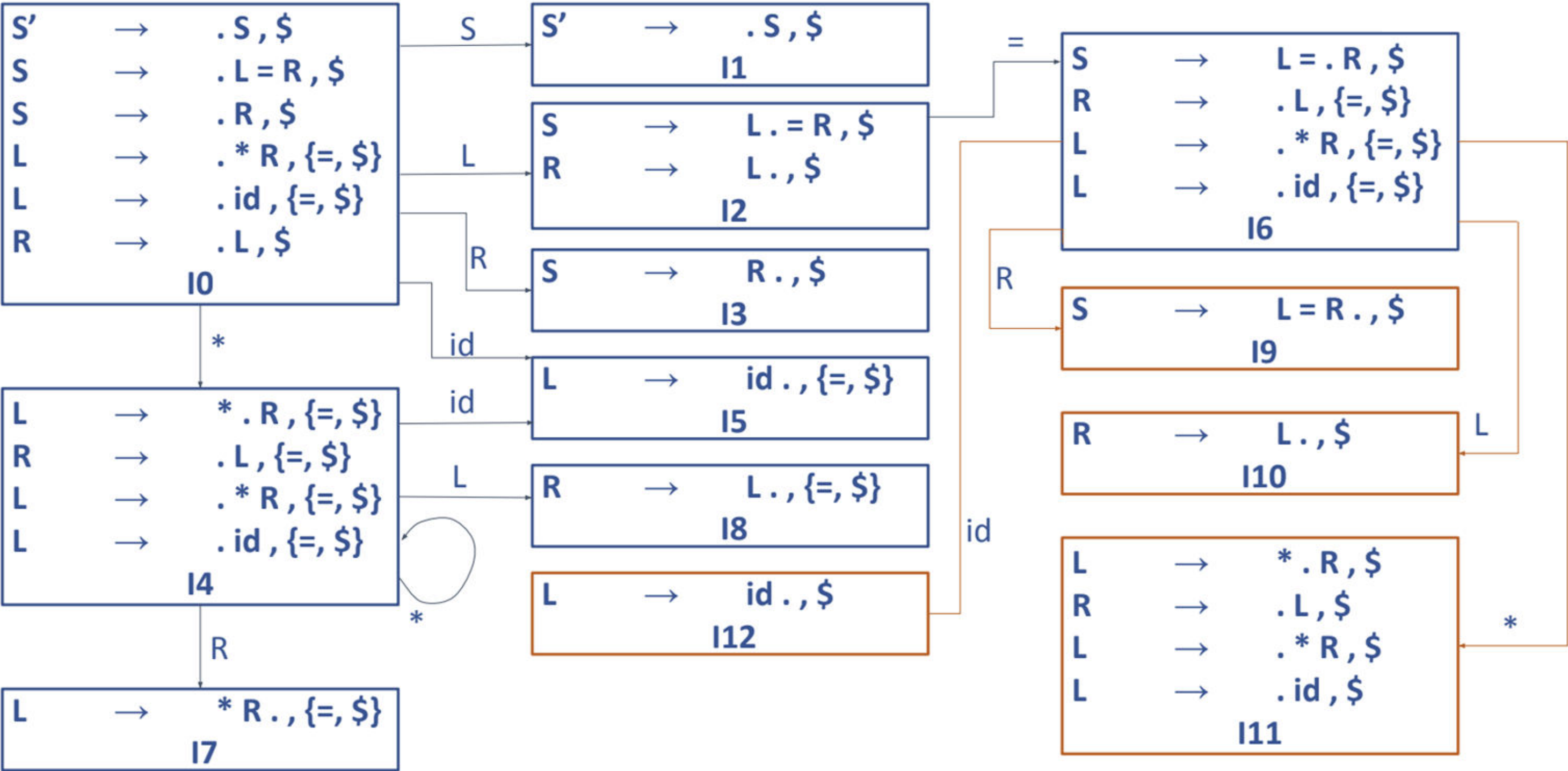
Production rules:

- $S' \rightarrow S$
- 1. $S \rightarrow L = R$
- 2. $S \rightarrow R$
- 3. $L \rightarrow * R$
- 4. $L \rightarrow id$
- 5. $R \rightarrow L$

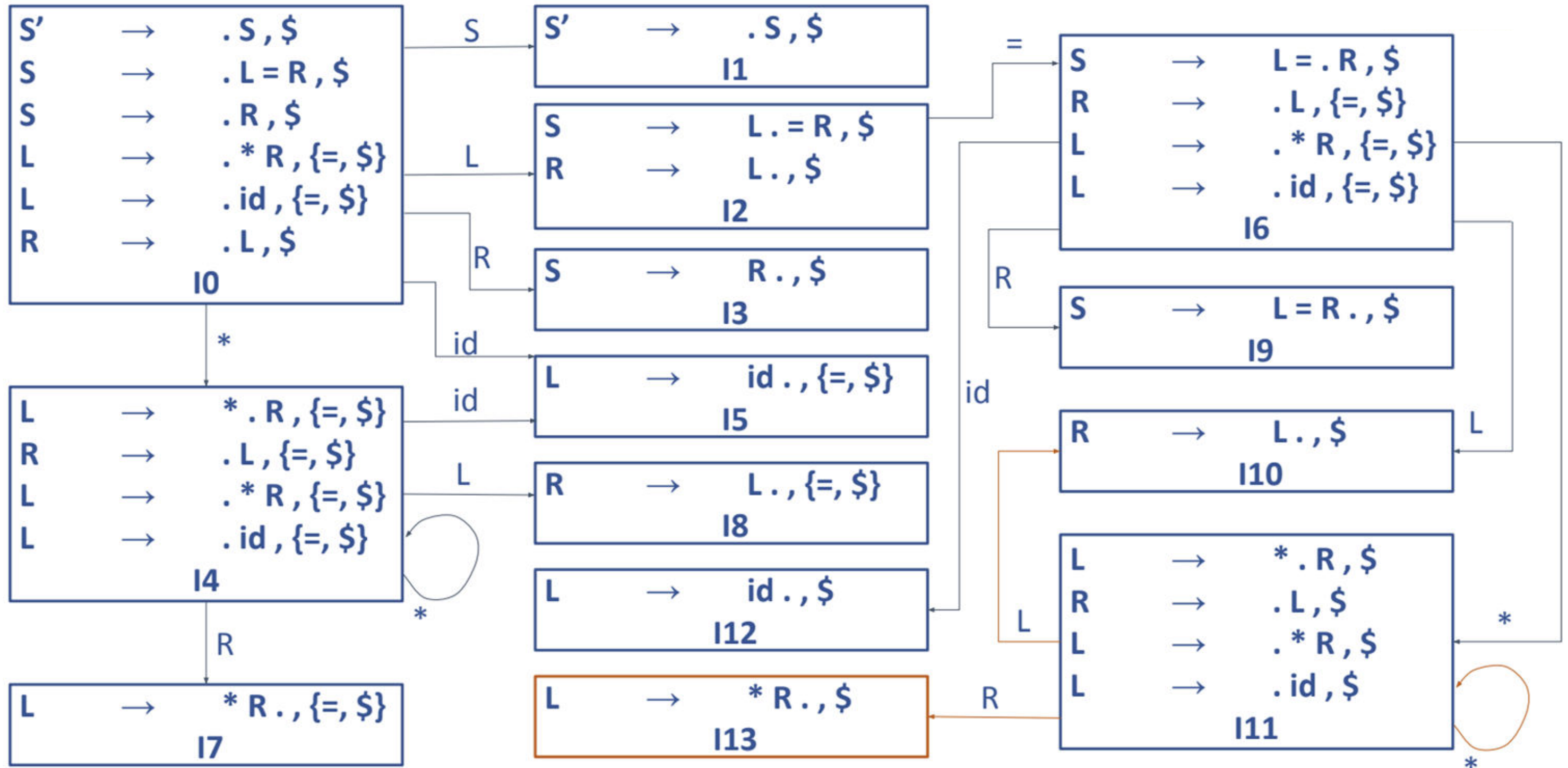
Example 21.1 solution (continued) : step 6



Example 21.1 solution (continued) : step 6



Example 21.1 solution (continued) : step 6



Example 21.1 solution (continued) - CLR parsing table

State	Action				Goto		
	*	id	=	\$	S	L	R
0							

Let us fill the CLR parsing table now.

State	Action				Goto		
	*	id	=	\$	S	L	R
0	s ₄	s ₅			1	2	3

Example 21.1 solution (continued) - CLR parsing table

State	Action				Goto		
	*	id	=	\$	S	L	R
0	s ₄	s ₅			1	2	3
1				Accept			
2			s ₆	r ₅			
3				r ₂			
4	s ₄	s ₅				8	7
5			r ₄	r ₄			

Example 21.1 solution (continued) - CLR parsing table

State	Action				Goto		
	*	id	=	\$	S	L	R
0	s ₄	s ₅			1	2	3
1				Accept			
2			s ₆	r ₅			
3				r ₂			
4	s ₄	s ₅				8	7
5			r ₄	r ₄			
6	s ₁₁	s ₁₂				10	9
7			r ₃	r ₃			
: : : : : : :							

Table continued on next slide.

Example 21.1 solution (continued) - CLR parsing table

State	Action				Goto		
	*	id	=	\$	S	L	R
	:	:	:	:	:		
8			r ₅	r ₅			

Example 21.1 solution (continued) - CLR parsing table

State	Action				Goto		
	*	id	=	\$	S	L	R
	:	:	:	:	:	:	:
8			r ₅	r ₅			
9				r ₁			
10				r ₅			
11	s ₁₁	s ₁₂				10	13
12				r ₄			

Example 21.1 solution (continued) - CLR parsing table

State	Action				Goto		
	*	id	=	\$	S	L	R
	:	:	:	:	:	:	:
8			r ₅	r ₅			
9				r ₁			
10				r ₅			
11	s ₁₁	s ₁₂				10	13
12				r ₄			
13				r ₃			

No conflicts
occured!

Example 21.1 solution (continued) - Merging states

Notice that there are some states with the same LR(0) itemsets but different lookaheads. All such pairs are mentioned below:

- I4 – I11
- I5 – I12
- I7 – I13
- I8 – I10

So, we merge them. For example, merging I4 and I11:

L → * . R , { = , \$ }

R → . L , { = , \$ }

L → . * R , { = , \$ }

L → . id , { = , \$ }

Example 21.1 solution (continued) - Merging states

LALR table will be smaller than CLR table in this case.

Note : While merging, check for any problems. Example-

$$M [4, *] = s_4$$

and

$$M [11, *] = s_{11}$$

This will not be a problem since s_{4-11} will be a new state.

Also take care of reduce-reduce conflicts.

Example 21.1 solution (continued) - LALR parsing table

State	Action				Goto		
	*	id	=	\$	S	L	R
0	s_{4-11}	s_{5-12}			1	2	3
1				Accept			
2			s_6	r_{5-12}			
3				r_2			
4 - 11	s_{4-11}	s_{5-12}				8 - 10	7 - 13
5 - 12			r_4	r_4			
6	s_{4-11}	s_{5-12}				8 - 10	9
7 - 13			r_3	r_3			
8 - 10			r_{5-12}	r_{5-12}			
9				r_1			

Alternatively, we can take the lower number for each merged state :

4 - 11 as 4
5 - 12 as 5
7 - 13 as 7

Example 21.1 (continued) - Parsing an input

Now, considering the LALR table, let us parse the following string:

w = “* id = id”

Note: While parsing there must always be a state number at the top of the stack.

Starting with the initial configuration:

Stack	I/p buffer	Action
\$ 0	* id = id \$	

Referring the table, a “*” on state 0 is “s₄”. It means that “*” and “4” must be pushed onto the stack because we read the symbol “*” and move to state 4.

Example 21.1 (continued) - Parsing an input

Stack	I/p buffer	Action
\$ 0	* id = id \$	A [0, *] = s ₄ Push * , Push 4
\$ 0 * 4	id = id \$	

Similarly, on reading “id”, the table states that the action that must be performed is “s₅” or pushing “id” and “5” onto the stack because we move to state 5.

Example 21.1 (continued) - Parsing an input

Stack	I/p buffer	Action
\$ 0	* id = id \$	A [0, *] = s ₄ Push * , Push 4
\$ 0 * 4	id = id \$	A [4, id] = s ₅ Push id , Push 5
\$ 0 * 4 id 5	= id \$	

Now, TOS is “5” and the next symbol in the input buffer is “=”. Using the table, we know that the action to take is “r₄”, i.e., we must reduce using the fourth production rule:

$L \rightarrow id$

We must pop both- symbol and state (“5” and “id”)- and push the left-hand-side of the rule (“L”) and call the Goto function on the state that was on top before pushing “L” and push its output on the stack.

Example 21.1 (continued) - Parsing an input

Stack	I/p buffer	Action
\$ 0	* id = id \$	A [0, *] = s ₄ Push * , Push 4
\$ 0 * 4	id = id \$	A [4, id] = s ₅ Push id , Push 5
\$ 0 * 4 id 5	= id \$	A [5, =] = r ₄ (L → id) Pop 5 , Pop id , Push L Push Goto [4, L] = 8
\$ 0 * 4 L 8	= id \$	

Do note that “id” was not consumed yet.
Repeat this process until all of the
input is read and we receive “Accept” in the
action column.

Example 21.1 (continued) - Parsing an input

Stack	I/p buffer	Action
\$ 0	* id = id \$	A [0, *] = s ₄ Push * , Push 4
\$ 0 * 4	id = id \$	A [4, id] = s ₅ Push id , Push 5
\$ 0 * 4 id 5	= id \$	A [5, =] = r ₄ (L → id) Pop 5 , Pop id , Push L Push Goto [4, L] = 8
\$ 0 * 4 L 8	= id \$	A [8, =] = r ₅ (R → L) Pop 8 , Pop L , Push R Push Goto [4, R] = 7
\$ 0 * 4 R 7	= id \$	A [7, =] = r ₃ (L → * R) Pop 7, Pop R, Pop 4, Pop * , Push L Push Goto [0, L] = 2
\$ 0 L 2	= id \$	

If the production rule during the reduce move has x symbols on RHS, pop 2*x symbols from stack.

Table continued on next slide.

Example 21.1 (continued) - Parsing an input

Stack	I/p buffer	Action
\$ 0 L 2	= id \$	A [2, =] = s ₆ Push = , Push 6
\$ 0 L 2 = 6	id \$	A [6, id] = s ₅ Push id , Push 5
\$ 0 L 2 = 6 id 5	\$	A [5, \$] = r ₄ (L → id) Pop 5 , Pop id , Push L Push Goto [6, L] = 8
\$ 0 L 2 = 6 L 8	\$	A [8, \$] = r ₅ (R → L) Pop 8 , Pop L , Push R Push Goto [6, R] = 9
\$ 0 L 2 = 6 R 9	\$	A [9, \$] = r ₁ (S → L = R) Pop L 2 = 6 R 9 , Push S Push Goto [0, S] = 1
\$ 0 S 1	\$	A [1, \$] = Accept

LR(1) and SLR(1)

- SLR(1) is weak because it has no contextual information.
- LR(1) is impractical because its contextual information makes the automaton too big.
- Can we retain the LR(1) contextual automaton's information without all its states?

This is where LALR comes into action.

- There may be some grammar, where there exist at least one set of two or more states that have the same LR(0) itemsets but **different look-aheads**.
- These itemsets are **merged** together to reduce the number of states.
- The resulting parsing table is the LALR parsing table.
- While merging states, there may arise some problems in the form of conflicts and thus LALR parser is weaker than CLR parser. But, it is more powerful than LR(0) parsers.
- Yacc is an LALR parser!

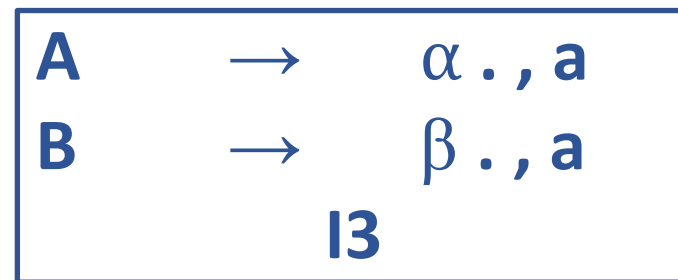
- Maintains context.
- Lookup sets based on the fine-grained LR(1) automaton.
- Each state's lookup relevant only for that state.
- Keeps the automaton small.
- Resulting automaton has same size as LR(0) automaton.

LALR(1) is Powerful!

- Every LR(0) grammar is LALR(1).
- Every SLR(1) grammar is LALR(1)
- Most(but not all) LR(1) grammars are LALR(1).

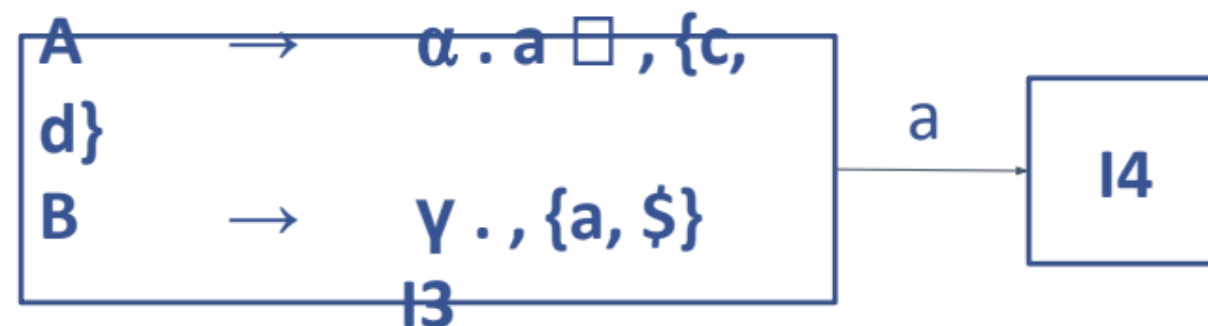
There exist a reduce-reduce (r/r) conflict in CLR if and only if there are two or more final items in a state with the same lookahead symbols.

Example:



A shift/reduce (s/r) conflict occurs in CLR when there is a final item and a shift on a terminal where that terminal also exists in the lookahead set of the final item.

Example:

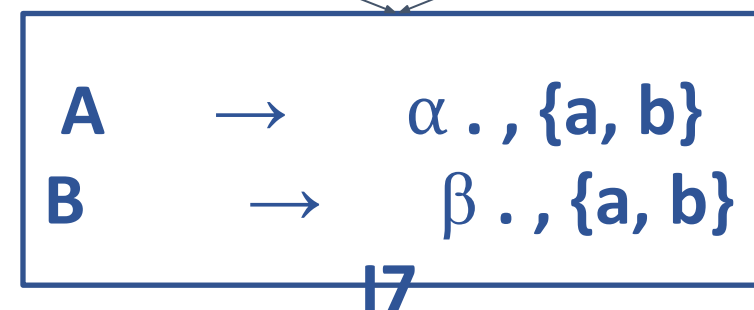
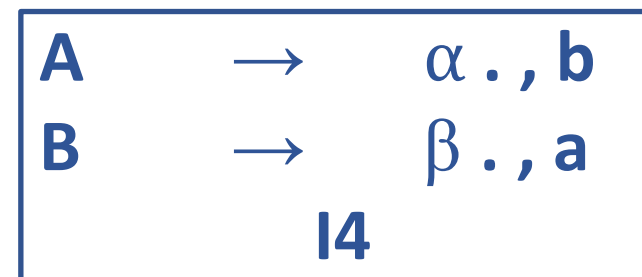
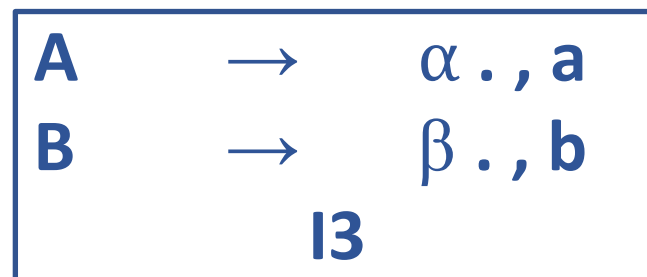


The reduce move will be placed under 'a' which also has a shift move

Merging LR(1) states can introduce a reduce/reduce conflict.

There exist a r/r conflict in LALR if and only if, upon merging, there are two or more final items in a state that have the same lookaheads.

Example:



Often these conflicts appear without any good reason; this is one limitation of LALR(1).

Merging LR(1) states cannot introduce a shift/reduce conflict.
Why?

Since the items have the same core, a shift/reduce conflict in a LALR(1) state would have to also exist in one of the LR(1) states it was merged from.

So, the same condition as that of CLR parser holds true if not already checked previously.

Example

problem

Homework problem

Consider the following

grammar: $S \rightarrow A a A b \mid B b B a$

$\rightarrow$

$A \quad \lambda$

Note: This grammar is not in

SLR $\rightarrow$

$B \quad \lambda$

$\rightarrow$



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Lavitra Kshitij
Madan

Compiler Design

Unit 2

Table-driven BUP

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, we will practice questions around:

- CLR Parsers
- LALR Parsers

We will learn about:

- Comparison between all table-driven bottom-up parsers

Consider the following

grammar: $A \rightarrow a A b \mid B b B a$

$\rightarrow$

$A \rightarrow \lambda$

Note: This grammar is not in

SLR $\rightarrow$

$B \rightarrow \lambda$

$\rightarrow$

Consider the following

grammar: $S \rightarrow A a A b \mid B$

$b B a$

$A \rightarrow \lambda$

$B \rightarrow \lambda$

Augmenting and numbering the

grammar: $S' \rightarrow S$

1. $S \rightarrow A a A b$

2. $S \rightarrow B b B a$

3. $A \rightarrow \lambda$

4. $B \rightarrow \lambda$

Example 1 solution (continued)

Step 1: Starting

Starting with the initial item.

$$S' \rightarrow . S , \$$$

Step 2: Adding productions of NTs after the dot

Recall how to calculate the lookahead for the child items The parent will specify the look-ahead.

Look-ahead(child-item) = follow(NT) in the parent item

$$S' \rightarrow . S [, \$] \Rightarrow \text{The look-ahead here is '$'}$$

S'	$\rightarrow$	$. S , \$$	}	Parent item
S	$\rightarrow$	$. A a A b , \$$	}	Child items
S	$\rightarrow$	$. B b B a , \$$		

Production rules:

- S'
1. $S \rightarrow A a A b$
 2. $S \rightarrow S \rightarrow B b B a$
 3. $A \rightarrow \lambda$
 4. $B \rightarrow \lambda$

Example 1 solution (continued)

Step 3: Repeating step 2 until no more items

For finding the lookaheads of the child items of items 2 and 3, consider the following:

$$S \rightarrow . A [a A b , \$]$$

Since “a” follows “A”, it is the look-ahead.

The items in the state now

are:

$$\begin{array}{lll} S' & \rightarrow & . S , \$ \\ S & \rightarrow & . A a A b , \$ \\ S & \rightarrow & . B b B a , \$ \\ A & \rightarrow & . , a \\ B & \rightarrow & . , b \end{array}$$

Production rules:

- S'
1. $S \rightarrow A a A b$
 2. $S \rightarrow S \rightarrow B b B a$
 3. $A \rightarrow \lambda$
 4. $B \rightarrow \lambda$

Example 1 solution (continued)

Step 4: Making closure

S'	$\rightarrow$	$. S , \$$
S	$\rightarrow$	$. A a A b , \$$
S	$\rightarrow$	$. B b B a , \$$
A	$\rightarrow$	$. , a$
B	$\rightarrow$	$. , b$
I0		

Production rules:

- $S' \rightarrow S$
- 1. $S \rightarrow A a A b$
- 2. $S \rightarrow B b B a$
- 3. $A \rightarrow \lambda$
- 4. $B \rightarrow \lambda$

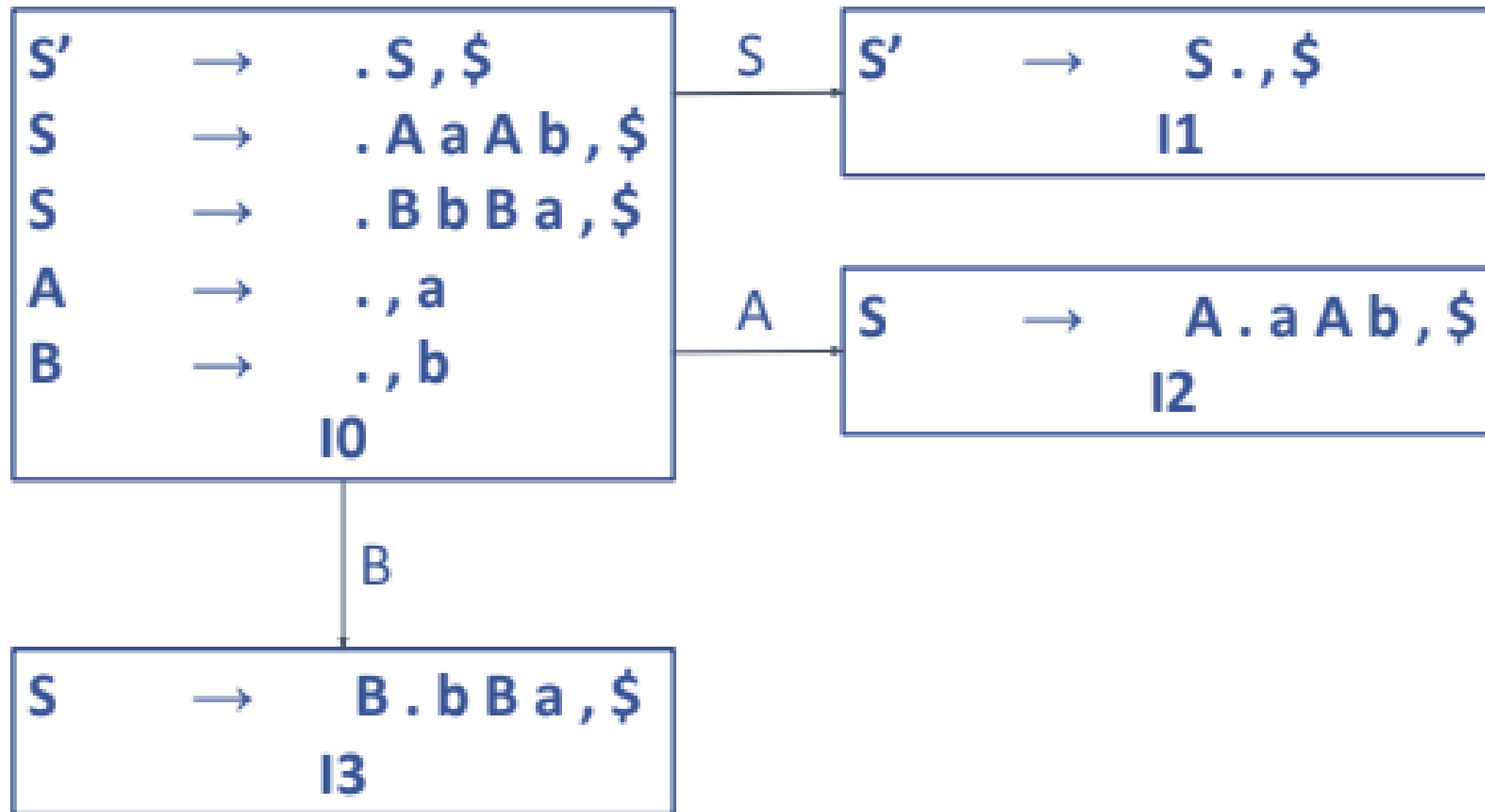
Step 5: Transitions using the goto function

This state has transitions on 'S', 'A' and 'B' to the following states:

- State I1 : $S' \rightarrow S . , \$$
- State I2 : $S \rightarrow A . a A b , \$$
- State I3 : $S \rightarrow B . b B a , \$$

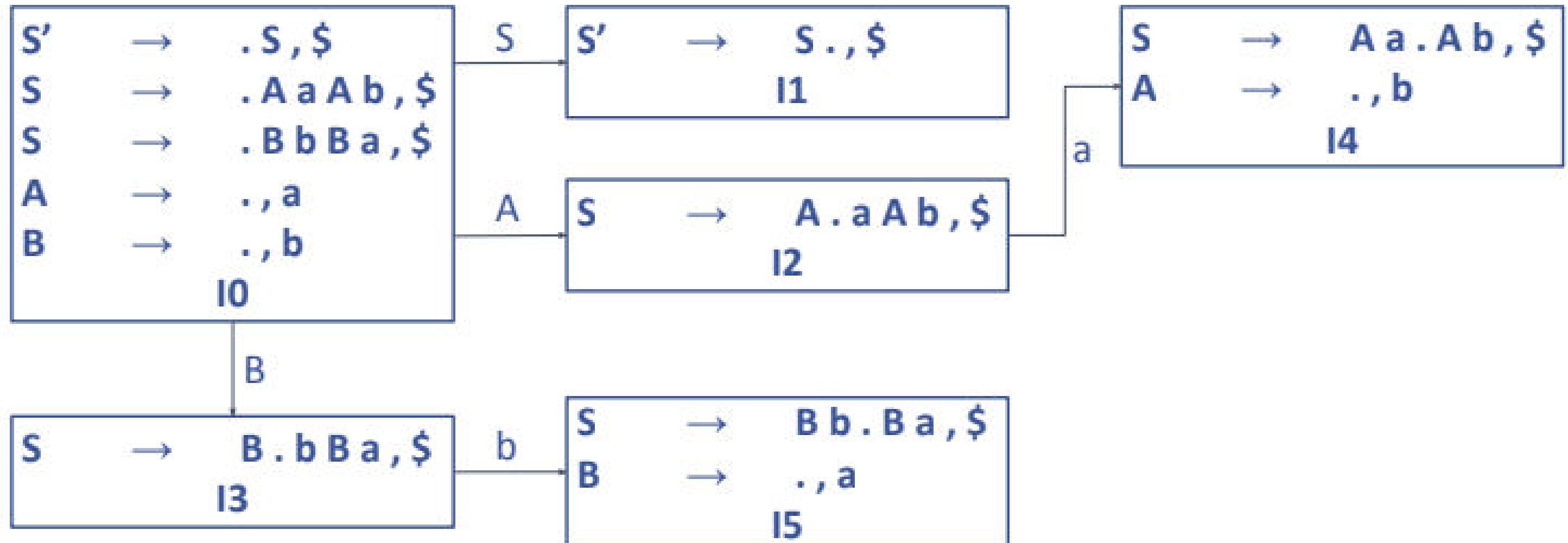
Example 1 solution (continued)

Step 5 continued



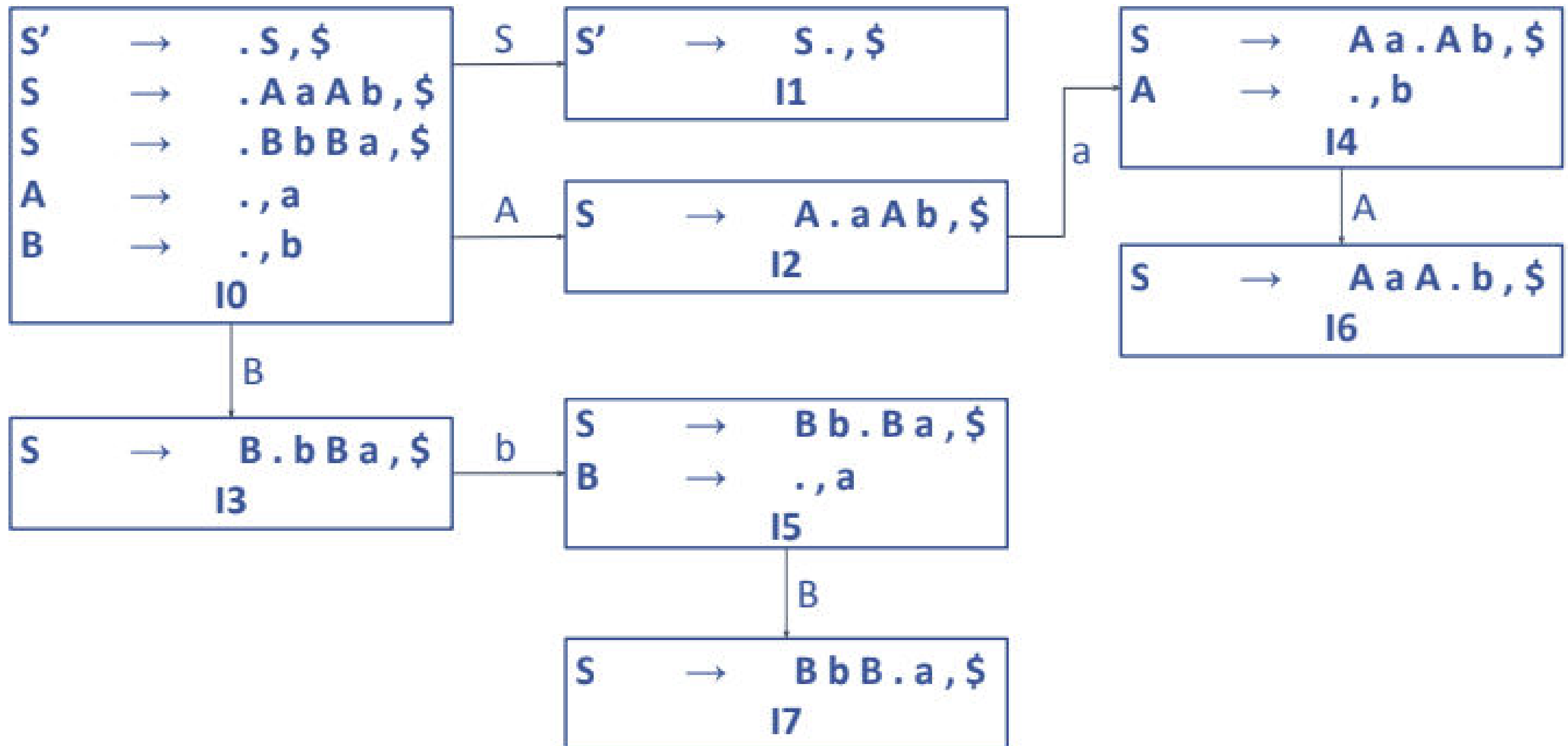
Example 1 solution (continued)

Step 6: Repeat steps 2-5 until no more states can be added



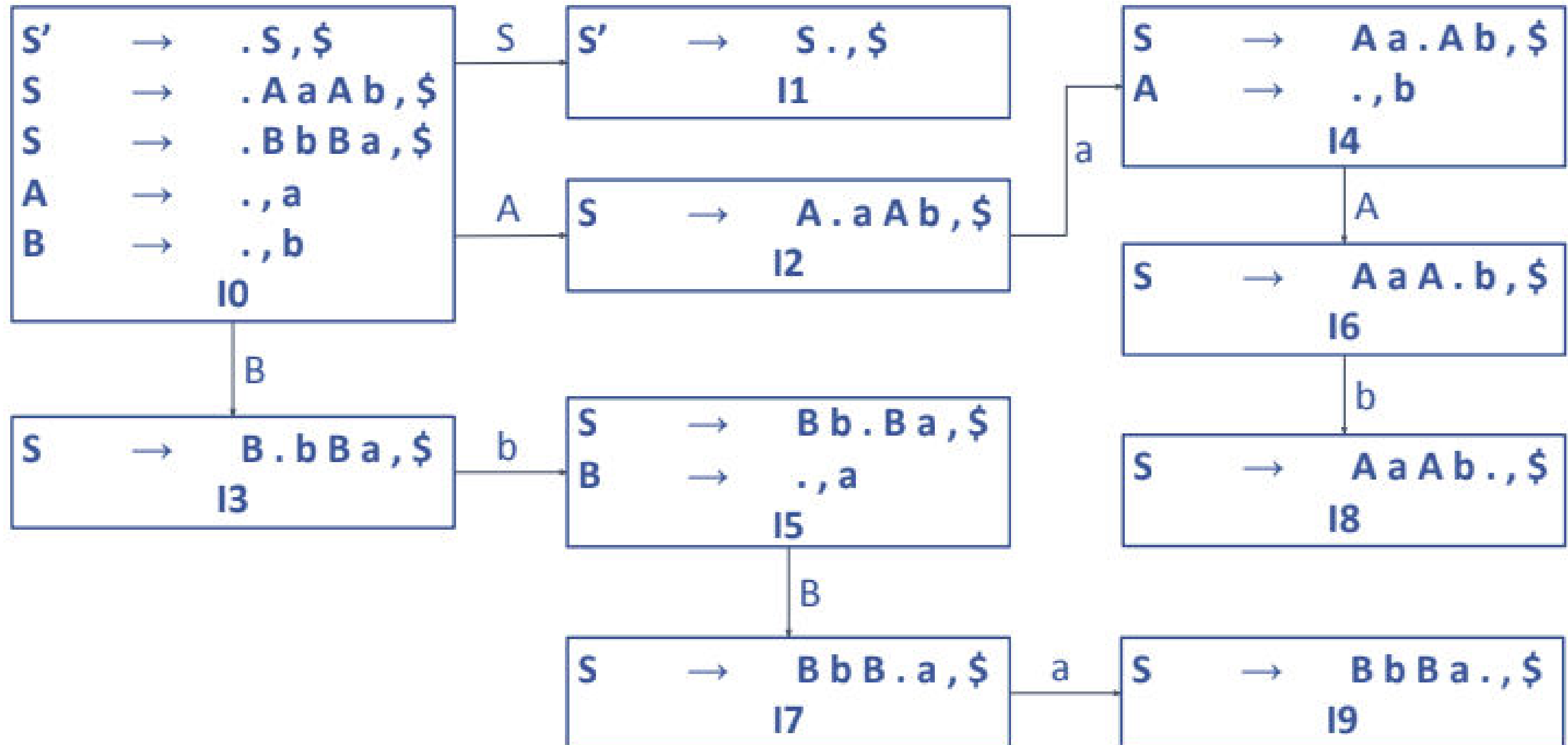
Example 1 solution (continued)

Step 6 continued



Example 1 solution (continued)

Step 6 continued



Example 1 solution (continued)

State	Action			Goto		
	a	b	\$	S	A	B
0						

Let us fill the CLR parsing table now.

State	Action			Goto		
	a	b	\$	S	A	B
0	r_3	r_4		1	2	3

Notice how the reduce move is placed only in the necessary columns. This is the power of LR(1).

Example 1 solution (continued)

Filling the table for the rest of the states.

State	Action			Goto		
	a	b	\$	S	A	B
0	r ₃	r ₄		1	2	3
1			Accept			
2	s ₄					
3		s ₅				

Example 1 solution (continued)

State	Action			Goto		
	a	b	\$	S	A	B
0	r ₃	r ₄		1	2	3
1			Accept			
2	s ₄					
3		s ₅				
4		r ₃			6	
5	r ₄					7

Example 1 solution (continued)

State	Action			Goto		
	a	b	\$	S	A	B
0	r ₃	r ₄		1	2	3
1			Accept			
2	s ₄					
3		s ₅				
4		r ₃			6	
5	r ₄					7
6		s ₈				
7	s ₉					

Example 1 solution (continued)

State	Action			Goto		
	a	b	\$	S	A	B
0	r ₃	r ₄		1	2	3
1			Accept			
2	s ₄					
3		s ₅				
4		r ₃			6	
5	r ₄					7
6		s ₈				
7	s ₉					
8			r ₁			
9			r ₂			

Note that no two states have the same LR(0) items with different lookaheads and hence no states can be merged. Therefore, LALR table is the same as CLR table in

Is the following grammar in CLR and

LALR?

$S \rightarrow A a \mid b A c \mid B c \mid b B a$

$A \rightarrow d$

$B \rightarrow d$

Is the following grammar in CLR and

LALR?

$S \rightarrow A a \mid b A c \mid B c \mid b B a$

$A \rightarrow d$

$B \rightarrow d$

Augmenting and numbering the

grammar: $S' \rightarrow S$

1. $S \rightarrow A a$

2. $S \rightarrow b A c$

3. $L \rightarrow B c$

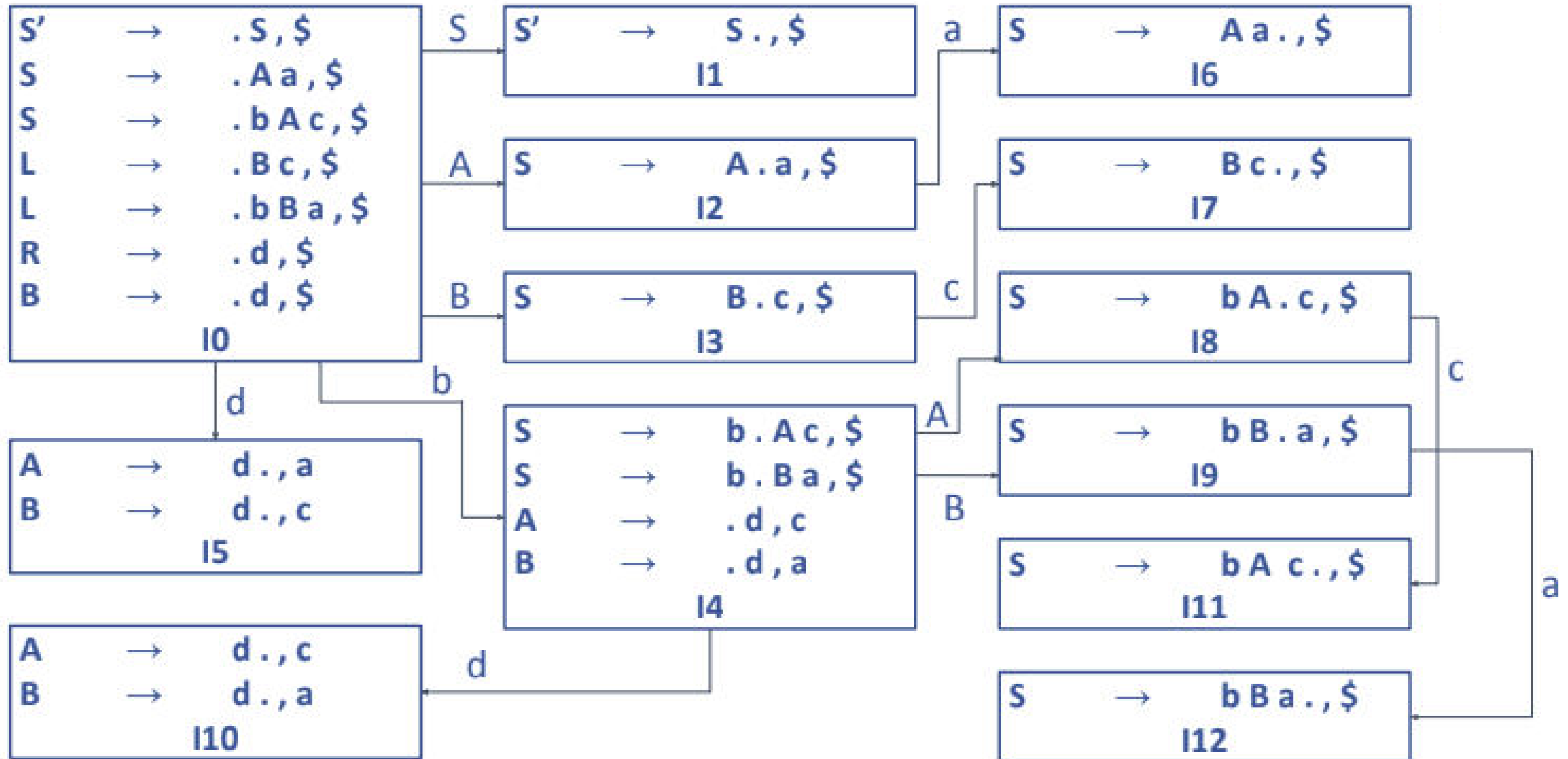
4. $L \rightarrow b B a$

5. $A \rightarrow d$

6. $B \rightarrow d$

Compiler Design

Example 2 solution (continued)



Example 2 solution (continued) - CLR parsing table

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0		s ₄		s ₅		1	2	3
1					Accept			
2	s ₆							
3			s ₇					
4					s ₁₀		8	9
5	r ₅		r ₆					
6					r ₁			

: : : : : : : :

Table continued on next slide.

Example 2 solution (continued) - CLR parsing table

State	Action					Goto		
	a	b	c	d	\$	S	A	B
	:	:	:	:	:	:	:	:
7					r ₃			
8			s ₁₁					
9	s ₁₂							
10	r ₆		r ₅					
11					r ₂			
12					r ₄			

Grammar part of
LR(1)

Example 2 solution (continued) - LALR parsing table

States 5 and 10 have the same LR(0) itemset but different lookaheads. But there is a catch.

Note the cells with text in orange colour.

When these states are merged, there will be a reduce-reduce conflict.

A snapshot of the LALR parsing table is given below:

State	Action				
	a	b	c	d	\$
5 - 10	r ₅ / r ₆		r ₆ / r ₅		

Consider the following

grammar: $\rightarrow A a \mid b A c \mid d c \mid b d$

$A \rightarrow d$

Recall that this grammar is not in LR(0) and
SLR Check if it is in CLR and/or LALR

Consider the following

grammar: $\rightarrow A a \mid b A c \mid d c \mid b d$

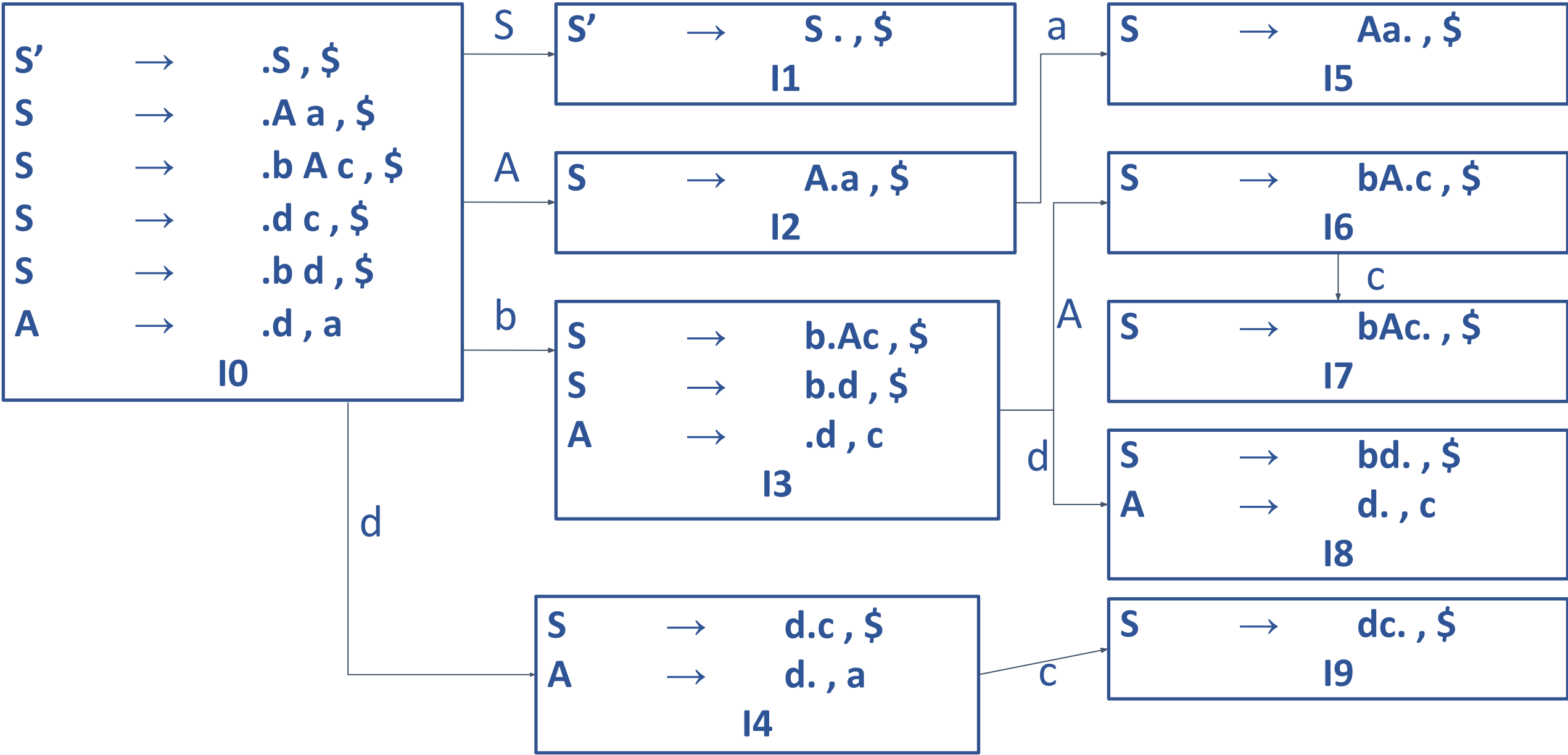
$A \rightarrow d$

Augmenting and numbering the

grammar: $S' \rightarrow S$

1. $S \rightarrow A a$
2. $S \rightarrow b A c$
3. $S \rightarrow d c$
4. $S \rightarrow b d$
5. $A \rightarrow d$

Example 3 solution (continued)



Example 3 solution (continued) - CLR parsing table

State	Action					Goto	
	a	b	c	d	\$	S	A
0		s ₃		s ₄		1	2
1					Accept		
2	s ₅						
3				s ₈			6
4	r ₅		s ₉				
5					r ₁		
6			s ₇				
7					r ₂		
8			r ₅		r ₄		
9					r ₃		

The given grammar is in CLR and LALR

Consider the following

grammar: $\rightarrow A A$

$$A \rightarrow \begin{matrix} a A & | \\ b \end{matrix}$$

Recall that this grammar is in LR(0)

Make its CLR, LALR parsing tables and parse the string "abb"

Consider the following

grammar: $\rightarrow A A$

$$A \rightarrow a A \mid b$$

Augmenting and numbering the

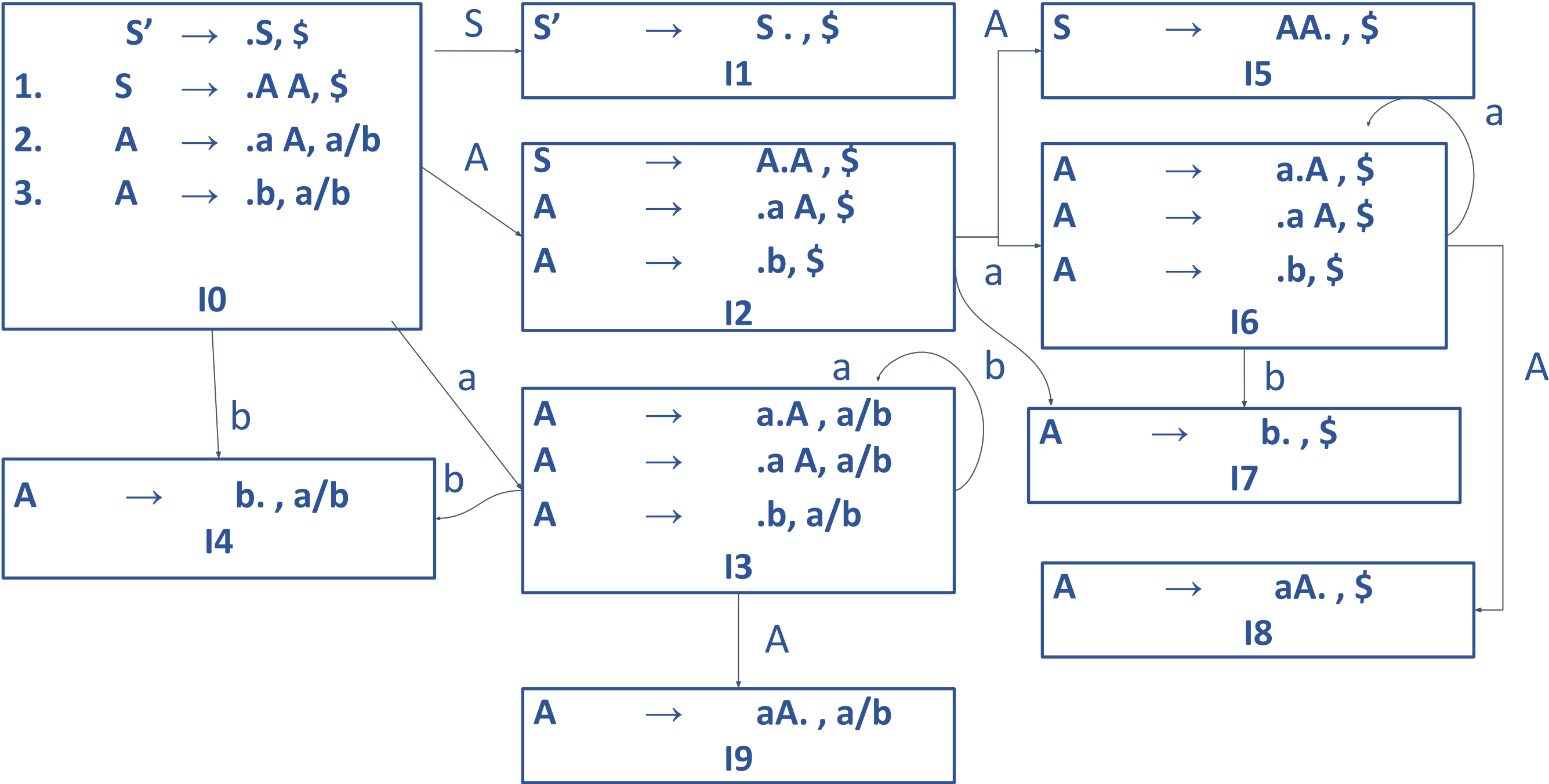
grammar: $S' \rightarrow S$

1. $S \rightarrow A A$

2. $A \rightarrow a A$

3. $A \rightarrow b$

Example 4 solution (continued)



Example 4 solution (continued) - CLR parsing table

State	Action			Goto	
	a	b	\$	S	A
0	s ₃	s ₄		1	2
1			Accept		
2	s ₆	s ₇			5
3	s ₃	s ₄			9
4	r ₃	r ₃			
5			r ₁		
6	s ₆	s ₇			8
7			r ₃		
8			r ₂		
9	r ₂	r ₂			

The given grammar is in CLR

Example 4 solution (continued) - CLR parsing table

State	Action			Goto	
	a	b	\$	S	A
0	s ₃	s ₄		1	2
1			Accept		
2	s ₆	s ₇			5
3,6	s _{3,6}	s _{4,7}			8,9
4,7	r ₃	r ₃	r ₃		
5			r ₁		
8,9	r ₂	r ₂	r ₂		

For LALR
merge:

- 13, 16
- 14, 17
- 18, 19

The given grammar is
in LALR

Consider the following

grammar:

$$\begin{aligned} E &\rightarrow T + E \mid T \\ T &\rightarrow id \end{aligned}$$

Recall that this grammar is in SLR(1)

Make its CLR, LALR parsing tables and parse the string “id + id”

Consider the following

grammar:

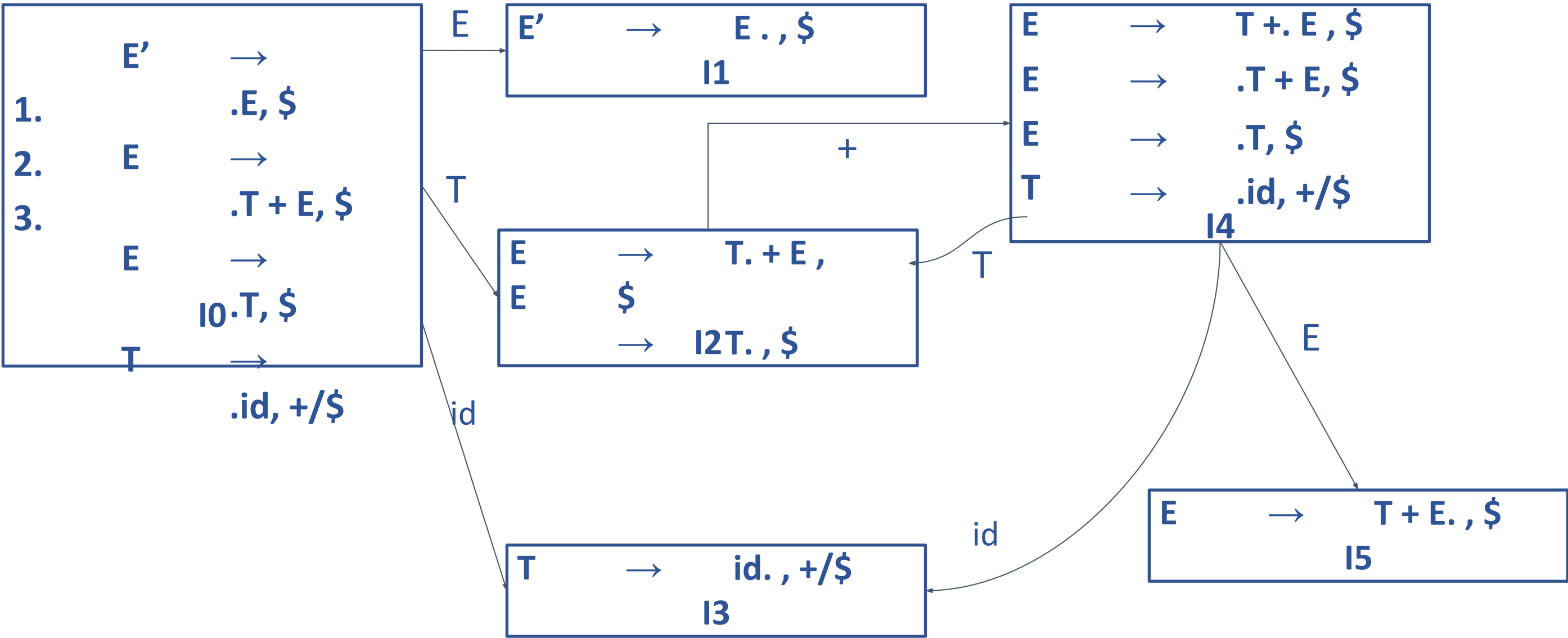
$$E \rightarrow T + E \mid T$$
$$T \rightarrow id$$

Augmenting and numbering the

grammar: $E' \rightarrow E$

1. $E \rightarrow T + E$
2. $E \rightarrow T$
3. $T \rightarrow id$

Example 5 solution (continued)



Example 5 solution (continued) - CLR parsing table

State	Action			Goto	
	id	+	\$	E	T
0	s ₃			1	2
1			Accept		
2		s ₄	r ₂		
3		r ₃	r ₃		
4	s ₃			5	2
5			r ₁		

The given grammar is in CLR and LALR

Consider the following

grammar:

$$\begin{aligned} & \rightarrow d A \mid \\ A & \rightarrow a B \\ B & \rightarrow b A \mid c \end{aligned}$$

Recall that this grammar is in LR(0)

Make its CLR, LALR parsing tables and parse the string "abc"

Consider the following

grammar:

$$S \rightarrow d A \mid$$
$$A \rightarrow a B$$
$$B \rightarrow b A \mid c$$

Augmenting and numbering the

grammar:

$$S' \xrightarrow{c} S$$

1. $S \rightarrow d A$

2. $S \rightarrow a B$

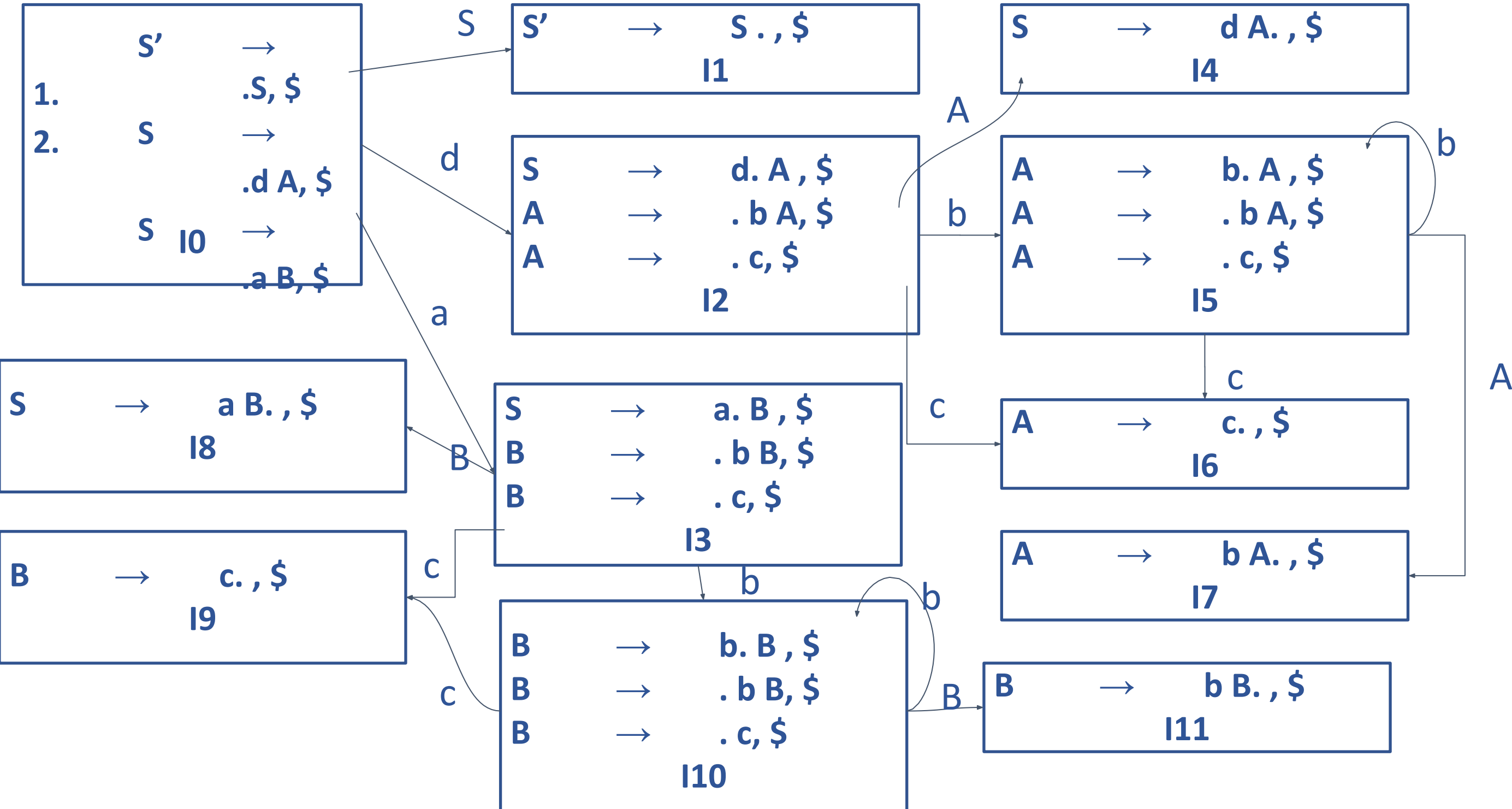
3. $A \rightarrow b A$

4. $A \rightarrow c$

5. $B \rightarrow b B$

6. $B \rightarrow c$

Example 6 solution (continued)



Example 6 solution (continued) - CLR parsing table

State	Action					Goto		
	a	b	c	d	\$	S	A	B
0	s ₃			s ₂		1		
1					Accept			
2		s ₅	s ₆				4	
3		s ₁₀	s ₉					8
4					r ₁			
5		s ₅	s ₆				7	
6					r ₄			
: : : : : : : : :								

The given grammar is in CLR and LALR

Example 6 solution (continued) - CLR parsing table

State	Action					Goto		
	a	b	c	d	\$	S	A	B
:	:	:	:	:	:	:	:	:
7					r ₃			
8					r ₂			
9					r ₆			
10		s ₁₀	s ₉					11
11					r ₅			

Consider the following grammar:

$$S \rightarrow S S + \mid S S * \mid a$$

Recall that this grammar is in LR(0) and parse the string

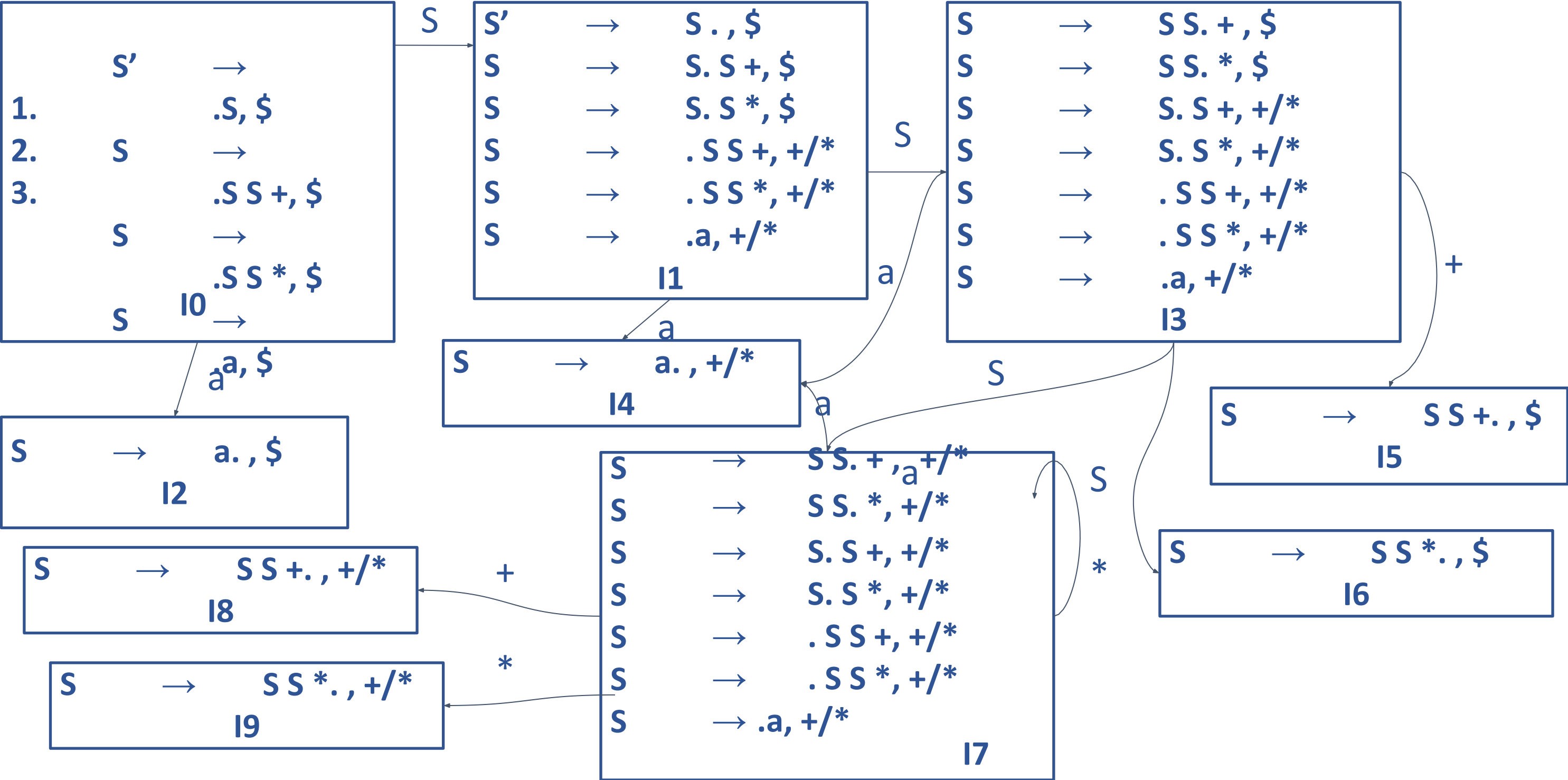
Make its CLR, LALR parsing
tables "aa+aa*"

Consider the following grammar:

$$S \rightarrow SS+ \mid SS* \mid a$$

Augmenting and numbering the grammar: $S' \rightarrow S$

1. $S \rightarrow SS+$
2. $S \rightarrow SS*$
3. $S \rightarrow a$



Example 7 solution (continued) - CLR parsing table

State	Action				Goto
	a	+	*	\$	
0	s ₂				1
1	s ₄			Accept	3
2				r ₃	
3	s ₂	s ₅	s ₆		7
4		r ₃	r ₃		
5				r ₁	
6				r ₂	

: : : : :

Example 7 solution (continued) - CLR parsing table

State	Action				Goto
	a	+	*	\$	S
	:	:	:	:	:
7	s ₄	s ₈	s ₉		7
8		r ₁	r ₁		
9		r ₂	r ₂		

Grammar part of
CLR

Example 7 solution (continued) - CLR parsing table

State	Action				Goto
	a	+	*	\$	
0	s ₂				1
1	s ₄			Accept	3
2,4		r ₃	r ₃	r ₃	
3,7	s _{2,4}	s _{5,8}	s _{6,9}		7
5,8		r ₁	r ₁	r ₁	
6,9		r ₂	r ₂	r ₂	

For LALR
merge:

- 12, 14
- 13, 17
- 15, 18
- 16, 19

Grammar part of
LALR

Find out if the given grammar is in LR(0), SLR, LALR and CLR.

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{id}$$

Is the given grammar in LL(1) and LALR(1)?

S	→	(X		E sq)		F
				(		
X	→	E)		F sq)		
E	→	A				
F	→	A				
A	→	λ				

Is the language finite?

Is the given grammar in LL, LR(0), SLR, LALR and LR?

$S \rightarrow (X \mid E] \mid F)$

$X \rightarrow E) \mid F]$

$E \rightarrow A$

$F \rightarrow A$

$A \rightarrow \lambda$

Provide a production with the shortest RHS that introduces s/r conflict in SLR parser for the given grammar:

$S \rightarrow A b$

$A \rightarrow c b A d$

$A \rightarrow c A d$

$A \rightarrow \lambda$

Is the given grammar in LL, LR(0), SLR, LALR and LR?

$S \rightarrow A b b x \quad | \quad B b b y$

$A \rightarrow x$

$B \rightarrow x$

Specify the value of i and j such that the grammar in LL(i) and LR(j).

Why is LR(1) so powerful?

Intuitively, for two reasons:

1) Lookahead makes handle-finding easier.

- The LR(0) automaton says whether there could be a handle later on based on no right context.
- The LR(1) automaton can predict whether it needs to reduce based on more information.

2) More states encode more information.

- LR(1) lookaheads are very good because there's a greater number of states to be in.
- Goal: Incorporate lookahead without increasing the number of states.

The limits of LR Parsers in general

LR parsers use shift and reduce actions to reduce the input to the start symbol.

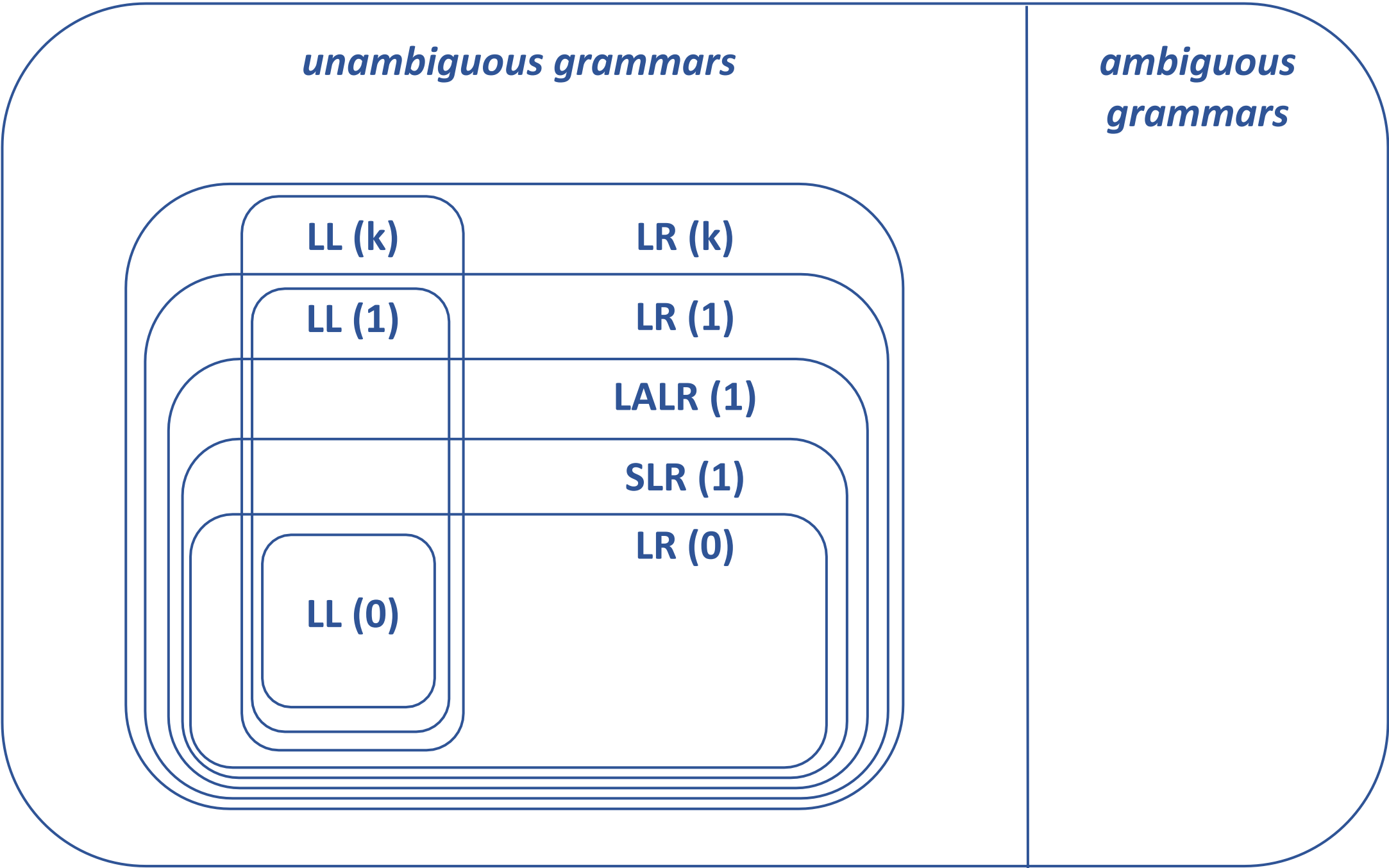
- LR parsers cannot deterministically handle shift/reduce or reduce/reduce conflicts.
- However, they can nondeterministically handle these conflicts by guessing which option to choose.

- What should the parser do when it encounters an error?
- Could just say “syntax error,” but we'd like more detailed messages.
- How do we resume parsing after an error? [error-recovery strategies discussed in previous lectures]

CLR will not make any reduction if there is an error in the string. LALR and SLR may make a few reductions before declaring error.

Summary of all the table-driven bottom-up parsers

Parser	LR(0)	SLR(1) or SLR (Simple LR)	LALR(1) or LALR (LookAhead LR)	CLR(1) or CLR or LR(1) or LR (Canonical LR)
DFA	LR(0) automata LR(0) item		LR(1) automata LR(1) item = LR(0) item + lookahead	
Reduce Move Placement in Action part of the Parsing table	Place the reduce move in the entire row for the state that contains a final item	Place the reduce move only in the Follow(LHS).	Place the reduce move in the lookahead	Place the reduce move in the lookahead
Power	Least Powerful	More powerful than LR(0)	More powerful than SLR	Most Powerful
Number of States	n : Number of States in a parser $n(\text{LR}(0)) = n(\text{SLR}) = n(\text{LALR}) \leq n(\text{CLR})$			



Note that this diagram refers to **grammars**, not languages, e.g. there may be an equivalent LR(1) grammar that accepts the same language as another non-LR(1) grammar. No ambiguous grammar is LL(1) or LR(1), so we must either rewrite the grammar to remove the ambiguity or resolve conflicts in the parser table or implementation.

The hierarchy of LR variants is clear: every LR(0) grammar is SLR(1) and every SLR(1) is LALR(1) which in turn is LR(1). But there are grammars that don't meet the requirements for the weaker forms that can be parsed by the more powerful variations.

Error repair:

- Both LL(1) and LALR(1) parsers possess the valid prefix property. What is on the stack will always be a valid prefix of a sentential form. Errors in both types of parsers can be detected at the earliest possible point without pushing the next input symbol onto the stack.
- LL(1) parser stack contain symbols that are predicted but not yet matched. This information can be valuable in determining proper repairs.
- LALR(1) parse stacks contain information about what has already been seen, but do not have the same information about the right context that is expected.
- This means deciding possible continuations is somewhat easier in an LL(1) parser.

Efficiency:

- Both require a stack of some sort to manage the input.
That stack can grow to a maximum depth of n , where n is the number of symbols in the input.
- If you are using the runtime stack (i.e. function calls) rather than pushing and popping on a data stack, you will probably pay some significant overhead for that convenience (i.e. a recursive descent parser takes that hit).
- If both parsers are using the same sort of stack, LL(1) and LALR(1) each examine every non-terminal and terminal when building the parse tree, and so parsing speeds tend to be comparable between the two.



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Kavya P

K

Compiler Design

Unit 2: Syntax Directed Definitions

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, you will learn about -

- Semantic Analysis
- What is Syntax Directed Translation?
- Semantic Rules
- Syntax-directed Definitions
- Types of Attributes
- S-Attributed SDD and its evaluation

Semantic Analysis

- Semantic Analysis computes additional information related to the meaning of the program once the syntactic structure is known.
- In typed languages as C, semantic analysis involves adding information to the symbol table and performing type checking.
- The information to be computed is beyond the capabilities of standard parsing techniques, therefore it is not regarded as syntax.
- For Semantic Analysis, we need both a Representation Formalism and an Implementation Mechanism.
- Syntax Directed Translations is an illustration of Representation Formalism.

What is a Syntax Directed Translation?

- The Principle of **Syntax Directed Translation** states that the meaning of an input sentence is related to its syntactic structure, i.e., to its Parse-Tree.
- Translations for programming language constructs guided by context-free grammars.
 - We associate **attributes** to the grammar symbols representing the language constructs.
 - Values for attributes are computed by **Semantic Rules** associated with grammar productions.

There are two notations for attaching semantic rules:

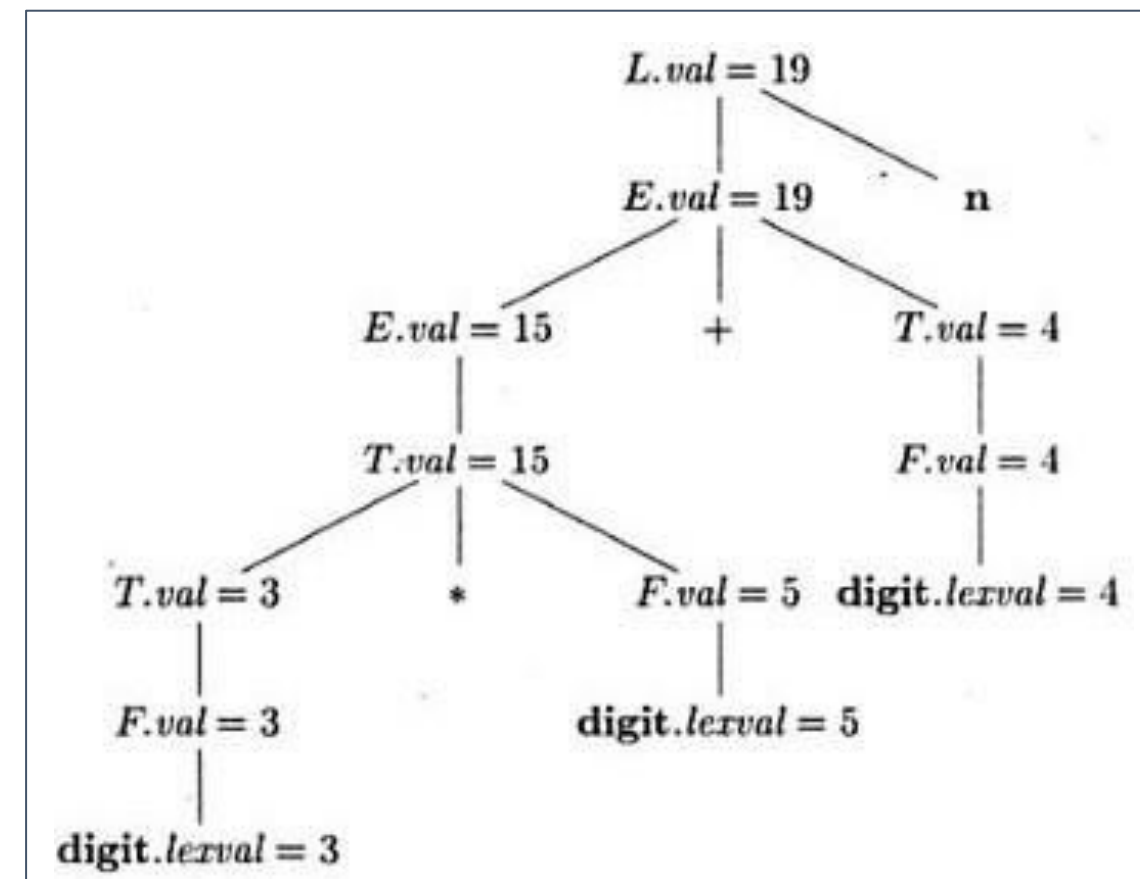
1. **Syntax Directed Definitions** - High-level specification hiding many implementation details.
2. **Translation Schemes** - More implementation oriented, indicates the order in which semantic rules are to be evaluated.

Evaluation of Semantic Rules may -

- Generate Code
- Insert information into the Symbol Table
- Perform Semantic Check
- Issue error messages
- etc.

Syntax Directed Definitions (SDD)

- **Syntax Directed Definitions (SDD)** are a generalization of context-free grammars in which -
 - Grammar symbols have an associated set of **attributes**.
 - Productions are associated with **Semantic Rules** for computing the values of attributes.
- This formalism generates **Annotated Parse Trees**, where each node of the tree is a record with a field for each attribute.
 - Example - **F.val** indicates the attribute **val** for grammar symbol **F**



Syntax Directed Definitions (SDD)

Formally, SDDs are defined as follows -

Each production, $A \rightarrow \alpha$, is associated with a set of semantic rules-

$$b := f(c_1, c_2, \dots, c_k)$$

where f is a function,

b is either

1. A synthesized attribute of A , and $c_1, c_2, \dots, c_k$ are attributes of the grammar symbols of the production, or
2. An inherited attribute of a grammar symbol in α , and $c_1, c_2, \dots, c_k$ are attributes of grammar symbols in α or attributes of A .

Example - SDD to implement a Simple Desk Calculator

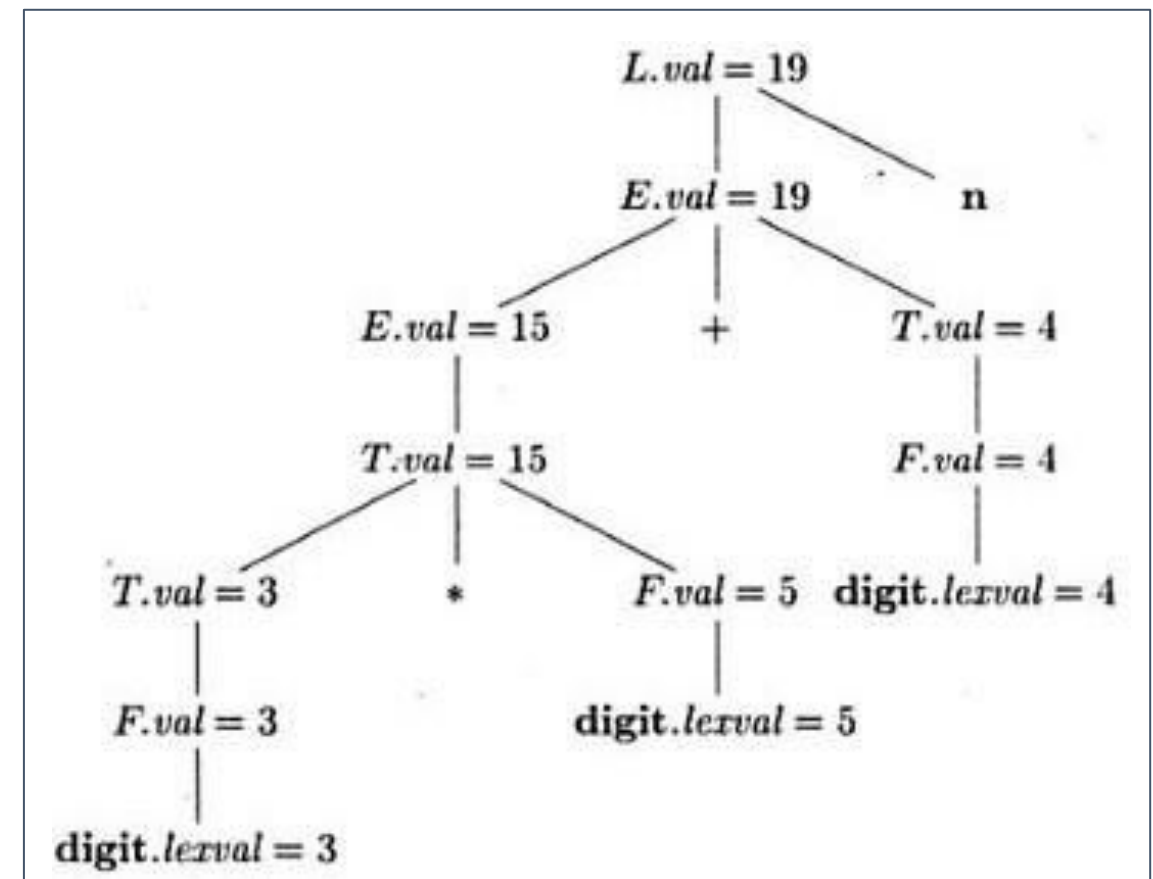
Production	Semantic Rule
$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val \}$
$E \rightarrow T$	$\{ E.val = T.val \}$
$T \rightarrow T_1 * F$	$\{ T.val = T_1.val * F.val \}$
$T \rightarrow F$	$\{ T.val = F.val \}$
$F \rightarrow num$	$\{ F.val = num.lexval \}$
$F \rightarrow id$	$\{ F.val = id.lexval \}$

num.lexval gets its value from the lexical analyser

Types of Attributes

- There are two kinds of attributes -
 - **Synthesized Attributes** - computed from the values of the attributes of the children nodes.
 - **Inherited Attributes** - computed from the values of the attributes of both the siblings and the parent nodes.

- The Syntax Directed Definition associates to each non terminal a synthesised attribute called **val**.
- Terminal symbols are assumed to have synthesised attributes supplied by the lexical analyser.
 - For example, in the given parse tree **digit.lexval** gets its value from the lexer.



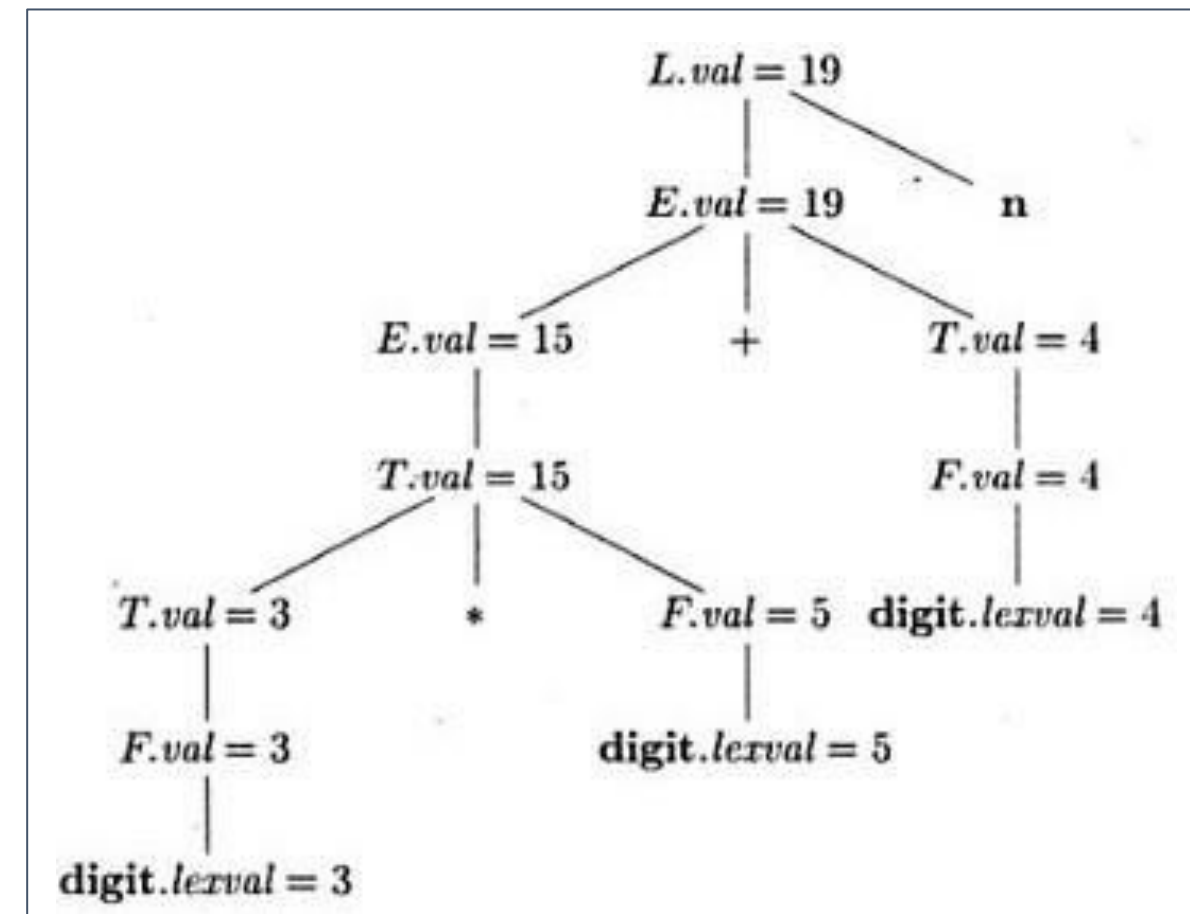
Synthesised
Attributes

- Procedure calls define values of Dummy synthesized attributes of the non-terminal on the left-hand side of the production.
 - For example, **print** in the given table -

Production	Semantic Rule
$L \rightarrow En$	$print(E.val)$
$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val \}$
$E \rightarrow T$	$\{ E.val = T.val \}$
$T \rightarrow T_1 * F$	$\{ T.val = T_1.val * F.val \}$
$T \rightarrow F$	$\{ T.val = F.val \}$
$F \rightarrow (E)$	$\{ F.val = E.val \}$
$F \rightarrow digit$	$\{ F.val = digit.lexval \}$
$F \rightarrow id$	$\{ F.val = id.lexval \}$

S-Attributed Definitions

- An SDD with only synthesized attributes is called an **S-Attributed Definition**.
- Evaluation order - Semantic rules in a S-Attributed Definition can be evaluated by a bottom-up or Post-Order traversal of the parse-tree.
- The above arithmetic grammar is an example of an S-Attributed Definition.
- The figure represents the annotated parse-tree for the input **3*5+4n**.



Inherited

Attributes

- Inherited Attributes are useful for expressing the dependence of a construct on the context in which it appears.
- Unlike synthesized attributes, the order in which the inherited attributes of the children are computed is important.
- Inherited attributes of the children can depend from both left and right siblings.
- Evaluation Order - Inherited attributes cannot be evaluated by a simple PreOrder traversal of the parse-tree.
 - Exception - Inherited attributes that **do not** depend on right children can be evaluated by a classical PreOrder traversal.

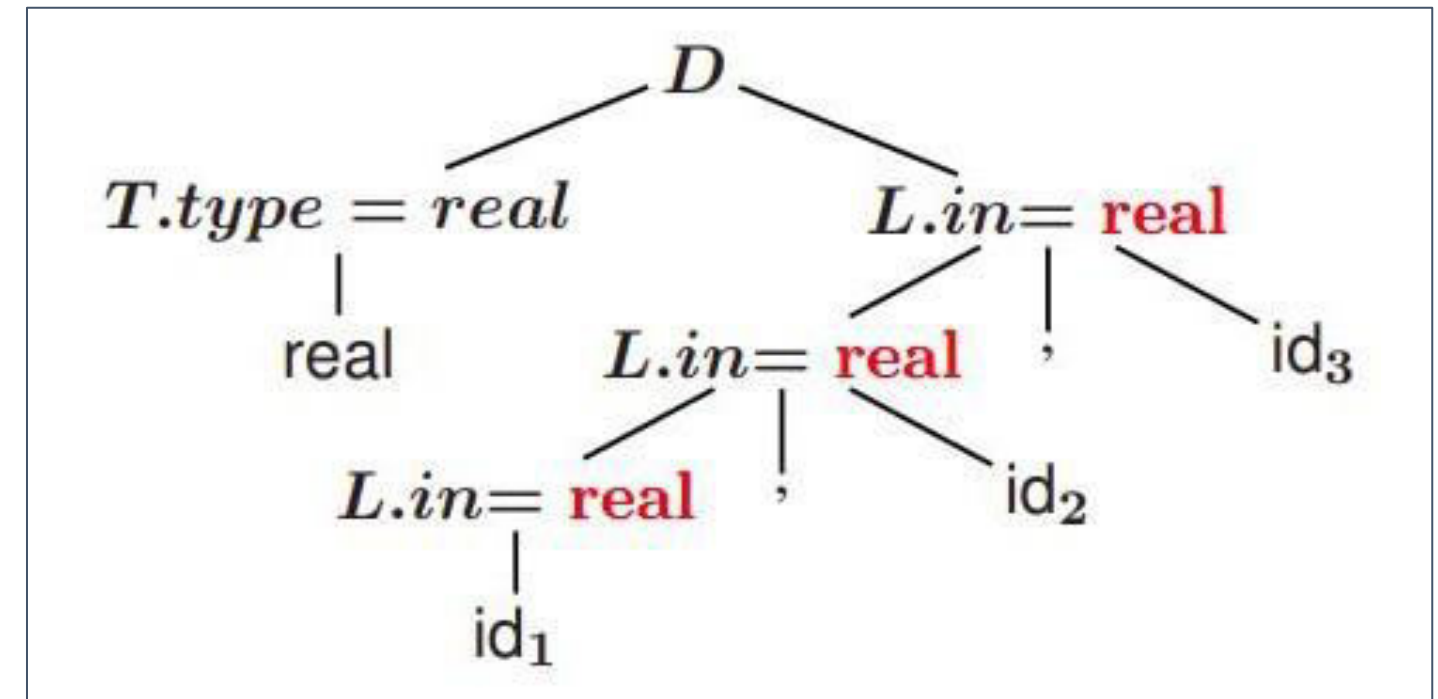
Inherited Attributes - Example

- Consider the following syntax directed definition for Type Declarations.
- The non terminal **T** has a synthesized attribute **type** determined by the keyword in the declaration.
- The production $D \rightarrow T L$ is associated with the semantic rule $L.in = T.type$; which set the inherited attribute **L.in**

Production	Semantic Rule
$D \rightarrow T L$	$\{ L.in = T.type; \}$
$T \rightarrow \text{int}$	$\{ T.type = \text{integer}; \}$
$T \rightarrow \text{real}$	$\{ T.type = \text{float}; \}$
$L \rightarrow L_1, \text{id}$	$\{ L_1.in = L.in;$ $\text{addType}(\text{id.entry}, L.in);$ $\}$
$L \rightarrow \text{id}$	$\{ \text{addType}(\text{id.entry}, L.in); \}$

Inherited Attributes - Example

- The given diagram illustrates the annotated parse-tree for the input **real id1, id2, id3**
- **L.in** is then inherited top-down the tree by the other L-nodes.
- At each **L-node** the procedure **addtype** inserts the type of the identifier into the symbol table.



Evaluating SDDs - Parse Tree method

Evaluating an SDD over a given input consists of the following steps

-

1. Construct Parse Tree for given input.
2. Construct Dependency graph.
3. Topologically sort the nodes of Dependency graph.
4. Produce as output Annotated Parse Tree.



**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu



Compiler Design

Preet Kanwal

Department of Computer Science & Engineering

Teaching Assistant : Kavya P

K

Compiler Design

Unit 2: Syntax Directed Definitions

Preet Kanwal

Department of Computer Science &
Engineering

In this lecture, you will learn about -

- Dependency graphs
- Topological Sort
- Evaluating SDDs - Example - Simple Desk Calculator
- S-Attributed SDD Examples :
 - Counting Binary bits
 - Binary to Decimal conversion
 - Counting parentheses
 - Identifying the type of an expression - int or float
 - Identifying the sign of the evaluated expression - positive or negative

Recap - Evaluating SDDs - Parse tree method

Evaluating an SDD over a given input consists of the following steps

-

1. Construct Parse Tree for given input.
2. Construct Dependency graph.
3. Topologically sort the nodes of Dependency graph.
4. Produce as output Annotated Parse Tree.

- Dependency Graphs are the most general technique used to evaluate syntax directed definitions with both synthesized and inherited attributes.
- A Dependency Graph shows the interdependencies among the attributes of the various nodes of a parse tree.
 - There is a node for each attribute.
 - If attribute **b** depends on an attribute **c** there is a link from the node for **c** to the node for **b** ($b \leftarrow c$).
- **Dependency Rule** - If an attribute **b** depends from an attribute **c**, then we need to fire the semantic rule for **c** first and then the semantic rule for **b**.

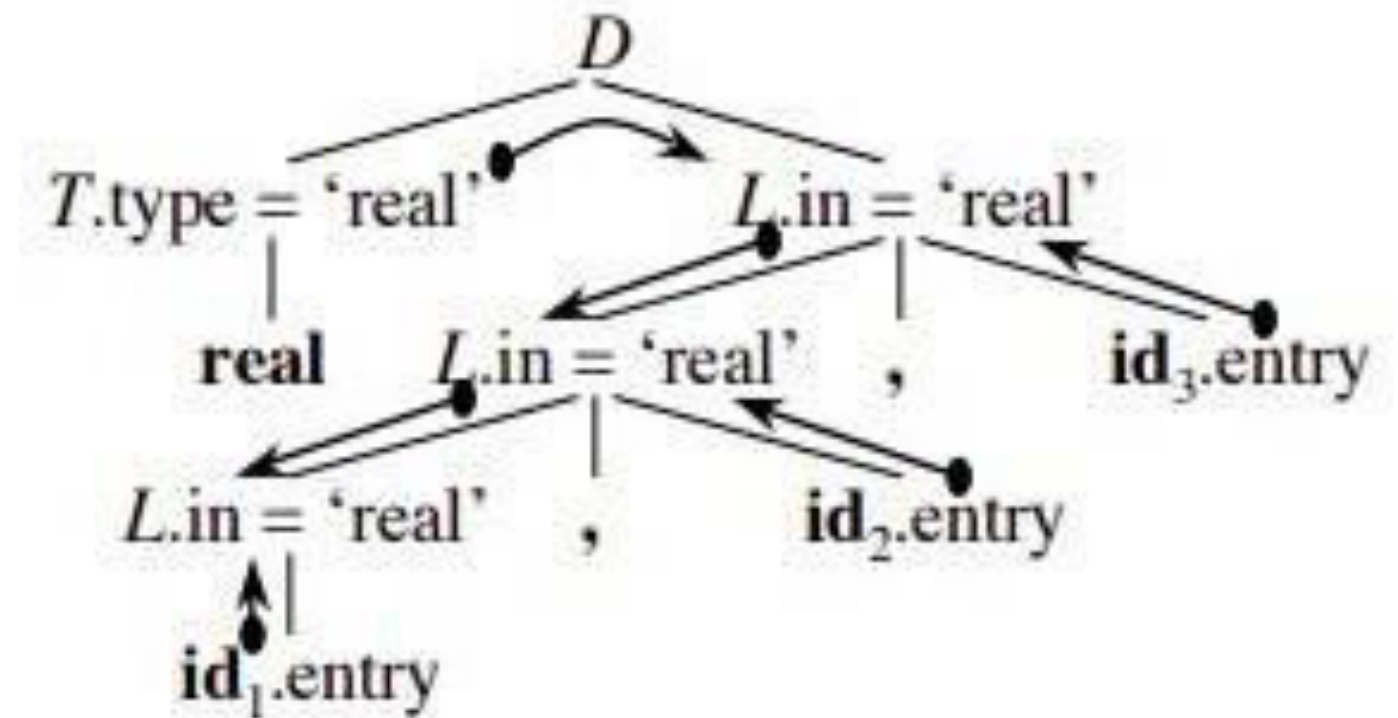
Dependency Graphs - Example

- Consider the earlier example of Syntax Directed Definition for Type Declarations.
- This SDD has both inherited and synthesised attributes.
- Construct the Annotated parse tree with dependency graph for the input **real id1, id2, id3**

Production	Semantic Rule
$D \rightarrow T L$	$\{ L.in = T.type; \}$
$T \rightarrow int$	$\{ T.type = integer; \}$
$T \rightarrow real$	$\{ T.type = float; \}$
$L \rightarrow L_1 , id$	$\{ L_1.in = L.in; \}$ $addType(id.entry, L.in);$ $\}$
$L \rightarrow id$	$\{ addType(id.entry, L.in); \}$

Dependency Graphs - Example

- The figure illustrates the annotated parse tree with the dependencies between attributes.



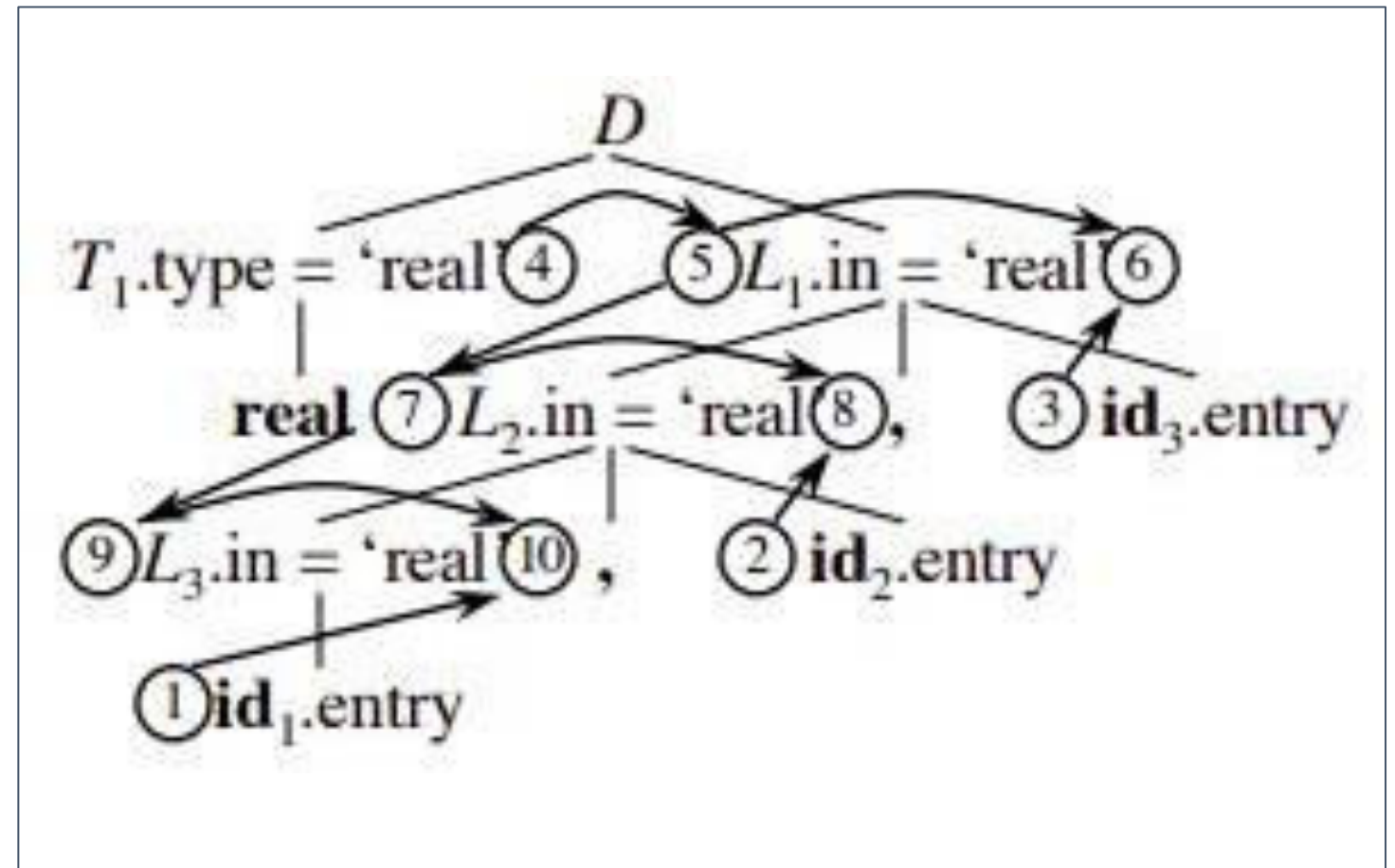
- The evaluation order of semantic rules depends from a Topological Sort derived from the dependency graph.
- Topological Sort - Any ordering $m_1, m_2, \dots, m_k$ such that if $m_i \rightarrow m_j$ is a link in the dependency graph then $m_i < m_j$.
- Any topological sort of a dependency graph gives a valid order to evaluate the semantic rules.

Topological Sort -

Example

- Consider the previous example of SDD for Type declarations
- The topological sort of the parse tree is as follows -

1. Get id1.entry
2. Get id2.entry
3. Get id3.entry
4. $T_1.type = 'real'$
5. $L_1.in = T_1.type$
6. $addtype(id3.entry, L_1.in)$
7. $L_2.in = L_1.in$
8. $addtype(id2.entry, L_2.in)$
9. $L_3.in = L_2.in$
10. $addtype(id1.entry, L_3.in)$



Evaluating SDDs - General Remarks

- Attributes can be evaluated by building a dependency graph at compile-time and then finding a topological sort.
- Disadvantages
 1. This method fails if the dependency graph has a cycle - We need a test for non-circularity.
 2. This method is time consuming due to the construction of the dependency graph.
- Alternative Approach
 - Design the syntax directed definition in such a way that attributes can be evaluated with a **fixed order** avoiding the need to build the dependency graph - for example, **S attributed definitions**.
 - This method is followed by many compilers.

Evaluating S-Attributed Definitions

- Synthesized Attributes can be evaluated by a bottom-up parser as the input is being analyzed avoiding the construction of a dependency graph.
- The parser keeps the values of the synthesized attributes in its stack.
- Whenever a reduction $A \rightarrow \alpha$ is made, the attribute for A is computed from the attributes of α which appear on the stack.
- Thus, a translator for an S-Attributed Definition can be simply implemented by extending the stack of an LR-Parser.

Example - Evaluating SDD to implement a Simple Desk Calculator

- Evaluate the following SDD for the input 3 + 4 * 5

Production	Semantic Rule
$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val \}$
$E \rightarrow T$	$\{ E.val = T.val \}$
$T \rightarrow T_1 * F$	$\{ T.val = T_1.val * F.val \}$
$T \rightarrow F$	$\{ T.val = F.val \}$
$F \rightarrow num$	$\{ F.val = num.lexval \}$

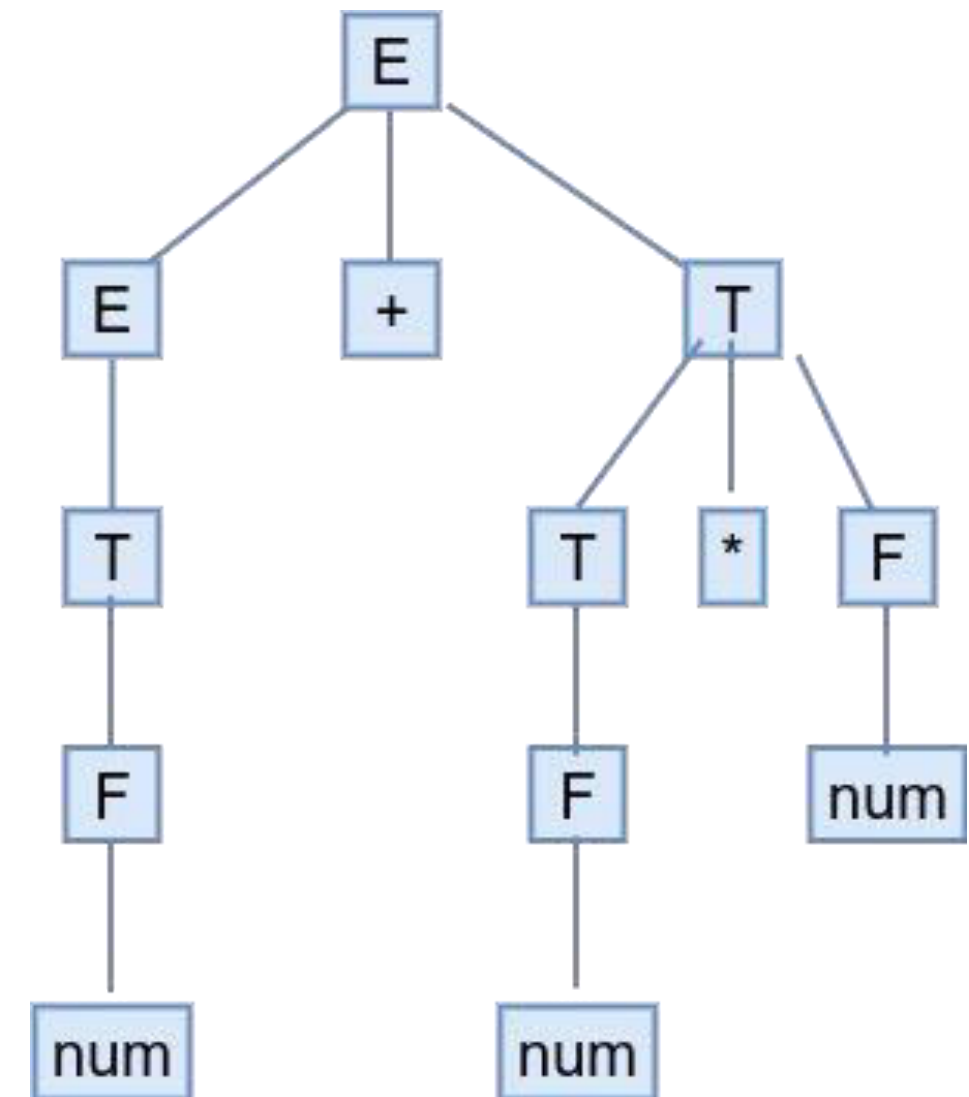
Example - Evaluating SDD to implement a Simple Desk Calculator

Evaluate the following SDD for the input $3 + 4 *$

5 Solution -

1. Construct Parse tree

- Nodes are Terminals or Non-Terminals



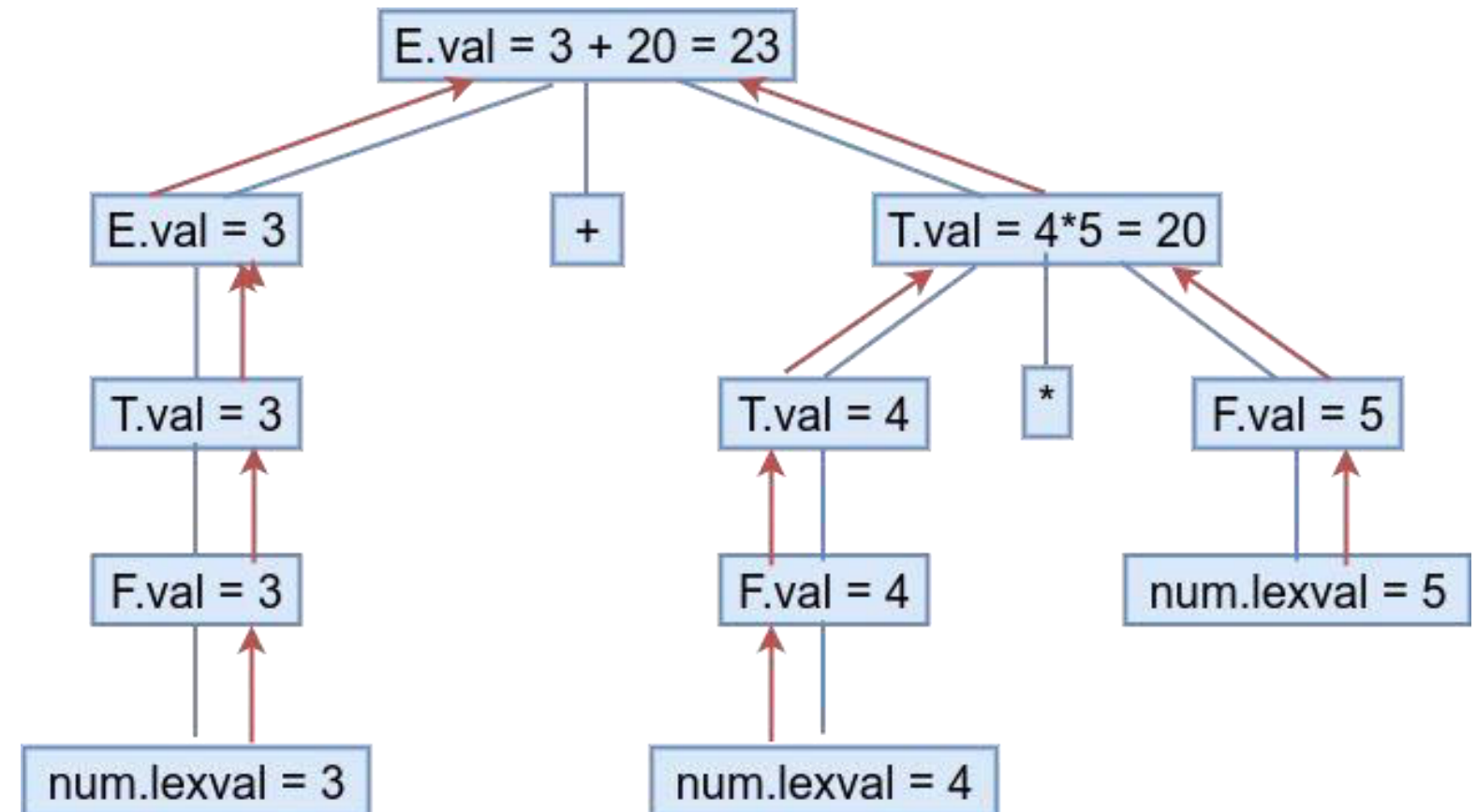
Example - Evaluating SDD to implement a Simple Desk Calculator

Evaluate the following SDD for the input $3 + 4 *$

5 Solution -

2. Construct dependency graph

- Nodes are attributes



Example - Evaluating SDD to implement a Simple Desk Calculator

Evaluate the following SDD for the input 3

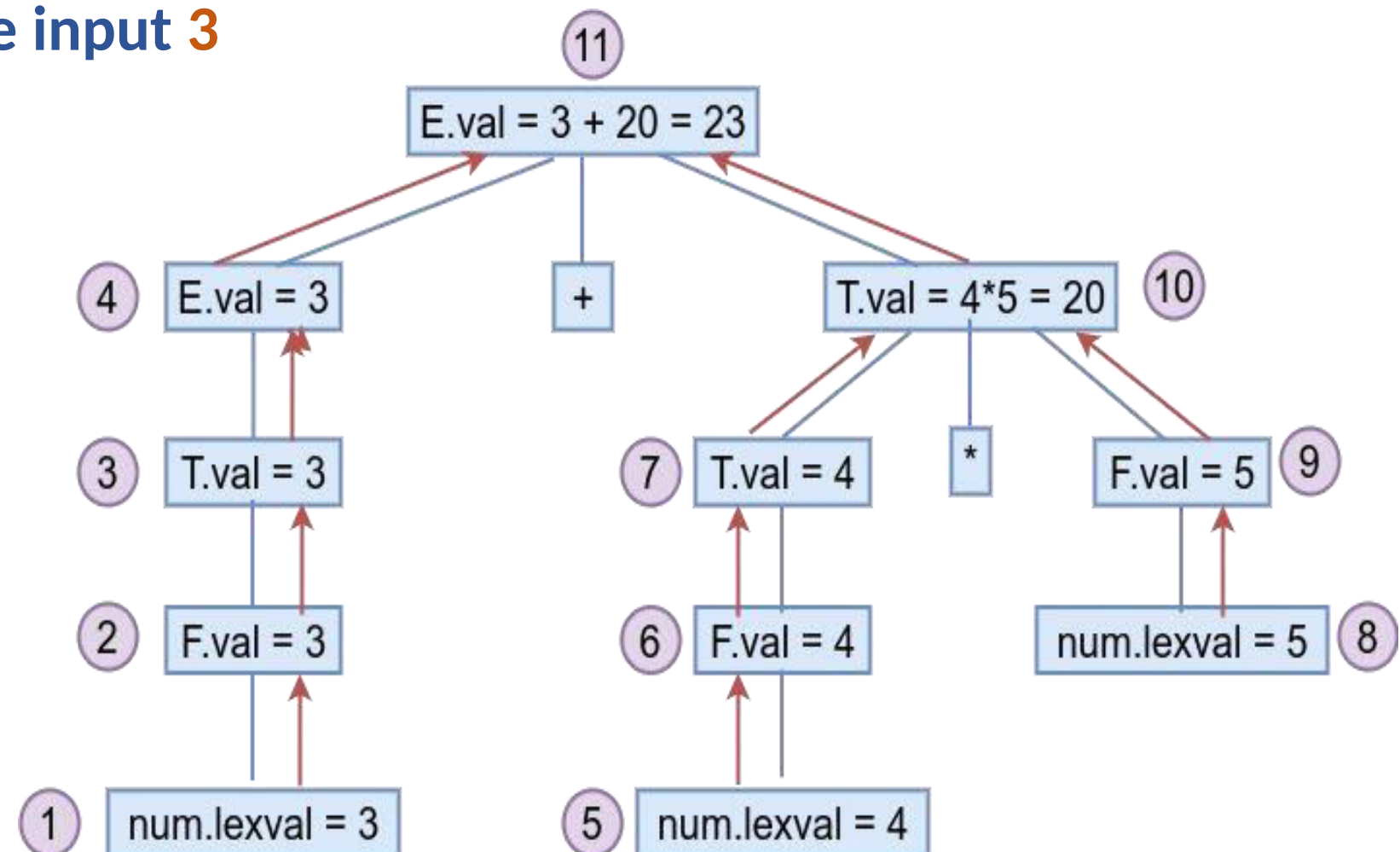
$+ 4 * 5$

Solution -

3. Decide Evaluation order of the nodes in Dependency graph

1,5,8,2,3,6,7,8,9,4,10,11

Output -Annotated Parse tree carrying out the input.



S-attributed SDDs - Examples



Example 1 - SDD to count no. of 1s in a Binary Number

Production	Semantic Rule
$L \rightarrow L_1 B$	$\{ L.count = L_1.count + B.count; \}$
$L \rightarrow B$	$\{ L.count = B.count; \}$
$B \rightarrow 0$	$\{ B.count = 0; \}$
$B \rightarrow 1$	$\{ B.count = 1; \}$

Example 2 - SDD to count no. of 0s in a Binary Number

Production	Semantic Rule
$L \rightarrow L_1 B$	$\{ L.count = L_1.count + B.count; \}$
$L \rightarrow B$	$\{ L.count = B.count; \}$
$B \rightarrow 0$	$\{ B.count = 1; \}$
$B \rightarrow 1$	$\{ B.count = 0; \}$

Example 3 - SDD to count no. of bits in a Binary Number

Production	Semantic Rule
$L \rightarrow L_1 B$	$\{ L.count = L_1.count + B.count; \}$
$L \rightarrow B$	$\{ L.count = B.count; \}$
$B \rightarrow 0$	$\{ B.count = 1; \}$
$B \rightarrow 1$	$\{ B.count = 1; \}$

Example 4 - SDD to convert Binary to Decimal

Production	Semantic Rule
$L \rightarrow L_1 B$	$\{ L.val = 2 * L_1.val + B.val; \}$
$L \rightarrow B$	$\{ L.val = B.val; \}$
$B \rightarrow 0$	$\{ B.val = 0; \}$
$B \rightarrow 1$	$\{ B.val = 1; \}$

Example 5 - SDD to convert Binary Fraction to Decimal

Production	Semantic Rule
$S \rightarrow L_1 .L_2$	$\{ S.val = L_1.val + L_2.val / 2^{L_2.Count} ; \}$
$L \rightarrow L_1 B$	$\{ L.val = 2 * L_1.val + B.val;$ $L.count = L_1.count +$ $B.count; \}$
$L \rightarrow B$	$\{ L.val = B.val;$ $L.count = B.count;$ $\}$
$B \rightarrow 0$	$\{ B.val = 0;$ $B.count = 1;$ $\}$
$B \rightarrow 1$	$\{ B.val = 1;$ $B.count =$

Example 6 - SDD to count no. of balanced Parentheses

Production	Semantic Rule
$S \rightarrow (S_1)$	$\{ S.count = S_1.count + 1; \}$
$S \rightarrow a$	$\{ S.count = 0; \}$

Example 7- SDD to convert infix to postfix

Production	Semantic Rule
$E \rightarrow E_1 +$	$\{ \text{printf}("+"); \}$
$E \rightarrow T$	
$T \rightarrow T_1 *$	$\{ \text{printf}("+"); \}$
$T \rightarrow F$	
$F \rightarrow \text{num}$	$\{ \text{printf}("%d", \text{num.lexval}); \}$

Example 8 - SDD to determine type of each Term and Expression

The given grammar is for expressions involving operator **+** and integer or floating-point operands.
Floating-point numbers are distinguished by having a decimal point.

$E \rightarrow E + T \mid T$

$T \rightarrow \text{num} . \text{num} \mid \text{num}$

Write an SDD to determine the type of each term **T** and expression.

Example 8 - SDD to determine type of each Term and Expression

Production	Semantic Rule
$E \rightarrow E_1 + T$	<pre>{ if(E₁.type == float T.type == float) { E.type = float; } else { E.type = integer; } }</pre>
$E \rightarrow T$	<pre>{ E.type = T.type }</pre>
$T \rightarrow \text{num} . \text{num}$	<pre>{ T.type = float ; }</pre>
$T \rightarrow \text{num}$	<pre>{ T.type = integer ; }</pre>

Example 9 - SDD to Identify the sign of the evaluated expression

- Given - an attribute grammar **G** for arithmetic expressions using multiplication, unary -, and unary +.
- Complete the SDD -
 - Add semantic actions to compute an attribute **E.sign** for non-terminal **E** to record whether the arithmetic value of **E** is positive or negative.
 - The attribute **sign** can have two values, either **POS** or **NEG**.
- Also, show the parse tree for input **2 * - 3**

Production	Semantic Rule
$S \rightarrow E$	<pre>{ if(E.sign == POS) print("Result is positive"); else print("Result is negative"); }</pre>
$E \rightarrow \text{num}$	
$E \rightarrow + E$	
$E \rightarrow - E$	
$E \rightarrow E_1 * E_2$	

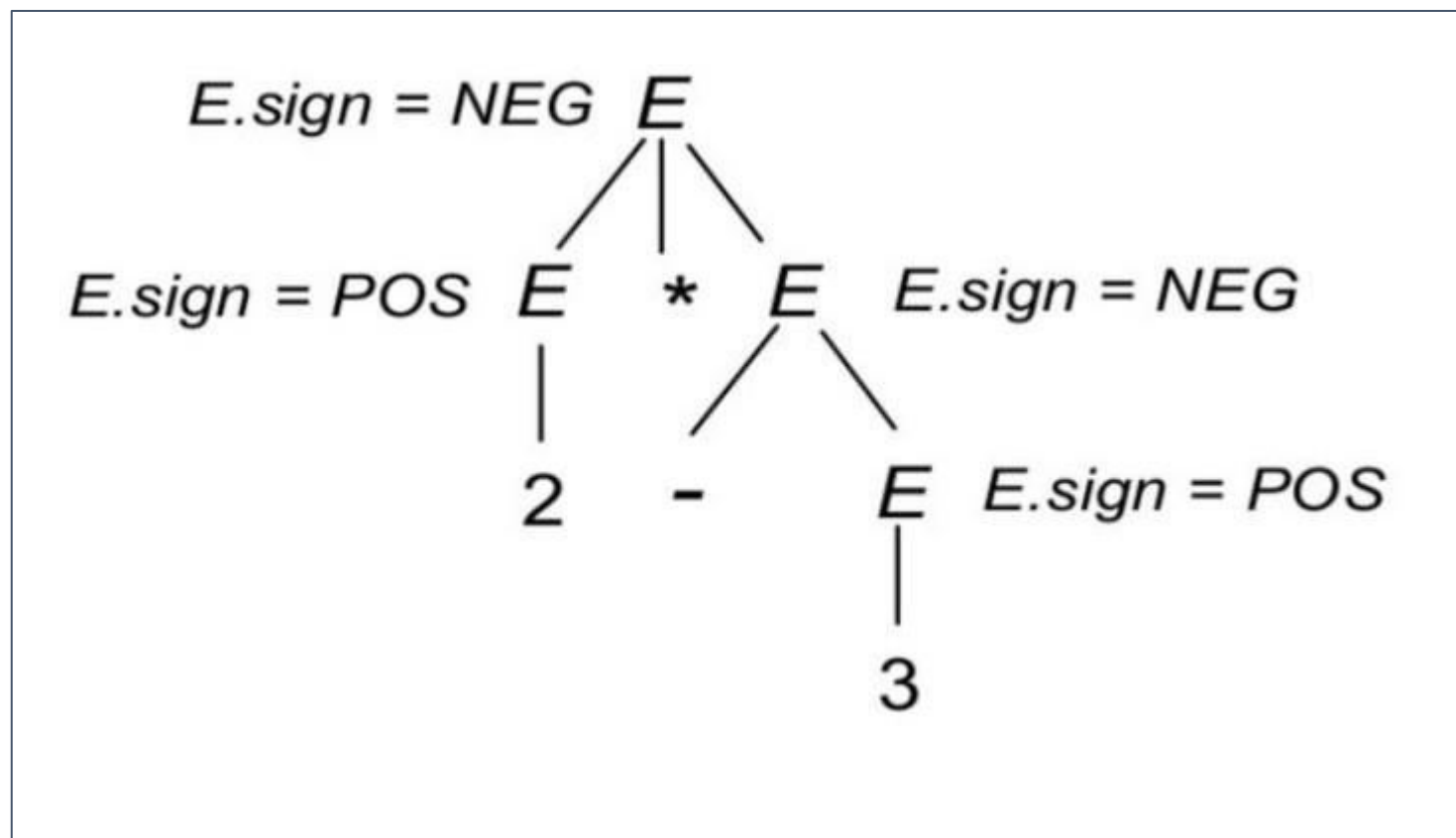
Example 9 - SDD to Identify the sign of the evaluated expression

Production	Semantic Rule
$S \rightarrow E$	<pre>{ if(E.sign == POS) print("Result is positive"); else print("Result is negative"); }</pre>
$E \rightarrow \text{num}$	<pre>{ E.sign == POS; }</pre>
$E \rightarrow + E_1$	<pre>{ E.sign = E₁.sign }</pre>
$E \rightarrow - E_1$	<pre>{ if (E.sign == POS) E.sign = NEG; else E.sign = POS; }</pre>
$E \rightarrow E_1 * E_2$	<pre>{ if (E₁.sign == E₂.sign E.sign = POS; else E.sign = NEG; }</pre>

Example 9 - SDD to Identify the sign of the evaluated expression (cont.)

Show the parse tree for input $2 * - 3$ (where 2 and 3 are “unsigned_ints”). Indicate at each node what the values of associated attributes are.

Solution -





**THANK
YOU**

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu